MYSQL AND SQL
THE COMPLETE PROFESSIONAL COURSE

From PostgreSQL Fluency to MySQL 8.x Mastery

Administration, Indexing, Optimization, JSON, Window Functions, Transactions, Python/pandas Integration, Data Analytics, Data Engineering, Backend Development, AI Engineering, Real-World Projects and Interview Preparation

---

Kajola Gbenga
Data Scientist & AI Engineer

Table of Contents

1. MySQL Fundamentals
2. MySQL Installation and Environment
3. Database and Schema Management
4. Table Creation and Data Types
5. Keys and Constraints
6. CRUD Operations
7. SELECT and Filtering
8. MySQL Functions
9. Aggregation and Analytical SQL
10. Joins
11. Subqueries
12. Common Table Expressions
13. Window Functions
14. Set Operations
15. Views
16. Indexes
17. Query Optimization
18. Storage Engines
19. Transactions and ACID
20. Stored Procedures
21. User-Defined Functions
22. Triggers
23. Events and Automation
24. JSON in MySQL
25. Regular Expressions
26. Temporary Tables and Derived Tables
27. Database Design and Normalization
28. MySQL Users and Security
29. MySQL Administration Concepts
30. Backup and Restore
31. Importing and Exporting Data
32. MySQL + Python
33. MySQL + pandas
34. MySQL for Data Analytics
35. MySQL for Data Science
36. MySQL for Data Engineering

37. MySQL for Backend Development
38. MySQL for AI Engineering
39. MySQL Replication and Scalability
40. MySQL Partitioning
41. Advanced MySQL Analytics
42. MySQL vs PostgreSQL
43. SQL Portability
44. Interview Preparation
45. SQL Coding Challenges
46. Real-World MySQL Projects
47. Final MySQL Capstone
48. Final Cheat Sheet

1. MySQL Fundamentals

1.1 What MySQL Is

MySQL is an open-source relational database management system (RDBMS) originally released in 1995, now owned and developed by Oracle Corporation. It is the world's most widely deployed open-source database, powering everything from small web applications to massive platforms like Facebook, YouTube, and Booking.com. Since you already know PostgreSQL, think of MySQL as a sibling RDBMS: both speak SQL, both are relational, both are open-source and free to use, but they differ meaningfully in architecture, syntax details, feature depth in certain areas, and philosophy, all of which this course highlights explicitly throughout.

1.2 MySQL vs SQL

SQL (Structured Query Language) is a standardized language for querying and managing relational data, defined by ANSI/ISO standards. MySQL is a specific database SYSTEM that implements SQL, along with its own extensions and, in some areas, its own interpretations of ambiguous parts of the standard. Just as you learned PostgreSQL as one implementation of SQL, MySQL is another; the core SELECT/INSERT/UPDATE/DELETE vocabulary you already know largely transfers directly, but specific functions, data types, and certain behaviors differ, exactly as this course will flag throughout.

1.3 MySQL vs PostgreSQL: The Big Picture

Code Line	Explanation
`Philosophy`	PostgreSQL emphasizes strict standards compliance and advanced features (rich data types, extensibility). MySQL historically emphasized speed and simplicity, and has closed most feature gaps by version 8.x, but retains a more pragmatic, sometimes more permissive default behavior.
`Storage architecture`	PostgreSQL has one storage engine built into the core. MySQL supports pluggable storage engines (Part 18); InnoDB is the modern default and what this course focuses on.
`Data types`	PostgreSQL has a richer native type system (arrays, ranges, native JSONB with indexing, custom types). MySQL's type system is comparatively leaner but fully sufficient for the overwhelming majority of applications (Part 4).
`Case sensitivity`	PostgreSQL identifiers are case-insensitive unless quoted, and lowercases unquoted identifiers by convention. MySQL's identifier case sensitivity depends on the underlying operating system's filesystem (case-sensitive on Linux, case-insensitive on Windows/macOS by default), a genuinely important practical difference covered in Part 3.
`Sequences`	PostgreSQL uses explicit SEQUENCE objects and SERIAL/IDENTITY columns. MySQL uses a simpler built-in AUTO_INCREMENT column attribute with no separate sequence object (Part 4 and Part 42).
`Schemas`	In PostgreSQL, a schema is a namespace WITHIN a database, and one database can have many schemas. In MySQL, 'database' and 'schema' are literally synonyms for the same thing, a genuinely important structural difference explained fully in Part 3.

This comparison recurs throughout the entire course; Part 42 provides the complete, exhaustive side-by-side reference.

1.4 MySQL vs SQL Server, Oracle, and SQLite (Brief Orientation)

Code Line	Explanation
`SQL Server`	Microsoft's commercial RDBMS; uses T-SQL (a SQL dialect with procedural extensions similar in spirit to, but syntactically different from, MySQL's own). Common in Windows/.NET-centric enterprises.
`Oracle Database`	A commercial, enterprise-grade RDBMS with its own PL/SQL procedural language; historically dominant in large enterprises, with a reputation for advanced features and high licensing cost.
`SQLite`	A serverless, file-based, embedded database, not a client-server system like MySQL or PostgreSQL at all; ideal for mobile apps, local caches, and lightweight embedded storage, not for concurrent multi-user server workloads.
`Where MySQL fits`	A free, open-source, client-server RDBMS well suited to web applications, analytics workloads, and general-purpose backend storage; alongside PostgreSQL, it is one of the two dominant open-source relational databases in professional use today.

1.5 RDBMS Concepts Recap (Brief, Since You Know PostgreSQL)

A quick reinforcement of concepts that transfer directly: a relational database organizes data into tables (relations), each table has rows (records/tuples) and columns (fields/attributes), keys establish identity and relationships between tables, and SQL is the language used to define structure (DDL), manipulate data (DML), and control access (DCL). Every one of these ideas carries over from PostgreSQL unchanged; what differs is the specific SQL dialect and the engine's internal behavior, which this course focuses on.

1.6 Client-Server Architecture

Like PostgreSQL, MySQL follows a client-server model: the MySQL Server process (mysqld) runs continuously, listening for connections on a network port (default 3306, versus PostgreSQL's default 5432), managing data storage, processing queries, and enforcing security. A client (a command-line tool, an application, a GUI) connects to the server over this port, authenticates, and sends SQL statements for execution.

1.7 MySQL Client Tools

Code Line	Explanation
`MySQL Server (mysqld)`	The actual database engine/daemon process that stores and manages data; this is the 'database' in the everyday sense.
`mysql (command-line client)`	A text-based interactive client for connecting to a server and running SQL directly; the MySQL equivalent of PostgreSQL's psql.
`MySQL Shell (mysqlsh)`	A newer, more powerful scripting-capable client supporting SQL, JavaScript, and Python modes, with advanced administration and cloud features; roughly analogous in spirit to psql but considerably more programmable.
`MySQL Workbench`	The official graphical client: query editor, visual schema/ER-diagram

Code Line	Explanation
	design, administration dashboards, and data export/import tools; the MySQL equivalent of pgAdmin.

1.8 Databases, Tables, Rows, Columns, Keys, Relationships (Recap)

These foundational terms mean exactly what they meant in PostgreSQL: a database is a named collection of related tables; a table is a structured collection of rows sharing the same columns; a row is one record; a column is one named, typed attribute; a primary key uniquely identifies each row; a foreign key references a primary (or unique) key in another table to establish a relationship. The one genuinely different term is 'schema', covered fully in Part 3.

1.9 Which SQL Concepts Are Portable vs MySQL-Specific

Code Line	Explanation
`Portable (ANSI/ISO SQL, transfers directly from PostgreSQL)`	SELECT/FROM/WHERE/GROUP BY/HAVING/ORDER BY, basic JOIN types, INSERT/UPDATE/DELETE, basic data types (INT, VARCHAR, DATE), PRIMARY KEY/FOREIGN KEY/UNIQUE/NOT NULL constraints, basic window functions (OVER, PARTITION BY), CTEs (WITH), UNION/UNION ALL, basic subqueries and EXISTS.
`MySQL-specific (differs from or has no PostgreSQL equivalent)`	AUTO_INCREMENT (vs SERIAL/IDENTITY), storage engines (InnoDB/MyISAM), ON DUPLICATE KEY UPDATE, the LIMIT/OFFSET short-hand LIMIT n, m, backtick-quoted identifiers, several string/date function names (Part 8), the Event Scheduler, and MySQL's specific JSON function set (Part 24).
How This Course Marks Differences: Throughout this course, every MySQL-specific feature or syntax difference is explicitly flagged with a 'PostgreSQL Comparison' callout, so you always know whether what you are learning is portable knowledge or MySQL-specific.	

2. MySQL Installation and Environment

2.1 Installing on Windows

- Download the MySQL Installer for Windows from dev.mysql.com/downloads/installer/.
- Run the installer and choose the 'Developer Default' setup type, which installs MySQL Server, MySQL Workbench, MySQL Shell, and sample data.
- During setup, configure the server as a 'Standalone MySQL Server', set the port (default 3306), and set a strong root password when prompted.
- Complete the wizard; the installer registers MySQL as a Windows service, so it starts automatically on boot.
- Verify installation by opening 'MySQL Command Line Client' from the Start menu and logging in with the root password.

2.2 Installing on macOS

```
# Using Homebrew (recommended)
brew update
brew install mysql
brew services start mysql

# Secure the installation (sets root password, removes test databases)
mysql_secure_installation
```

Homebrew installs the MySQL Server binaries and manages the service lifecycle via brew services; mysql_secure_installation runs an interactive script to set a root password and lock down default, insecure settings, closely mirroring the security hardening step you would perform after a fresh PostgreSQL install.

2.3 Installing on Linux (Ubuntu/Debian Example)

```
sudo apt update
sudo apt install mysql-server
sudo systemctl start mysql
sudo systemctl enable mysql
sudo mysql_secure_installation
```

Code Line	Explanation
`sudo apt install mysql-server`	Installs the MySQL Server package and its default configuration from the distribution's package repository.
`sudo systemctl start mysql / enable mysql`	Starts the MySQL service immediately, and enables it to start automatically on every future boot, exactly the same systemd pattern used for PostgreSQL.
`sudo mysql_secure_installation`	Interactive hardening script: sets/validates a strong root password, removes anonymous users, disables remote root login, and removes the test database, all sensible defaults to apply immediately after installation.

2.4 Installing MySQL Workbench and MySQL Shell

MySQL Workbench and MySQL Shell can each be installed separately from dev.mysql.com/downloads/ on any platform, or bundled automatically via the Windows installer's Developer Default option. On macOS/Linux, both are also available via Homebrew (brew install --cask mysql-workbench) or the distribution's package manager (apt install mysql-shell).

2.5 Connecting to the Server

```
mysql -u root -p
mysql -u root -p -h 127.0.0.1 -P 3306
```

Code Line	Explanation
`-u root`	Specifies the username to connect as; root is the default administrative superuser, directly analogous to PostgreSQL's postgres superuser role.
`-p`	Prompts for a password interactively rather than exposing it in the command line (a safer habit than -pYourPassword, which leaves the password visible in shell history).
`-h 127.0.0.1`	Specifies the host to connect to; omitted, it defaults to localhost via a Unix socket on Linux/macOS.
`-P 3306`	Specifies the port; MySQL's default is 3306, distinct from PostgreSQL's default of 5432, worth remembering if both are ever installed on the same machine.
PostgreSQL Comparison: MySQL uses an uppercase -P for the port flag and a lowercase -p for password; PostgreSQL's psql uses a lowercase -p for port instead. This is a small but genuinely easy-to-mistype difference for anyone coming from psql.	

2.6 Configuration: Port, Username, Password

The server's configuration file (my.cnf on Linux/macOS, my.ini on Windows) controls settings like the listening port, data directory location, and buffer sizes (Part 29 covers server configuration and architecture in depth). Changing the port requires editing this file and restarting the service; the default port and typical settings are almost always fine for learning and development.

2.7 GUI vs Command-Line Workflows

Code Line	Explanation
`Command-line client (mysql)`	Best for scripting, automation, quick queries, and situations without graphical access (like a remote server via SSH); every example in this course is written to run correctly here.
`MySQL Workbench`	Best for visual schema design, browsing data interactively, building ER diagrams (Part 27), and exploring query results in a spreadsheet-like grid; recommended for the database design and analytics exercises throughout this course.
`MySQL Shell`	Best for administration scripting and situations needing JavaScript/Python scripting alongside SQL, such as bulk administrative tasks or working with MySQL's newer X DevAPI document-store features (outside this course's relational-focused

Code Line	Explanation
	scope).

3. Database and Schema Management

3.1 MySQL's Critical Terminology Difference: Database = Schema

This is one of the single most important structural differences to internalize coming from PostgreSQL. In PostgreSQL, a SERVER hosts multiple DATABASES, and each database can contain multiple SCHEMAS (namespaces), each of which can hold tables; the fully qualified path is database.schema.table. In MySQL, there is no separate schema layer: CREATE DATABASE and CREATE SCHEMA are literally two spellings of the exact same command, and a table's fully qualified path is simply database_name.table_name.

> **Critical PostgreSQL Comparison:** If you write CREATE SCHEMA my_schema; in MySQL, you have created a full MySQL database, not a namespace inside an existing one, the way you would in PostgreSQL. There is no MySQL equivalent of 'multiple schemas inside one database' the way PostgreSQL supports it; the closest MySQL analog to using multiple PostgreSQL schemas for organization is simply using multiple MySQL databases, or a consistent table-naming convention (like sales_orders, sales_customers) within one database.

3.2 CREATE DATABASE

```
CREATE DATABASE company_db;
CREATE DATABASE IF NOT EXISTS company_db;
CREATE DATABASE company_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Code Line	Explanation
`CREATE DATABASE company_db;`	Creates a new, empty database; this is portable, standard SQL syntax that PostgreSQL also supports.
`IF NOT EXISTS`	Prevents an error if a database with this name already exists; the statement simply does nothing instead of failing, useful in idempotent setup scripts.
`CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci`	Explicitly sets the character encoding and collation (sorting/comparison rules) for the database; utf8mb4 is the modern, correct choice for full Unicode support (including emoji), and is MySQL's true UTF-8 implementation, unlike its legacy, now-discouraged utf8 alias which only supports a 3-byte subset of Unicode.

> **MySQL-Specific Gotcha:** Always use utf8mb4, never the legacy utf8 alias, for any new MySQL database. utf8mb4 is a MySQL-specific naming quirk; PostgreSQL's UTF8 encoding is already the full standard, with no equivalent 3-byte-vs-4-byte distinction to worry about.

3.3 DROP DATABASE

```
DROP DATABASE company_db;
DROP DATABASE IF EXISTS company_db;
```

Permanently deletes a database and every table, view, and object inside it, with no confirmation prompt and no built-in undo; exactly as dangerous as PostgreSQL's DROP DATABASE, and just as portable in syntax.

3.4 USE

```
USE company_db;
```

Code Line	Explanation
`USE company_db;`	Sets company_db as the active/default database for the current session, so subsequent unqualified table references (SELECT * FROM employees) resolve within it. This is MySQL-specific syntax; PostgreSQL has no direct USE equivalent, since a psql/client connection is tied to one specific database for its entire session, and switching requires a brand new connection (\c dbname in psql).
PostgreSQL Comparison: This is a genuinely convenient MySQL-specific feature: a single MySQL client connection can freely switch between databases with USE at any time. In PostgreSQL, switching databases mid-session is not possible at all; you must reconnect entirely. This reflects the deeper architectural difference from section 3.1: MySQL databases are more like lightweight, freely switchable namespaces within one server connection.	

3.5 SHOW DATABASES and SHOW TABLES

```
SHOW DATABASES;
USE company_db;
SHOW TABLES;
SHOW TABLES FROM company_db;
```

Code Line	Explanation
`SHOW DATABASES;`	Lists every database on the server; this is MySQL-specific syntax. The PostgreSQL equivalent is \l inside psql, or querying pg_catalog.pg_database directly.
`SHOW TABLES;`	Lists every table in the currently active database (set via USE); the PostgreSQL equivalent is \dt inside psql.
`SHOW TABLES FROM company_db;`	Lists tables in a specific named database without first switching to it with USE, a convenient MySQL-specific shortcut.

MySQL's SHOW command family (also SHOW COLUMNS, SHOW CREATE TABLE, SHOW INDEX, and many more used throughout this course) is a broad, MySQL-specific introspection mechanism; PostgreSQL instead relies on psql's backslash meta-commands and direct queries against the information_schema and pg_catalog system catalogs. MySQL also supports information_schema (it is part of the SQL standard), so INFORMATION_SCHEMA.TABLES works in both databases and is the more portable choice when writing cross-database tooling.

3.6 A Practical, Portable Alternative: information_schema

```
SELECT table_name, table_rows, data_length
FROM information_schema.tables
WHERE table_schema = 'company_db';
```

Since information_schema is part of the ANSI SQL standard, this exact style of query (with minor column-name differences) works in both MySQL and PostgreSQL, making it a genuinely portable way to introspect a database's structure, as opposed to MySQL's own SHOW commands or psql's own backslash commands, neither of which transfers to the other system.

4. Table Creation and Data Types

4.1 CREATE TABLE

```
CREATE TABLE employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2),
    hire_date DATE DEFAULT (CURRENT_DATE),
    department_id INT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Code Line	Explanation
`AUTO_INCREMENT`	MySQL's mechanism for an automatically generated, incrementing integer, most commonly used for primary keys. Portable-syntax-wise this is MySQL-specific; PostgreSQL's equivalent is SERIAL (a shorthand that creates a hidden sequence) or, in modern PostgreSQL, GENERATED ALWAYS AS IDENTITY. MySQL has no separate sequence object at all; AUTO_INCREMENT is simply a column attribute (fully explored in Part 42).
`ENGINE=InnoDB`	Explicitly specifies the storage engine (Part 18); InnoDB is the modern default in MySQL 8.x and should almost always be used for transactional application tables. This concept has no PostgreSQL equivalent at all, since PostgreSQL has exactly one built-in storage engine.
`DEFAULT CHARSET=utf8mb4`	Sets the table's default character set, overridable per-column; as covered in Part 3.2, always use utf8mb4 for full Unicode support.

4.2 ALTER TABLE, DROP TABLE, TRUNCATE TABLE

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100);
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2);
ALTER TABLE employees CHANGE COLUMN email work_email VARCHAR(100);
ALTER TABLE employees DROP COLUMN work_email;
DROP TABLE employees;
TRUNCATE TABLE employees;
```

Code Line	Explanation
`MODIFY COLUMN`	Changes a column's type/attributes while keeping its name; MySQL-specific syntax. PostgreSQL instead uses ALTER TABLE ... ALTER COLUMN ... TYPE ..., a meaningfully different keyword.
`CHANGE COLUMN old_name new_name type`	MySQL-specific syntax that can rename AND retype a column in one statement; PostgreSQL requires a separate RENAME COLUMN statement instead.
`TRUNCATE TABLE`	Removes every row instantly and resets AUTO_INCREMENT back to its starting value, but (like PostgreSQL's TRUNCATE) cannot be rolled back inside some contexts and bypasses row-level triggers in MySQL entirely, unlike DELETE.

4.3 Numeric Data Types

Code Line	Explanation
`TINYINT`	1 byte; range -128 to 127 (or 0-255 UNSIGNED). Ideal for small flags/counters. No direct PostgreSQL equivalent this small; PostgreSQL's smallest integer is SMALLINT (2 bytes).
`SMALLINT`	2 bytes; -32,768 to 32,767. Matches PostgreSQL's SMALLINT directly.
`MEDIUMINT`	3 bytes; approx -8 million to 8 million. MySQL-specific; PostgreSQL has no 3-byte integer type at all.
`INT / INTEGER`	4 bytes; approx -2.1 billion to 2.1 billion. Matches PostgreSQL's INTEGER directly.
`BIGINT`	8 bytes; matches PostgreSQL's BIGINT directly.
`DECIMAL(p,s) / NUMERIC(p,s)`	Exact, fixed-point storage; identical concept and syntax to PostgreSQL's NUMERIC. Always use for money and any value where floating-point rounding error is unacceptable.
`FLOAT / DOUBLE`	Approximate, floating-point storage; matches PostgreSQL's REAL/DOUBLE PRECISION in concept. Never use for currency.

PostgreSQL Comparison: MySQL offers more granular small-integer types (TINYINT, MEDIUMINT) than PostgreSQL, which only offers SMALLINT/INTEGER/BIGINT. Choosing the smallest sufficient type in MySQL genuinely saves storage at scale; this level of granularity has no direct PostgreSQL parallel.

4.4 Character Data Types

Code Line	Explanation
`CHAR(n)`	Fixed-length, space-padded string; identical concept to PostgreSQL's CHAR(n).
`VARCHAR(n)`	Variable-length string up to n characters; identical concept to PostgreSQL's VARCHAR(n), though MySQL enforces a maximum row size that affects how many/how large VARCHAR columns a single table can practically have (InnoDB row limit around 65,535 bytes).
`TEXT`	Variable-length text up to ~65,535 bytes. PostgreSQL's TEXT type has no fixed length limit at all, a genuine difference; for very large text, MySQL requires stepping up to MEDIUMTEXT or LONGTEXT explicitly.
`TINYTEXT / MEDIUMTEXT / LONGTEXT`	MySQL-specific graduated text sizes: TINYTEXT (~255 bytes), MEDIUMTEXT (~16MB), LONGTEXT (~4GB). PostgreSQL has no equivalent family; its single TEXT type already covers this entire range and more.

4.5 Date and Time Data Types

Code Line	Explanation
`DATE`	Calendar date only (YYYY-MM-DD); matches PostgreSQL's DATE.
`DATETIME`	Date and time with no timezone awareness, range 1000-9999. MySQL-specific name; PostgreSQL's closest equivalent is TIMESTAMP (without time zone).
`TIMESTAMP`	Date and time, but automatically converted to/from UTC and limited to the range 1970-2038 (the classic Unix epoch limit); genuinely

Code Line	Explanation
	different semantics from PostgreSQL's TIMESTAMP, which has no such conversion behavior or narrow range by default. In MySQL, DATETIME is generally the safer, more predictable choice for general-purpose timestamps, while TIMESTAMP is well suited specifically for 'row last modified' style auditing columns due to its auto-update capability (Part 22).
`TIME`	Time of day or a duration; matches PostgreSQL's TIME.
`YEAR`	A single-byte year value (1901-2155); entirely MySQL-specific with no PostgreSQL equivalent at all.

Critical PostgreSQL Comparison: This is one of the most consequential differences for anyone coming from PostgreSQL: MySQL's TIMESTAMP silently converts values to and from UTC based on the session's time_zone setting, and is limited to 2038. PostgreSQL's TIMESTAMP never does this conversion (TIMESTAMPTZ is the type that does). When in doubt in MySQL, use DATETIME for general date/time storage and reserve TIMESTAMP specifically for auto-updating 'last modified' columns.

4.6 Other Data Types

Code Line	Explanation
`BOOLEAN / BOOL`	MySQL implements this as a synonym for TINYINT(1); there is no true native boolean storage type, and values are just 0/1 under the hood. PostgreSQL, by contrast, has a genuine native BOOLEAN type with true/false/null semantics enforced at the storage level.
`JSON`	Native JSON storage with validation and functions (fully covered in Part 24); comparable in spirit to PostgreSQL's JSONB, though with some functional and indexing differences explained there.
`BINARY(n) / VARBINARY(n)`	Fixed/variable-length raw binary data; matches PostgreSQL's BYTEA in purpose, though PostgreSQL uses a single flexible type rather than this fixed/variable pair.
`ENUM('a','b','c')`	A string column restricted to a predefined list of allowed values, stored internally as a compact integer. Entirely MySQL-specific; PostgreSQL's closest equivalent is a genuine CREATE TYPE ... AS ENUM, a separate, reusable named type rather than an inline column attribute.
`SET('a','b','c')`	A column that can hold zero or more values from a predefined list, stored as a bitmask. Entirely MySQL-specific with no PostgreSQL equivalent; a many-valued tag-like column would instead be modeled with a proper array type or a separate join table in PostgreSQL.
`Spatial types (GEOMETRY, POINT, POLYGON, etc.)`	MySQL's built-in spatial data support, roughly comparable in purpose to PostgreSQL's PostGIS extension, though PostGIS is considerably more feature-complete for advanced GIS work.

4.7 Common Data Type Mistakes

- Using FLOAT/DOUBLE for money instead of DECIMAL, introducing rounding errors in financial calculations.
- Using the legacy utf8 charset alias instead of utf8mb4, silently breaking storage of emoji and many non-Latin characters.
- Choosing TIMESTAMP for general-purpose date storage without realizing its UTC conversion behavior and 2038 range limit.

- Over-sizing VARCHAR/INT columns 'just in case' rather than sizing to the genuine expected range, which wastes storage at scale and can reduce how many columns fit within InnoDB's row-size limits.
- Using ENUM for a value set that is likely to change over time; every addition requires an ALTER TABLE, unlike a simple lookup table with a foreign key.

5. Keys and Constraints

5.1 Primary Keys

```
CREATE TABLE departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE
);
```

Identical concept to PostgreSQL: a primary key uniquely identifies each row and cannot be NULL. The syntax is portable; only AUTO_INCREMENT (versus PostgreSQL's SERIAL/IDENTITY) differs, as already covered in Part 4.1.

5.2 Foreign Keys and Referential Integrity

```
CREATE TABLE employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Code Line	Explanation
`FOREIGN KEY ... REFERENCES ...`	Portable, standard SQL syntax, identical to PostgreSQL.
`ENGINE=InnoDB required`	This is a critical MySQL-specific caveat: foreign keys are ONLY enforced by the InnoDB storage engine. The older MyISAM engine (Part 18) silently accepts foreign key syntax but never actually enforces it. PostgreSQL has no equivalent caveat, since it has only one storage engine and foreign keys are always enforced.

5.3 ON DELETE / ON UPDATE Behaviors

Code Line	Explanation
`CASCADE`	Automatically deletes (or updates) matching child rows when the parent row is deleted (or its key updated); identical behavior and syntax to PostgreSQL.
`SET NULL`	Sets the foreign key column to NULL in child rows when the parent is deleted; the referencing column must itself be nullable. Identical to PostgreSQL.
`RESTRICT`	Prevents the delete/update if matching child rows exist, raising an error; this is MySQL's (and standard SQL's) default behavior if no clause is specified, identical to PostgreSQL's default.
`NO ACTION`	In MySQL (InnoDB), functionally equivalent to RESTRICT in practice; PostgreSQL distinguishes NO ACTION as deferrable-checkable at the end of a transaction, a subtlety MySQL does not implement the same way.

5.4 Unique Constraints, NOT NULL, DEFAULT

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'active',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

All portable, standard syntax identical to PostgreSQL. DEFAULT CURRENT_TIMESTAMP is one of the few places MySQL allows a function call directly as a column default without extra parentheses, unlike some other expressions which require DEFAULT (expression) syntax in MySQL 8.x.

5.5 CHECK Constraints

```
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    price DECIMAL(10,2) CHECK (price >= 0),
    stock_qty INT CHECK (stock_qty >= 0)
);
```

Important Version Note: CHECK constraints were silently PARSED but NOT ENFORCED in MySQL versions before 8.0.16. As of MySQL 8.0.16 and later, CHECK constraints are fully enforced, matching PostgreSQL's long-standing behavior. If working with an older MySQL version, be aware CHECK may appear to succeed but do nothing at all.

5.6 Composite Keys

```
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

A composite (multi-column) primary key, common in many-to-many join tables; fully portable syntax identical to PostgreSQL. Column order in a composite key matters for indexing purposes (Part 16.6).

6. CRUD Operations

6.1 INSERT: Single and Multi-Row

```
INSERT INTO departments (department_name) VALUES ('Engineering');

INSERT INTO departments (department_name) VALUES
    ('Sales'),
    ('Marketing'),
    ('Finance');
```

Fully portable syntax, identical to PostgreSQL. Multi-row INSERT with several VALUES tuples in one statement is significantly faster than separate INSERT statements in both databases, since it reduces round trips and (in MySQL specifically) can batch InnoDB's transaction log writes.

6.2 SELECT, UPDATE, DELETE (Recap)

```
SELECT * FROM employees WHERE department_id = 1;
UPDATE employees SET salary = salary * 1.05 WHERE department_id = 1;
DELETE FROM employees WHERE employee_id = 42;
```

All fully portable and identical in syntax and behavior to PostgreSQL. As in PostgreSQL, always run UPDATE/DELETE with a WHERE clause deliberately reviewed first (often as a SELECT with the same WHERE clause) to avoid accidentally affecting every row in the table.

6.3 INSERT ... SELECT

```
INSERT INTO archived_orders (order_id, customer_id, order_date, total)
SELECT order_id, customer_id, order_date, total
FROM orders
WHERE order_date < '2024-01-01';
```

Copies the result of a SELECT directly into another table in one statement; fully portable syntax identical to PostgreSQL, and a common pattern for archiving, staging (Part 36), and denormalized reporting tables.

6.4 REPLACE (MySQL-Specific)

```
REPLACE INTO products (product_id, name, price) VALUES (101, 'Widget', 9.99);
```

Code Line	Explanation
`Behavior`	If a row with the same primary/unique key already exists, REPLACE deletes it entirely and inserts a brand new row with the given values; if no conflicting row exists, it behaves exactly like a plain INSERT.
`Critical caveat`	Because REPLACE performs an actual DELETE + INSERT, it fires DELETE and INSERT triggers (Part 22), resets AUTO_INCREMENT-independent columns not explicitly listed back to their defaults, and can cascade an ON DELETE CASCADE on any child tables, none of which happens with the ON DUPLICATE KEY UPDATE pattern below.

Code Line	Explanation
PostgreSQL Comparison: REPLACE has NO direct equivalent in standard SQL or in PostgreSQL. PostgreSQL's closest concept is INSERT ... ON CONFLICT DO UPDATE, which behaves much more like MySQL's INSERT ... ON DUPLICATE KEY UPDATE below (an in-place update) rather than REPLACE's destructive delete-then-insert. Prefer ON DUPLICATE KEY UPDATE over REPLACE in almost all real applications.	

6.5 INSERT ... ON DUPLICATE KEY UPDATE (MySQL-Specific)

```
INSERT INTO products (product_id, name, price, stock_qty)
VALUES (101, 'Widget', 9.99, 50)
ON DUPLICATE KEY UPDATE
    price = VALUES(price),
    stock_qty = stock_qty + VALUES(stock_qty);
```

Code Line	Explanation
`Behavior`	Attempts a normal INSERT; if it would violate a primary key or unique constraint, MySQL instead performs the specified UPDATE on the existing conflicting row, entirely in place, with no delete involved.
`VALUES(column)`	Inside the UPDATE clause, VALUES(price) refers to the value that WOULD have been inserted for that column, letting the update reference the attempted new values directly.
`stock_qty = stock_qty + VALUES(stock_qty)`	A genuinely common upsert pattern: if the product already exists, ADD the new stock quantity to the existing quantity rather than overwriting it, while price is simply overwritten.
PostgreSQL Comparison: This is the direct, safer MySQL equivalent of PostgreSQL's INSERT INTO ... ON CONFLICT (column) DO UPDATE SET .... The core upsert (update-or-insert) concept is portable, but the exact syntax, VALUES() vs PostgreSQL's EXCLUDED pseudo-table, differs completely between the two systems.	

6.6 Safe Deletion Practices

- Always run the intended WHERE clause as a SELECT COUNT(*) or SELECT * first, to confirm exactly which rows will be affected before running the DELETE or UPDATE.
- Wrap risky multi-statement operations in an explicit transaction (Part 19) so a mistake can be rolled back before committing.
- Consider enabling MySQL Workbench's or the client's 'safe updates' mode (sql_safe_updates) during ad hoc exploration, which blocks UPDATE/DELETE statements that lack a WHERE clause referencing a key column.

7. SELECT and Filtering

7.1 SELECT, Aliases, Calculated Columns

```
SELECT
    first_name,
    last_name,
    salary,
    salary * 12 AS annual_salary,
    CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

Fully portable syntax: AS for aliases, and calculated/expression columns, work identically to PostgreSQL. CONCAT() itself is covered fully in Part 8; note MySQL's CONCAT() silently returns NULL if ANY argument is NULL, a behavior difference from PostgreSQL's || operator (which treats NULL as empty string in some contexts) explored further in Part 8.1.

7.2 DISTINCT, WHERE, ORDER BY, LIMIT, OFFSET

```
SELECT DISTINCT department_id FROM employees;

SELECT * FROM employees
WHERE salary > 50000
ORDER BY salary DESC
LIMIT 10;

SELECT * FROM employees
ORDER BY salary DESC
LIMIT 10 OFFSET 20;

SELECT * FROM employees ORDER BY salary DESC LIMIT 20, 10;
```

Code Line	Explanation
`LIMIT 10 OFFSET 20`	Portable syntax, identical to PostgreSQL: skip 20 rows, then return the next 10.
`LIMIT 20, 10`	A MySQL-specific shorthand for the exact same thing: LIMIT offset, row_count. This comma form does NOT exist in PostgreSQL at all; PostgreSQL only accepts the explicit LIMIT/OFFSET keyword form. Prefer the explicit LIMIT n OFFSET m form for portability, even in MySQL.

7.3 Comparison and Logical Operators

```
SELECT * FROM products WHERE price BETWEEN 10 AND 50;
SELECT * FROM products WHERE category IN ('Electronics', 'Books');
SELECT * FROM products WHERE NOT (category = 'Clearance');
SELECT * FROM customers WHERE name LIKE 'A%';
SELECT * FROM customers WHERE name REGEXP '^A.*son$';
```

Code Line	Explanation
`BETWEEN, IN, NOT, LIKE`	All fully portable, identical syntax and behavior to PostgreSQL.
`REGEXP`	MySQL's pattern-matching operator using regular expressions (implemented via the ICU regex engine as of MySQL 8.0.4+). PostgreSQL's equivalent operator is ~ (case-sensitive) or ~* (case-insensitive); MySQL uses the word REGEXP (or its synonym RLIKE) instead of a symbolic operator. Full REGEXP function coverage is in Part 25.

7.4 MySQL's Handling of NULL

```
SELECT * FROM employees WHERE department_id IS NULL;
SELECT * FROM employees WHERE department_id IS NOT NULL;
SELECT * FROM employees WHERE department_id = NULL;   -- always returns ZERO rows
```

Code Line	Explanation
`IS NULL / IS NOT NULL`	The only correct way to test for NULL, identical to PostgreSQL; NULL represents 'unknown', so no equality comparison (including = NULL) ever evaluates to true.
`department_id = NULL`	A classic, portable SQL trap present identically in both databases: this always returns zero rows, silently, with no error, because comparing anything to an unknown value produces an unknown (not true) result.

MySQL's three-valued logic (TRUE/FALSE/UNKNOWN) and NULL-sorting behavior (NULLs sort first in ascending order by default, exactly like PostgreSQL) are fully portable and require no relearning.

7.5 The <=> NULL-Safe Equality Operator (MySQL-Specific)

```
SELECT * FROM employees WHERE department_id <=> NULL;
```

PostgreSQL Comparison: <=> is a MySQL-specific 'NULL-safe equal' operator: unlike =, it treats NULL <=> NULL as TRUE, letting a single comparison correctly match NULL values without a separate IS NULL check. PostgreSQL's equivalent is the standard SQL construct IS NOT DISTINCT FROM (also portable to MySQL 8.x, which supports it too). <=> is convenient but MySQL-only; IS [NOT] DISTINCT FROM is the more portable choice.

8. MySQL Functions

This part covers MySQL's built-in function library, flagging every meaningful naming or behavioral difference from PostgreSQL.

8.1 String Functions

Code Line	Explanation
`CONCAT(a, b, ...)`	Joins strings; identical name to PostgreSQL's CONCAT(). Unlike PostgreSQL's || operator, MySQL's CONCAT() returns NULL if ANY argument is NULL (PostgreSQL's CONCAT() function, notably, treats NULL as empty string, a genuine behavioral difference even though both are named CONCAT).
`CONCAT_WS(sep, a, b, ...)`	Concatenates with a separator, automatically skipping NULL arguments; identical name and behavior to PostgreSQL's CONCAT_WS().
`LOWER() / UPPER()`	Identical to PostgreSQL.
`LENGTH()`	Returns the string length in BYTES. For multi-byte UTF-8 text, this differs from the character count; MySQL-specific gotcha (PostgreSQL's LENGTH() returns character count directly).
`CHAR_LENGTH()`	Returns the character count, correctly handling multi-byte UTF-8; this is the function to use for genuine 'how many characters' logic, matching PostgreSQL's LENGTH() behavior.
`TRIM() / LTRIM() / RTRIM()`	Identical to PostgreSQL.
`SUBSTRING(str, pos, len)`	Identical concept to PostgreSQL's SUBSTRING(); MySQL also supports the shorthand SUBSTR().
`LEFT(str, n) / RIGHT(str, n)`	Return the leftmost/rightmost n characters. MySQL-specific named functions; PostgreSQL requires composing SUBSTRING() with a calculated position/length instead, since it has no direct LEFT()/RIGHT().
`REPLACE(str, from, to)`	Replaces all occurrences of a substring; identical name and behavior to PostgreSQL's REPLACE().

8.2 Numeric Functions

Code Line	Explanation
`ROUND(n, d)`	Identical to PostgreSQL.
`CEIL() / CEILING()`	Identical to PostgreSQL's CEIL().
`FLOOR()`	Identical to PostgreSQL.
`ABS()`	Identical to PostgreSQL.
`MOD(n, m)`	Identical to PostgreSQL's MOD(); the % operator also works identically in both.
`POWER(x, y)`	Identical to PostgreSQL.

8.3 Date/Time Functions

Code Line	Explanation
`NOW()`	Current date and time; identical name to PostgreSQL's NOW().
`CURDATE()`	Current date only. MySQL-specific name; PostgreSQL's equivalent is CURRENT_DATE.

Code Line	Explanation
`CURTIME()`	Current time only. MySQL-specific name; PostgreSQL's equivalent is CURRENT_TIME.
`DATE(expr)`	Extracts just the date part from a DATETIME/TIMESTAMP; identical concept, matching PostgreSQL's ::date cast or DATE() function.
`YEAR() / MONTH() / DAY()`	Extract the respective component as an integer. MySQL-specific named functions; PostgreSQL's equivalent is EXTRACT(YEAR FROM expr) or DATE_PART('year', expr).
`DATEDIFF(d1, d2)`	Returns the number of days between two dates (d1 - d2). MySQL-specific signature; PostgreSQL allows direct date subtraction (date1 - date2) to get an integer number of days.
`TIMESTAMPDIFF(unit, d1, d2)`	Returns the difference between two datetimes in the specified unit (SECOND, MINUTE, HOUR, DAY, MONTH, YEAR). Entirely MySQL-specific; PostgreSQL uses AGE() or manual interval arithmetic instead.
`DATE_ADD(date, INTERVAL n unit) / DATE_SUB(...)`	Add/subtract an interval from a date. MySQL-specific function-call syntax; PostgreSQL uses direct interval arithmetic instead: date + INTERVAL '1 day'.
`DATE_FORMAT(date, format)`	Formats a date/time as a string using MySQL's own format-specifier codes (%Y, %m, %d, and so on). Entirely MySQL-specific syntax; PostgreSQL's equivalent is TO_CHAR(date, 'YYYY-MM-DD'), using a completely different format-code vocabulary.

```
SELECT DATE_ADD(CURDATE(), INTERVAL 30 DAY) AS due_date;
SELECT TIMESTAMPDIFF(YEAR, '1990-05-15', CURDATE()) AS age;
SELECT DATE_FORMAT(NOW(), '%Y-%m-%d %H:%i:%s') AS formatted;
```

Key Portability Warning: Date/time functions are the single area with the most naming and syntax differences between MySQL and PostgreSQL. When porting SQL from PostgreSQL, expect to rewrite virtually every date function call: EXTRACT/AGE/TO_CHAR/interval-arithmetic in PostgreSQL generally become YEAR()/MONTH()/TIMESTAMPDIFF()/DATE_FORMAT()/DATE_ADD() in MySQL.

8.4 Conditional Functions

Code Line	Explanation
`CASE WHEN ... THEN ... ELSE ... END`	Fully portable standard SQL, identical to PostgreSQL.
`IF(condition, true_val, false_val)`	A compact single-condition conditional. Entirely MySQL-specific; PostgreSQL has no IF() function used as an expression this way (PL/pgSQL has an IF statement, but that is procedural, not an inline expression).
`IFNULL(expr, replacement)`	Returns replacement if expr is NULL. MySQL-specific name; PostgreSQL's equivalent is COALESCE(expr, replacement), which also works in MySQL (COALESCE is portable standard SQL, present in both).
`NULLIF(a, b)`	Returns NULL if a equals b, otherwise returns a; identical to PostgreSQL.
`COALESCE(a, b, c, ...)`	Returns the first non-NULL argument; fully portable standard SQL, identical to PostgreSQL.

```
SELECT name, IF(stock_qty > 0, 'In Stock', 'Out of Stock') AS availability FROM
products;
```

```
SELECT name, IFNULL(discount, 0) AS discount FROM products;
```

8.5 Type Conversion: CAST and CONVERT

```
SELECT CAST('123' AS UNSIGNED);
SELECT CAST(salary AS CHAR);
SELECT CONVERT('2024-01-15', DATE);
```

CAST(expr AS type) is portable standard SQL, identical to PostgreSQL. CONVERT(expr, type) is a MySQL-specific alternative syntax with the same purpose (PostgreSQL's CONVERT() function instead performs character-set conversion, a completely different purpose, so this is a genuine naming collision to be careful of when moving code between the two systems).

9. Aggregation and Analytical SQL

9.1 COUNT, SUM, AVG, MIN, MAX (Recap)

```
SELECT
    COUNT(*) AS total_orders,
    SUM(total) AS total_revenue,
    AVG(total) AS avg_order_value,
    MIN(total) AS smallest_order,
    MAX(total) AS largest_order
FROM orders;
```

Fully portable, identical syntax and behavior to PostgreSQL, including COUNT(*) counting all rows vs COUNT(column) counting only non-NULL values in that column.

9.2 GROUP BY and HAVING

```
SELECT
    region,
    COUNT(*) AS order_count,
    SUM(total) AS region_revenue
FROM orders
GROUP BY region
HAVING SUM(total) > 100000
ORDER BY region_revenue DESC;
```

Code Line	Explanation
`GROUP BY region`	Collapses rows into one group per distinct region value; fully portable.
`HAVING SUM(total) > 100000`	Filters GROUPS after aggregation (unlike WHERE, which filters individual rows before aggregation); fully portable, identical to PostgreSQL.

PostgreSQL Comparison: MySQL, by default, is historically more lenient than PostgreSQL about columns in the SELECT list that are neither aggregated nor in the GROUP BY clause (it would pick an arbitrary value from the group). As of MySQL 5.7+ with the default ONLY_FULL_GROUP_BY SQL mode enabled, MySQL now enforces the same strict standard behavior as PostgreSQL: every non-aggregated selected column must appear in GROUP BY. Older MySQL configurations or explicitly disabled ONLY_FULL_GROUP_BY can still allow the looser, riskier behavior; always assume the strict standard rule applies unless you have specifically verified otherwise.

9.3 Realistic Business Problem: Sales by Region and Product

```
SELECT
    p.category,
    o.region,
    COUNT(DISTINCT o.order_id) AS num_orders,
    SUM(oi.quantity * oi.unit_price) AS revenue,
    ROUND(AVG(oi.quantity * oi.unit_price), 2) AS avg_line_value
FROM orders o
```

```
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category, o.region
HAVING COUNT(DISTINCT o.order_id) >= 5
ORDER BY revenue DESC;
```

A realistic multi-table aggregation joining orders, order line items, and products to compute per-category, per-region revenue, filtered to only combinations with meaningful order volume; every clause here is fully portable SQL that would run unchanged in PostgreSQL, demonstrating how much of the analytical query-writing skill you already have carries over directly.

9.4 Realistic Business Problem: Employee Headcount and Payroll by Department

```
SELECT
    d.department_name,
    COUNT(*) AS headcount,
    SUM(e.salary) AS total_payroll,
    ROUND(AVG(e.salary), 2) AS avg_salary,
    MAX(e.salary) AS highest_salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_name
ORDER BY total_payroll DESC;
```

10. Joins

Join mechanics are almost entirely portable between MySQL and PostgreSQL; this part reinforces the concepts briefly and focuses on the few genuine syntax and behavior differences.

10.1 INNER JOIN

```
SELECT o.order_id, c.customer_name, o.total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Returns only rows with a match in both tables; fully portable, identical syntax and semantics to PostgreSQL.

10.2 LEFT JOIN and RIGHT JOIN

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;   -- customers with NO orders
```

LEFT JOIN keeps every row from the left table, filling unmatched right-side columns with NULL; identical to PostgreSQL, including this common 'find rows with no match' pattern. RIGHT JOIN is the mirror image and also fully portable; in practice, most developers in both databases prefer to always write LEFT JOIN and simply swap table order, keeping queries more consistent to read.

10.3 CROSS JOIN

```
SELECT p.product_name, s.store_name
FROM products p
CROSS JOIN stores s;
```

Produces the Cartesian product, every row from the first table paired with every row from the second; fully portable, identical to PostgreSQL. Genuinely useful for generating combinations (every product in every store, as a base for an inventory-planning table) but easy to trigger accidentally by forgetting a join condition.

10.4 SELF JOIN

```
SELECT e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

A table joined to itself using two aliases, typically for hierarchical or self-referencing relationships (an employee-manager structure here); fully portable, identical concept to PostgreSQL.

10.5 One-to-One, One-to-Many, Many-to-Many

Code Line	Explanation
`One-to-one`	Each row in table A matches at most one row in table B, typically enforced with a UNIQUE foreign key; identical modeling approach in both databases.
`One-to-many`	One row in the 'one' table (departments) can match many rows in the

Code Line	Explanation
	'many' table (employees); the most common relationship shape, identical in both databases.
Many-to-many	Modeled with a join/junction table holding two foreign keys (as in order_items from Part 5.6, linking orders and products); identical modeling approach in both databases.

10.6 Duplicate Rows and Missing Matches

```
-- A JOIN through a many-to-many junction can multiply rows unexpectedly
SELECT c.customer_name, SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_name;
```

Joining through order_items (many rows per order) means a naive SUM(o.total) at the order level, if o.total were also joined in directly without GROUP BY care, would double- or triple-count; this exact class of mistake, and how to avoid it, is identical in both MySQL and PostgreSQL and is a frequent source of subtly wrong analytics numbers.

10.7 Multiple Joins in One Query

```
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    e.first_name AS sales_rep,
    oi.quantity * oi.unit_price AS line_total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN employees e ON o.sales_rep_id = e.employee_id
ORDER BY o.order_id;
```

A five-table join chain (orders, customers, order_items, products, and an optional sales rep via LEFT JOIN, since not every order may have an assigned rep); fully portable syntax, and exactly the kind of query real analytics and reporting work requires constantly.

> **PostgreSQL Comparison:** MySQL's join execution strategy historically differed from PostgreSQL's (older MySQL versions relied heavily on nested-loop joins and lacked hash joins for a long time; MySQL 8.0.18+ added hash join support, closing much of this gap). The SQL syntax and logical results are identical between the two databases; only the internal execution plan can differ, a topic revisited in Part 17.

11. Subqueries

11.1 Scalar and Single-Row Subqueries

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

A scalar subquery returns exactly one value, usable anywhere a single value is expected; fully portable, identical to PostgreSQL.

11.2 Multi-Row Subqueries: IN, NOT IN

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE total > 1000);
```

Fully portable. Caution (identical in both databases): NOT IN with a subquery that could return NULL silently produces zero rows for the whole query, a classic trap; NOT EXISTS (section 11.4) is the safer alternative in both systems.

11.3 Correlated Subqueries

```
SELECT e1.name, e1.salary, e1.department_id
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e1.department_id
);
```

A correlated subquery references a column from the outer query (e1.department_id), so it re-executes conceptually once per outer row, here finding employees earning above their OWN department's average; fully portable syntax and behavior identical to PostgreSQL.

11.4 EXISTS and NOT EXISTS

```
SELECT c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

SELECT c.customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

EXISTS/NOT EXISTS check only for the presence/absence of matching rows, without caring about their actual values; fully portable, and generally the safest, most reliable choice for 'has a match' / 'has no match' logic in both MySQL and PostgreSQL, avoiding the NULL trap that NOT IN carries.

11.5 When to Prefer Joins, Subqueries, or CTEs

Code Line	Explanation
`Prefer a JOIN`	When you need columns from both tables in the final result set, not just a filtering condition.
`Prefer a subquery (EXISTS/IN)`	When you only need to filter based on a relationship's existence or a computed value, without needing any columns from the related table in the output.
`Prefer a CTE`	When a subquery's logic is complex enough to benefit from a clear name and would otherwise need to be repeated multiple times in one query (fully explored next, in Part 12).

This decision framework is identical in both MySQL and PostgreSQL; the modern query optimizers in both systems are generally capable of rewriting many subquery forms into equivalent, efficient join plans internally, so the choice is often more about readability than raw performance for common cases (Part 17 covers verifying this with EXPLAIN).

12. Common Table Expressions

12.1 MySQL 8.x CTE Support

MySQL introduced full CTE support, including recursive CTEs, in version 8.0. This is a genuinely important version distinction: MySQL 5.7 and earlier have NO CTE support at all; if you ever encounter MySQL 5.x code, WITH queries simply will not work and must be rewritten as subqueries or temporary tables (Part 26). This entire part assumes MySQL 8.0+, consistent with this course's modern focus.

12.2 Basic CTEs

```
WITH high_value_orders AS (
    SELECT * FROM orders WHERE total > 1000
)
SELECT customer_id, COUNT(*) AS num_orders
FROM high_value_orders
GROUP BY customer_id;
```

Fully portable syntax, identical to PostgreSQL's WITH clause: names a subquery result so it can be referenced by name in the main query, improving readability over an equivalent nested subquery.

12.3 Multiple and Chained CTEs

```
WITH regional_totals AS (
    SELECT region, SUM(total) AS revenue
    FROM orders
    GROUP BY region
),
ranked_regions AS (
    SELECT region, revenue,
           RANK() OVER (ORDER BY revenue DESC) AS revenue_rank
    FROM regional_totals
)
SELECT * FROM ranked_regions WHERE revenue_rank <= 3;
```

Multiple CTEs, separated by commas, where a later CTE (ranked_regions) references an earlier one (regional_totals); fully portable, identical chaining syntax to PostgreSQL.

12.4 Recursive CTEs

```
WITH RECURSIVE employee_hierarchy AS (
    SELECT employee_id, first_name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.first_name, e.manager_id, eh.level + 1
    FROM employees e
    JOIN employee_hierarchy eh ON e.manager_id = eh.employee_id
)
SELECT * FROM employee_hierarchy ORDER BY level, employee_id;
```

Code Line	Explanation
`WITH RECURSIVE`	Required keyword to enable recursion; fully portable, identical syntax to PostgreSQL.
`Anchor member`	The first SELECT (manager_id IS NULL) establishes the starting point: top-level employees with no manager.
`Recursive member`	The second SELECT, joined to the CTE's own name, repeatedly finds the next level down until no more matches are found; identical logical structure to PostgreSQL's recursive CTEs.

PostgreSQL Comparison: MySQL's recursive CTE syntax and semantics are essentially identical to PostgreSQL's, a rare case of near-perfect parity in an advanced feature. One MySQL-specific safeguard: the cte_max_recursion_depth system variable (default 1000) limits recursion depth to prevent runaway queries; PostgreSQL has no equivalent hard limit by default, relying instead on statement_timeout or manual query design to prevent infinite recursion.

12.5 CTEs for Analytics and Data Transformation

```
WITH monthly_revenue AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(total) AS revenue
    FROM orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    month,
    revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS month_over_month_change
FROM monthly_revenue
ORDER BY month;
```

A CTE cleanly separates the aggregation step (monthly_revenue) from the analytical window-function step (month-over-month change, fully covered in Part 13) that depends on it, keeping each logical step readable on its own; this staged-CTE pattern is one of the most common and valuable techniques in professional analytical SQL, identical in both MySQL and PostgreSQL.

13. Window Functions

MySQL 8.0 introduced full window function support, reaching parity with PostgreSQL's long-standing implementation. As with CTEs, this is version-critical: MySQL 5.7 and earlier have NO window function support at all.

13.1 OVER(), PARTITION BY, ORDER BY

```
SELECT
    employee_id, department_id, salary,
    AVG(salary) OVER (PARTITION BY department_id) AS dept_avg_salary
FROM employees;
```

Fully portable syntax and semantics, identical to PostgreSQL: OVER() turns an aggregate into a window function that computes per-row without collapsing rows via GROUP BY; PARTITION BY resets the calculation per group.

13.2 Window Frames

```
SELECT
    order_date, total,
    SUM(total) OVER (
        ORDER BY order_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS rolling_3day_total
FROM daily_sales;
```

ROWS BETWEEN ... AND ... defines the exact frame of rows included in the calculation for each row; fully portable, identical syntax to PostgreSQL.

13.3 ROW_NUMBER, RANK, DENSE_RANK, NTILE

```
SELECT
    name, department_id, salary,
    ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS row_num,
    RANK()       OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS
dense_rnk,
    NTILE(4)     OVER (ORDER BY salary DESC) AS salary_quartile
FROM employees;
```

Code Line	Explanation
`ROW_NUMBER()`	A unique, sequential number per row within its partition, with no ties; identical to PostgreSQL.
`RANK()`	Ties share the same rank, and the next rank skips accordingly (1, 1, 3); identical to PostgreSQL.
`DENSE_RANK()`	Ties share the same rank, but the next rank does NOT skip (1, 1, 2); identical to PostgreSQL.
`NTILE(n)`	Divides rows into n roughly equal buckets; identical to PostgreSQL, useful for quartile/decile-style segmentation.

13.4 LAG, LEAD, FIRST_VALUE, LAST_VALUE

```
SELECT
    month, revenue,
    LAG(revenue, 1) OVER (ORDER BY month) AS prev_month_revenue,
    LEAD(revenue, 1) OVER (ORDER BY month) AS next_month_revenue,
    FIRST_VALUE(revenue) OVER (ORDER BY month) AS first_month_revenue,
    LAST_VALUE(revenue) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_month_revenue
FROM monthly_revenue;
```

Code Line	Explanation
`LAG(col, n) / LEAD(col, n)`	Access a value from n rows before/after the current row within the ordered window; fully portable, identical to PostgreSQL.
`FIRST_VALUE() / LAST_VALUE()`	Access the first/last value in the window frame; fully portable syntax, but note the explicit ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING frame is required for LAST_VALUE() to see the true last row of the whole partition in BOTH MySQL and PostgreSQL, since the implicit default frame only extends to the current row, a commonly missed detail in both systems, not a MySQL-specific quirk.

13.5 Running Totals

```
SELECT
    order_date, total,
    SUM(total) OVER (ORDER BY order_date) AS running_total
FROM orders
ORDER BY order_date;
```

13.6 Moving Averages

```
SELECT
    order_date, total,
    AVG(total) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7day
FROM daily_sales
ORDER BY order_date;
```

13.7 Customer Ranking Analysis

```
SELECT
    customer_id,
    SUM(total) AS lifetime_value,
    RANK() OVER (ORDER BY SUM(total) DESC) AS customer_rank
FROM orders
```

```
GROUP BY customer_id
ORDER BY customer_rank
LIMIT 10;
```

Combining a window function with GROUP BY: the aggregation (SUM per customer) happens first, and RANK() then operates on the aggregated results, ranking the top 10 customers by lifetime spend. Fully portable syntax.

13.8 Year-over-Year Analysis

```
WITH yearly AS (
    SELECT YEAR(order_date) AS yr, SUM(total) AS revenue
    FROM orders
    GROUP BY YEAR(order_date)
)
SELECT
    yr, revenue,
    LAG(revenue) OVER (ORDER BY yr) AS prev_year_revenue,
    ROUND(
        (revenue - LAG(revenue) OVER (ORDER BY yr)) / LAG(revenue) OVER (ORDER BY
yr) * 100,
        2
    ) AS yoy_growth_pct
FROM yearly
ORDER BY yr;
```

A complete, realistic year-over-year growth calculation combining a CTE, GROUP BY aggregation, and LAG(); the only MySQL-specific element is the YEAR() function itself (Part 8.3), everything else is fully portable analytical SQL.

14. Set Operations

14.1 UNION and UNION ALL

```
SELECT customer_name, 'Active' AS status FROM active_customers
UNION
SELECT customer_name, 'Archived' AS status FROM archived_customers;

SELECT product_id FROM current_inventory
UNION ALL
SELECT product_id FROM discontinued_inventory;
```

Code Line	Explanation
`UNION`	Combines result sets and removes duplicate rows; fully portable, identical to PostgreSQL.
`UNION ALL`	Combines result sets and KEEPS duplicates; fully portable, and generally faster than UNION since it skips the deduplication step, identical performance consideration in both databases.

14.2 INTERSECT and EXCEPT: A Genuine Version-Specific Gap

Critical Version Difference: INTERSECT and EXCEPT were NOT supported in MySQL until version 8.0.31 (released late 2022). Any MySQL version before 8.0.31 has no INTERSECT or EXCEPT keywords at all, and they must be simulated with INNER JOIN (for INTERSECT) or LEFT JOIN ... WHERE right.key IS NULL / NOT EXISTS (for EXCEPT). PostgreSQL has supported both since its earliest versions. Always verify your MySQL server's exact version before relying on these keywords.

```
-- MySQL 8.0.31+: native syntax, identical to PostgreSQL
SELECT customer_id FROM this_month_customers
INTERSECT
SELECT customer_id FROM last_month_customers;

-- Portable equivalent that works on ANY MySQL 8.x version
SELECT DISTINCT t.customer_id
FROM this_month_customers t
INNER JOIN last_month_customers l ON t.customer_id = l.customer_id;
```

```
-- MySQL 8.0.31+: native syntax
SELECT customer_id FROM this_month_customers
EXCEPT
SELECT customer_id FROM last_month_customers;

-- Portable equivalent that works on ANY MySQL 8.x version
SELECT DISTINCT t.customer_id
FROM this_month_customers t
WHERE NOT EXISTS (
    SELECT 1 FROM last_month_customers l WHERE l.customer_id = t.customer_id
);
```

15. Views

15.1 Creating Views

```
CREATE VIEW active_customer_orders AS
SELECT o.order_id, c.customer_name, o.total, o.order_date
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.status = 'active';

SELECT * FROM active_customer_orders WHERE total > 500;
```

Fully portable syntax, identical to PostgreSQL: a view is a stored, named query that can be queried like a table, without duplicating the underlying logic.

15.2 Updating and Dropping Views

```
CREATE OR REPLACE VIEW active_customer_orders AS
SELECT o.order_id, c.customer_name, o.total, o.order_date, o.region
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.status = 'active';

DROP VIEW active_customer_orders;
```

CREATE OR REPLACE VIEW redefines an existing view atomically; fully portable, identical to PostgreSQL. DROP VIEW removes it entirely.

15.3 Updatable Views

```
CREATE VIEW simple_products AS
SELECT product_id, name, price FROM products;

UPDATE simple_products SET price = 12.99 WHERE product_id = 101;
```

A view based on a single table with no aggregation, DISTINCT, GROUP BY, or joins is generally updatable, meaning an UPDATE/INSERT/DELETE against the view passes through to the underlying table; this rule and its many disqualifying conditions (joins, aggregates, window functions, and so on) are essentially identical between MySQL and PostgreSQL, both following the same standard SQL updatability principles.

15.4 Complex Views

```
CREATE VIEW customer_lifetime_value AS
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS total_orders,
    COALESCE(SUM(o.total), 0) AS lifetime_value,
    MAX(o.order_date) AS last_order_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.customer_id, c.customer_name;
```

A view built on a join and aggregation, useful for centralizing a commonly needed business metric (customer lifetime value) so every report and query can reference one consistent, tested definition instead of re-deriving it repeatedly; this view is NOT updatable due to the aggregation, exactly the same rule that would apply in PostgreSQL.

15.5 Views vs Tables, CTEs, and Derived Tables

Code Line	Explanation
`View`	A stored, named, reusable query definition; always reflects live, current underlying data, and is accessible by name from any query or connection, just like a table.
`Table`	Stores actual physical data on disk. A view stores no data of its own (except a MATERIALIZED VIEW, which PostgreSQL supports natively but MySQL does NOT support at all as of version 8.x, a genuine MySQL-specific gap covered again in Part 42).
`CTE`	A named subquery scoped to ONE single query, not stored or reusable across different queries the way a view is (Part 12).
`Derived table`	An unnamed (or query-local-named) subquery in the FROM clause, scoped to one query only, conceptually the ancestor of the CTE (fully covered in Part 26).

Important MySQL Gap: PostgreSQL supports CREATE MATERIALIZED VIEW, which physically stores (caches) a view's result and must be manually or automatically REFRESHed. MySQL has no native materialized view feature at all; the common MySQL workaround is a real table populated and periodically refreshed via a scheduled Event (Part 23) or an application/ETL job, a meaningfully more manual process than PostgreSQL's built-in support.

16. Indexes

16.1 What an Index Is (Recap)

An index is a separate data structure that lets the database find rows without scanning an entire table, exactly as in PostgreSQL. The core concept, tradeoffs, and B-tree structure are the same; this part focuses on MySQL's specific index types and InnoDB-specific behavior.

16.2 Primary, Secondary, and Unique Indexes

```
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    INDEX idx_customer (customer_id),
    UNIQUE INDEX idx_order_ref (order_reference)
);

CREATE INDEX idx_order_date ON orders(order_date);
DROP INDEX idx_order_date ON orders;
```

Code Line	Explanation
`PRIMARY KEY`	Automatically creates a unique index, and in InnoDB specifically, this index IS the table itself physically (a clustered index, section 16.7); this is a genuine structural difference from PostgreSQL, where the table's physical row order is generally unrelated to the primary key unless CLUSTER is run manually.
`INDEX / KEY (secondary index)`	An ordinary non-unique index; INDEX and KEY are exact synonyms in MySQL's CREATE TABLE syntax.
`UNIQUE INDEX`	Enforces uniqueness while also speeding up lookups; identical concept to PostgreSQL's UNIQUE INDEX.
`DROP INDEX idx_name ON table_name;`	MySQL requires specifying the table name when dropping an index, since index names are only unique per-table in MySQL. PostgreSQL index names are unique per-DATABASE (schema-qualified), so PostgreSQL's DROP INDEX idx_name; needs no table reference, a genuine syntax difference.

16.3 Composite Indexes

```
CREATE INDEX idx_customer_date ON orders(customer_id, order_date);
```

A composite (multi-column) index; fully portable concept, identical to PostgreSQL. Composite-index column order is critical in both systems (section 16.6 below).

16.4 Prefix Indexes (MySQL-Specific)

```
CREATE INDEX idx_email_prefix ON customers(email(10));
```

PostgreSQL Comparison: A prefix index indexes only the first N characters of a string column, saving

space at the cost of slightly reduced selectivity. This exact syntax, column(length), is MySQL-specific. PostgreSQL has no direct equivalent syntax; the closest PostgreSQL approach is a functional/expression index on LEFT(column, length), a meaningfully different mechanism.

16.5 Functional (Expression) Indexes

```
CREATE INDEX idx_lower_email ON customers((LOWER(email)));
```

MySQL 8.0.13+ supports functional indexes on an expression rather than a raw column, speeding up queries that filter on a transformed value (like case-insensitive email lookup); the double parentheses around the expression are required syntax. PostgreSQL has supported expression indexes for considerably longer, with very similar syntax (CREATE INDEX ON table (LOWER(email));, no extra parentheses needed there).

16.6 Composite-Index Column Order

A composite index on (customer_id, order_date) can efficiently serve queries filtering on customer_id alone, or on customer_id AND order_date together, but CANNOT efficiently serve a query filtering on order_date alone, since the index is physically sorted by customer_id first. This 'leftmost prefix' rule is identical in both MySQL and PostgreSQL, and choosing the most selective, most frequently filtered column first in a composite index is a universal indexing principle, not MySQL-specific.

16.7 Clustered Indexes (InnoDB-Specific)

Critical PostgreSQL Comparison: This is one of the most important InnoDB-specific concepts for anyone coming from PostgreSQL. In InnoDB, the primary key IS the table's physical storage order (a clustered index); every secondary index stores the primary key value, not a physical row pointer, and must perform an extra lookup back into the primary key's data to retrieve other columns (unless the query is fully covered, section 16.8). PostgreSQL tables are heap-organized by default (no inherent physical ordering tied to the primary key), and every index, including the primary key's own index, points to a physical row location (a TID) directly. This means primary key choice matters MORE for physical performance in InnoDB than in PostgreSQL: a monotonically increasing primary key (like AUTO_INCREMENT) keeps new-row inserts efficient at the end of the table's physical storage, while a random primary key (like a UUID) can cause significant, InnoDB-specific page-fragmentation performance problems that a PostgreSQL heap table would not experience the same way.

16.8 Covering Indexes

```
CREATE INDEX idx_covering ON orders(customer_id, order_date, total);

-- This query can be answered ENTIRELY from the index, with no lookup into the table
SELECT order_date, total FROM orders WHERE customer_id = 42;
```

A covering index includes every column a query needs (in the WHERE clause and the SELECT list), letting MySQL satisfy the query entirely from the index itself, without the extra lookup into the underlying table row (an 'index-only' access, visible as 'Using index' in EXPLAIN, Part 17); identical concept and significant performance benefit in both MySQL and PostgreSQL.

16.9 Full-Text Indexes

```
CREATE FULLTEXT INDEX idx_ft_description ON products(description);

SELECT * FROM products
WHERE MATCH(description) AGAINST('wireless bluetooth' IN NATURAL LANGUAGE MODE);
```

MySQL's built-in full-text search, available on InnoDB (since 5.6) and MyISAM. Conceptually similar in purpose to PostgreSQL's tsvector/tsquery full-text search system, but with meaningfully different, much simpler syntax (MATCH...AGAINST versus PostgreSQL's @@ operator and to_tsvector()/to_tsquery() functions) and generally less linguistic sophistication (stemming, ranking flexibility) than PostgreSQL's implementation.

16.10 Spatial Indexes

MySQL supports SPATIAL INDEX on spatial data type columns (Part 4.6) for geographic queries; conceptually parallel to PostgreSQL's GiST-based spatial indexes used by the PostGIS extension, though PostGIS is generally considered more mature and feature-rich for serious GIS work.

16.11 Index Selectivity and When Indexes Hurt Performance

- An index on a low-cardinality column (like a boolean status flag with only 2-3 distinct values) often provides little to no benefit, since the optimizer may reasonably choose a full table scan instead; identical principle in both databases.
- Every index adds overhead to every INSERT/UPDATE/DELETE, since the index itself must also be maintained; too many indexes on a write-heavy table can meaningfully slow down writes, identical tradeoff in both databases.
- In InnoDB specifically, because secondary indexes store the primary key value (section 16.7), a large or inefficiently-typed primary key makes every single secondary index larger too, a compounding cost that has no direct PostgreSQL parallel, since PostgreSQL's secondary indexes reference a fixed-size physical row pointer regardless of primary key size or type.

17. Query Optimization

17.1 EXPLAIN

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
```

Code Line	Explanation
`type`	The join/access type for this step, from best to worst roughly: system, const, eq_ref, ref, range, index, ALL (a full table scan, generally the one to worry about on a large table).
`possible_keys / key`	Which indexes COULD have been used, and which one was ACTUALLY chosen by the optimizer.
`rows`	The optimizer's ESTIMATED number of rows it will need to examine; a rough proxy for query cost.
`Extra`	Additional notes; 'Using index' (a covering index, section 16.8, ideal), 'Using filesort' (an expensive extra sorting step), 'Using temporary' (an expensive temporary table needed, often from GROUP BY or DISTINCT on unindexed columns) are the most important ones to watch for.

Critical PostgreSQL Comparison: MySQL's EXPLAIN output format and column names (type, key, rows, Extra) are ENTIRELY different from PostgreSQL's EXPLAIN output (which shows a tree of nodes like 'Seq Scan', 'Index Scan', 'Hash Join' with cost estimates in a different notation). The underlying goal, understanding the execution plan, is identical, but you must relearn how to read the output format itself; this is one of the most immediately noticeable day-to-day differences coming from PostgreSQL.

17.2 EXPLAIN ANALYZE (MySQL 8.0.18+)

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;
```

Unlike plain EXPLAIN (which only estimates), EXPLAIN ANALYZE actually EXECUTES the query and reports real, measured timing and row counts alongside the plan; introduced in MySQL 8.0.18, roughly matching a capability PostgreSQL has offered for considerably longer. The output format is a readable, indented tree, distinct from both plain MySQL EXPLAIN's tabular format and PostgreSQL's own EXPLAIN ANALYZE tree format.

17.3 Table Scans vs Index Scans

A full table scan (type: ALL) reads every row; an index scan uses an index to jump directly to relevant rows. Exactly as in PostgreSQL, a full table scan is not automatically bad, for a small table, or a query that genuinely needs most of the table's rows, a full scan can be faster than the overhead of using an index; the optimizer's cost-based decision, and how to verify it with EXPLAIN, is the same skill in both databases.

17.4 Before-and-After Optimization Example

```
-- BEFORE: no index on customer_id, full table scan
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- type: ALL, rows: 500000, key: NULL
```

```
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- AFTER: uses the new index
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- type: ref, rows: 12, key: idx_orders_customer
```

The estimated rows examined drops from 500,000 (the whole table) to 12 (just this customer's orders) after adding the index, and type improves from ALL to ref; this same before/after diagnostic workflow with EXPLAIN is identical in spirit to the PostgreSQL optimization process you already know, just with MySQL's own output format.

17.5 Slow Queries and the Slow Query Log

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1;
```

MySQL's slow query log records any query taking longer than long_query_time seconds, a first-line diagnostic tool for finding optimization targets in a running application; conceptually parallel to PostgreSQL's log_min_duration_statement setting, with different configuration variable names.

18. Storage Engines

This is one of the most fundamentally MySQL-specific topics in this entire course, with genuinely no PostgreSQL equivalent concept at all, since PostgreSQL has exactly one built-in storage engine (heap tables) for the whole system.

18.1 What a Storage Engine Is

A storage engine is the underlying component responsible for how MySQL physically stores, retrieves, indexes, and locks data on disk. MySQL's pluggable storage engine architecture lets different TABLES within the same database use different engines, each with different tradeoffs around transactions, locking, and performance. This is architecturally unlike anything in PostgreSQL, where every table always uses the same single storage mechanism.

18.2 InnoDB (The Modern Default)

- Full ACID-compliant transaction support (Part 19).
- Foreign key constraint enforcement (the ONLY engine that enforces them, per Part 5.2).
- Row-level locking, allowing high concurrency (many transactions can modify different rows of the same table simultaneously without blocking each other).
- A clustered primary key index (Part 16.7).
- MVCC (Multi-Version Concurrency Control) for consistent, non-blocking reads during concurrent writes.
- Crash recovery via its own transaction log (Part 29).

InnoDB has been MySQL's default storage engine since version 5.5, and should be the default choice for essentially every application table today.

18.3 MyISAM (Legacy)

- No transaction support at all: no COMMIT/ROLLBACK, no atomicity.
- No foreign key enforcement, even if the syntax is written, it is silently ignored.
- Table-level locking only: a single write locks the ENTIRE table, blocking all other reads and writes, a severe concurrency bottleneck under real multi-user load.
- Historically faster for pure, simple read-heavy, single-user workloads and certain full-text search use cases, though InnoDB has closed most of this gap in modern MySQL.

> **Practical Guidance:** MyISAM should be treated as legacy for new application development. It survives mainly in older systems and specific niche cases. If you inherit a MySQL database using MyISAM tables, be aware that foreign keys and transactions silently do not work at all, a serious data-integrity risk with no PostgreSQL parallel to worry about, since PostgreSQL has never had a non-transactional table option.

18.4 Memory, CSV, and Archive Engines (Brief)

Code Line	Explanation
`MEMORY`	Stores table data entirely in RAM for extremely fast access; data is

Code Line	Explanation
	lost on server restart. Useful for temporary lookup/staging tables within a session. No direct PostgreSQL parallel (PostgreSQL's UNLOGGED TABLE is the closest concept, trading durability for speed but still persisting to disk).
`CSV`	Stores data as a literal, plain CSV file on disk, directly readable/writable by external tools; a MySQL-specific niche engine with no PostgreSQL equivalent.
`ARCHIVE`	Highly compressed, INSERT-and-SELECT-only storage (no UPDATE/DELETE support at all) designed for storing large volumes of infrequently accessed historical/audit data cheaply; no direct PostgreSQL equivalent, though PostgreSQL's table partitioning combined with compression extensions can achieve similar goals differently.

18.5 Row-Level Locking, Clustered Indexes, and MVCC (InnoDB Deep Dive)

Code Line	Explanation
`Row-level locking`	InnoDB locks only the specific rows a transaction touches, not the whole table, allowing many concurrent transactions on different rows; conceptually similar in GOAL to PostgreSQL's own row-level locking, though the internal locking implementation details differ.
`Clustered index`	Already covered in depth in section 16.7: the primary key IS the physical table storage order in InnoDB, a genuine structural difference from PostgreSQL's heap-organized tables.
`MVCC`	InnoDB uses undo logs (Part 29) to let readers see a consistent snapshot of data without blocking concurrent writers, and vice versa; conceptually very similar in GOAL to PostgreSQL's own MVCC implementation, though the specific mechanisms differ: PostgreSQL keeps old row versions directly in the table's heap (requiring VACUUM to reclaim space), while InnoDB keeps old versions in a separate undo log structure that is automatically purged.

18.6 Why InnoDB Is Preferred for Transactional Applications

Any application requiring data integrity guarantees, financial transactions, order processing, user account management, needs atomic, consistent, isolated, durable operations (ACID, fully covered in Part 19) and enforced foreign key relationships. MyISAM provides neither. This makes InnoDB the only reasonable choice for virtually all modern application development, exactly mirroring the reasoning that makes PostgreSQL's own (only) storage layer the correct choice there by default, just made explicit as a deliberate engine choice in MySQL rather than being architecturally guaranteed.

```
SELECT table_name, engine FROM information_schema.tables WHERE table_schema =
'company_db';
ALTER TABLE legacy_table ENGINE = InnoDB;
```

Checking and converting a table's engine; converting a large MyISAM table to InnoDB rewrites the entire table and can take significant time and disk space on a large, live table, worth planning carefully during a maintenance window (Part 29).

19. Transactions and ACID

19.1 START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

SAVEPOINT before_fee;
UPDATE accounts SET balance = balance - 5 WHERE account_id = 1;

ROLLBACK TO SAVEPOINT before_fee;

COMMIT;
```

Code Line	Explanation
`START TRANSACTION / BEGIN`	Both start a new transaction in MySQL; fully portable syntax (PostgreSQL also accepts both, though BEGIN is more idiomatic there while START TRANSACTION is more idiomatic in MySQL).
`COMMIT`	Permanently saves every change made since the transaction began; identical to PostgreSQL.
`ROLLBACK`	Discards every change made since the transaction began (or since a named SAVEPOINT); identical to PostgreSQL.
`SAVEPOINT / ROLLBACK TO SAVEPOINT`	Creates a named point within a transaction to roll back to partially, without discarding the entire transaction; identical syntax and behavior to PostgreSQL, requires InnoDB (Part 18).

PostgreSQL Comparison: MySQL defaults to autocommit=1 (ON): every individual statement outside an explicit transaction is automatically committed immediately. This matches PostgreSQL's default behavior in psql as well, so this specific default is NOT a difference; both systems require an explicit START TRANSACTION/BEGIN to group multiple statements atomically.

19.2 The Four ACID Properties (Recap)

Code Line	Explanation
`Atomicity`	A transaction's statements all succeed together or all fail together, with no partial effect; identical guarantee in both databases when using InnoDB.
`Consistency`	A transaction moves the database from one valid state to another, never violating constraints; identical concept in both databases.
`Isolation`	Concurrent transactions do not see each other's uncommitted changes, to a degree controlled by the isolation level (section 19.3); the DEFAULT isolation level differs between the two databases, a genuinely important distinction covered next.
`Durability`	Once committed, changes survive a crash, guaranteed by InnoDB's redo log (Part 29); conceptually identical to PostgreSQL's write-ahead log (WAL), with different internal implementation.

19.3 Isolation Levels

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT @@transaction_isolation;
```

Code Line	Explanation
`READ UNCOMMITTED`	Allows dirty reads (seeing another transaction's uncommitted changes); identical concept in both, rarely used in practice.
`READ COMMITTED`	Only ever sees committed data; prevents dirty reads but allows non-repeatable reads. This is PostgreSQL's DEFAULT isolation level.
`REPEATABLE READ`	A transaction sees a consistent snapshot for its entire duration; prevents dirty and non-repeatable reads. This is InnoDB/MySQL's DEFAULT isolation level, a genuine and important default difference from PostgreSQL. InnoDB's REPEATABLE READ also prevents most phantom reads via a mechanism called next-key locking, going further than the bare SQL standard requires for this level.
`SERIALIZABLE`	The strictest level: transactions behave as if executed one at a time, sequentially; identical concept and behavior goal in both databases, at the cost of the most locking/blocking.

Critical PostgreSQL Comparison: MySQL's default isolation level is REPEATABLE READ. PostgreSQL's default isolation level is READ COMMITTED. This is a genuinely important behavioral difference: code or assumptions built around PostgreSQL's looser default may behave differently under MySQL's stricter default, particularly around what a long-running transaction can and cannot see change underneath it. Always verify the isolation level explicitly for any code where this distinction matters, rather than assuming either default.

19.4 Dirty Reads, Non-Repeatable Reads, Phantom Reads

Code Line	Explanation
`Dirty read`	Reading another transaction's uncommitted (and possibly soon-to-be-rolled-back) changes; identical concept in both databases.
`Non-repeatable read`	Reading the same row twice within one transaction and getting different values, because another transaction committed a change in between; identical concept in both databases.
`Phantom read`	Re-running the same query twice within one transaction and getting a different SET of rows (new rows appeared or disappeared), because another transaction committed an insert/delete in between; identical concept in both databases, though InnoDB's REPEATABLE READ level specifically mitigates this more aggressively than the bare SQL standard requires, as noted above.

19.5 Locking and Deadlocks

```
SELECT * FROM accounts WHERE account_id = 1 FOR UPDATE;
```

FOR UPDATE locks the selected rows for the duration of the transaction, preventing other transactions from modifying (or in some isolation levels, even reading) them until this transaction commits or rolls back; fully portable syntax, identical to PostgreSQL. A deadlock occurs when two transactions each hold a lock the other needs; InnoDB automatically detects deadlocks and forcibly rolls back one of the two transactions (returning an

error the application must catch and retry), conceptually identical to how PostgreSQL handles deadlock detection, though the specific victim-selection algorithm differs internally.

20. Stored Procedures

20.1 Creating Procedures

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByDept(IN dept_id INT)
BEGIN
    SELECT employee_id, first_name, last_name, salary
    FROM employees
    WHERE department_id = dept_id;
END //

DELIMITER ;

CALL GetEmployeesByDept(3);
```

Code Line	Explanation
`DELIMITER //`	Temporarily changes the statement terminator from the default ; to //, so the semicolons INSIDE the procedure body do not prematurely end the CREATE PROCEDURE statement itself. This is an entirely MySQL-specific client convention (needed because the mysql command-line client parses on semicolons by default); PostgreSQL's psql instead uses dollar-quoting ($$ ... $$) to delimit function bodies, a completely different, and generally less error-prone, mechanism.
`CALL GetEmployeesByDept(3);`	Executes the procedure; CALL is standard SQL and portable, though PostgreSQL historically used functions for almost everything CALL now also supports as of PostgreSQL 11+ (true PROCEDUREs, distinct from FUNCTIONs, are a relatively recent PostgreSQL addition).

Practical Note: The DELIMITER command is a client-side convention, not part of the SQL language itself; it will not work if sent as a query through most programming language database drivers (Part 32), only inside the interactive mysql client or a script run through it. Application code creating procedures programmatically typically sends the CREATE PROCEDURE body as a single statement without needing DELIMITER at all.

20.2 Parameters: IN, OUT, INOUT

```
DELIMITER //
CREATE PROCEDURE GetDepartmentStats(
    IN dept_id INT,
    OUT emp_count INT,
    OUT avg_salary DECIMAL(10,2)
)
BEGIN
    SELECT COUNT(*), AVG(salary) INTO emp_count, avg_salary
    FROM employees WHERE department_id = dept_id;
END //
DELIMITER ;
```

```
CALL GetDepartmentStats(3, @count, @avg);
SELECT @count, @avg;
```

Code Line	Explanation
IN	A parameter passed INTO the procedure; the default if unspecified, identical concept to PostgreSQL.
OUT	A parameter the procedure sets and returns TO the caller via a session variable (@count, @avg here); MySQL-specific mechanism using @-prefixed user variables. PostgreSQL functions instead RETURN values directly (or use OUT parameters conceptually similarly, but without needing a separate SELECT ... INTO session-variable retrieval step).
INOUT	A parameter that is both read on entry and can be modified and returned; identical concept in both databases.

20.3 Variables, Conditional Logic, Loops

```
DELIMITER //
CREATE PROCEDURE ClassifyEmployee(IN emp_id INT)
BEGIN
    DECLARE emp_salary DECIMAL(10,2);
    DECLARE tier VARCHAR(20);

    SELECT salary INTO emp_salary FROM employees WHERE employee_id = emp_id;

    IF emp_salary >= 100000 THEN
        SET tier = 'Senior';
    ELSEIF emp_salary >= 60000 THEN
        SET tier = 'Mid';
    ELSE
        SET tier = 'Junior';
    END IF;

    UPDATE employees SET salary_tier = tier WHERE employee_id = emp_id;
END //
DELIMITER ;
```

DECLARE, IF/ELSEIF/ELSE/END IF, SET are all part of MySQL's procedural extension language for stored routines. This is conceptually parallel to PostgreSQL's PL/pgSQL (a separate, dedicated procedural language), but the syntax differs meaningfully throughout: MySQL's IF...THEN...ELSEIF...END IF versus PL/pgSQL's near-identical-looking but distinctly different IF...THEN...ELSIF...END IF (note ELSIF, one fewer E, in PostgreSQL), and numerous other small syntax differences that make direct copy-paste porting unreliable.

```
DELIMITER //
CREATE PROCEDURE GiveRaises(IN dept_id INT, IN pct DECIMAL(5,2))
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE emp_id INT;
    DECLARE cur CURSOR FOR
        SELECT employee_id FROM employees WHERE department_id = dept_id;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO emp_id;
        IF done THEN
            LEAVE read_loop;
        END IF;
        UPDATE employees SET salary = salary * (1 + pct / 100) WHERE employee_id =
emp_id;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;
```

Code Line	Explanation
`CURSOR / OPEN / FETCH / CLOSE`	MySQL's cursor mechanism for row-by-row procedural processing; conceptually parallel to PostgreSQL's PL/pgSQL cursors, with different declaration syntax.
`DECLARE CONTINUE HANDLER FOR NOT FOUND`	MySQL's error/condition-handling mechanism: this specific handler catches the 'no more rows' condition from FETCH and sets a flag to exit the loop cleanly. This handler-based error handling style is genuinely MySQL-specific in its exact syntax; PL/pgSQL uses its own EXCEPTION WHEN ... THEN block structure instead, syntactically quite different.

21. User-Defined Functions

21.1 Creating Functions

```
DELIMITER //

CREATE FUNCTION CalculateTax(amount DECIMAL(10,2), rate DECIMAL(5,4))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN ROUND(amount * rate, 2);
END //

DELIMITER ;

SELECT product_name, price, CalculateTax(price, 0.08) AS tax
FROM products;
```

Code Line	Explanation
`RETURNS DECIMAL(10,2)`	Declares the function's return type; required, and portable in concept to PostgreSQL's RETURNS clause.
`DETERMINISTIC`	A required declaration (or its opposite, NOT DETERMINISTIC) telling MySQL whether the function always returns the same output for the same input; MySQL enforces this more strictly than PostgreSQL in certain contexts (like being usable in generated/virtual columns or with certain binary logging replication modes, Part 39) where a function must be proven deterministic. PostgreSQL's closest equivalent concept is its own volatility markers (IMMUTABLE, STABLE, VOLATILE), a similar idea expressed with different keywords and a finer-grained three-way distinction rather than MySQL's simpler two-way one.

21.2 Functions vs Procedures

Code Line	Explanation
`Function`	Must RETURN a single scalar value; can be used directly INSIDE a SQL expression (SELECT CalculateTax(price, 0.08) FROM products), exactly like a built-in function.
`Procedure`	Does not directly return a value in an expression; invoked separately with CALL, and can return results via OUT parameters (Part 20.2) or by simply running a SELECT that streams back a result set. Cannot be used inline inside another SQL expression.

This functional/procedural distinction is conceptually identical between MySQL and PostgreSQL, though PostgreSQL historically blurred the line by letting ordinary FUNCTIONs do almost everything procedures now do, before introducing true PROCEDUREs as a separate concept in version 11.

21.3 A Practical Example: Data Validation Function

```
DELIMITER //
```

```
CREATE FUNCTION IsValidEmail(email VARCHAR(255))
RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    RETURN email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$';
END //
DELIMITER ;

SELECT customer_name, email FROM customers WHERE NOT IsValidEmail(email);
```

A reusable validation function usable directly in a WHERE clause, centralizing the email-validity regex logic in one place rather than repeating it across many queries; a genuinely practical, common use case for a small deterministic function in both MySQL and PostgreSQL.

22. Triggers

22.1 The Six Trigger Types

MySQL supports triggers firing BEFORE or AFTER each of INSERT, UPDATE, and DELETE, six combinations total. The core concept is fully portable to PostgreSQL, though PostgreSQL additionally requires a separate CREATE FUNCTION plus CREATE TRIGGER (the trigger references a function that implements the logic), while MySQL writes the trigger's procedural body directly inline within CREATE TRIGGER itself, a meaningful structural syntax difference.

22.2 Audit Logging Example

```
CREATE TABLE salary_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit (employee_id, old_salary, new_salary)
        VALUES (OLD.employee_id, OLD.salary, NEW.salary);
    END IF;
END //
DELIMITER ;
```

Code Line	Explanation
`OLD.salary / NEW.salary`	Reference the row's value before and after the update respectively; identical concept and identical OLD/NEW naming to PostgreSQL's trigger row variables.
`FOR EACH ROW`	Specifies this is a row-level trigger, firing once per affected row; MySQL supports ONLY row-level triggers (FOR EACH ROW is technically required syntax in MySQL, unlike PostgreSQL, which also supports statement-level triggers that fire once per statement regardless of how many rows were affected, a capability MySQL entirely lacks).

22.3 Automatic Timestamps

```
CREATE TABLE articles (
    article_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

PostgreSQL Comparison: MySQL offers ON UPDATE CURRENT_TIMESTAMP as a direct COLUMN ATTRIBUTE, automatically refreshing a timestamp on every row update WITHOUT needing a trigger at all. This is a genuinely convenient MySQL-specific shortcut; PostgreSQL has no equivalent column attribute, and achieving the same 'auto-update timestamp on any change' behavior in PostgreSQL requires writing an actual BEFORE UPDATE trigger and function, even for this simple, common case.

22.4 Validation Trigger

```
DELIMITER //
CREATE TRIGGER trg_validate_salary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //
DELIMITER ;
```

Code Line	Explanation
`SIGNAL SQLSTATE '45000'`	MySQL's mechanism for raising a custom error from within procedural code, aborting the operation; '45000' is the standard generic 'unhandled user-defined exception' SQLSTATE code. PostgreSQL's equivalent is RAISE EXCEPTION 'message', syntactically simpler but conceptually identical in purpose.

22.5 Change Tracking Example

```
DELIMITER //
CREATE TRIGGER trg_track_deletion
BEFORE DELETE ON customers
FOR EACH ROW
BEGIN
    INSERT INTO deleted_customers_log (customer_id, customer_name, deleted_at)
    VALUES (OLD.customer_id, OLD.customer_name, NOW());
END //
DELIMITER ;
```

A common soft-delete-style audit pattern: preserving a record of deleted data before it is physically removed, using BEFORE DELETE specifically so OLD values are still available; identical logical pattern to PostgreSQL, just with MySQL's inline trigger-body syntax.

22.6 Trigger Limitations and Maintainability Concerns

- A trigger CANNOT modify the table it is defined on from within an AFTER trigger in certain recursive scenarios, and cannot call a stored PROCEDURE that starts or commits a transaction (identical restriction in PostgreSQL).

- Excessive trigger logic scattered across many tables makes it genuinely difficult to trace WHY a value changed, since the change might originate from application code, a trigger, another trigger cascading from the first, or a stored procedure, an issue identical in severity in both databases.
- Triggers add invisible overhead to every INSERT/UPDATE/DELETE; a complex trigger chain can silently make simple-looking write operations significantly slower than they appear.
- MySQL allows only ONE trigger per table per event/timing combination (for example, only one BEFORE INSERT trigger per table) prior to MySQL 5.7.2; MySQL 5.7.2+ allows multiple triggers per event with an explicit FOLLOWS/PRECEDES ordering clause. PostgreSQL has always allowed multiple triggers per event, firing in alphabetical order by trigger name by default, a genuine historical difference worth knowing when working with an older MySQL version.

23. Events and Automation

The MySQL Event Scheduler is another meaningfully MySQL-specific feature area: a built-in, in-database job scheduler with no direct PostgreSQL core equivalent at all (PostgreSQL relies on the external OS-level cron, or the popular pg_cron extension, to achieve the same goal).

23.1 Enabling the Event Scheduler

```
SHOW VARIABLES LIKE 'event_scheduler';
SET GLOBAL event_scheduler = ON;
```

The Event Scheduler is OFF by default and must be explicitly enabled; this is a genuinely easy step to forget, after which every CREATE EVENT will exist but silently never actually run.

23.2 Creating a One-Time Event

```
CREATE EVENT clean_old_sessions
ON SCHEDULE AT '2026-12-31 23:59:00'
DO
    DELETE FROM user_sessions WHERE expires_at < NOW();
```

23.3 Creating a Recurring Event

```
DELIMITER //
CREATE EVENT refresh_daily_summary
ON SCHEDULE EVERY 1 DAY
STARTS '2026-01-01 02:00:00'
DO
BEGIN
    TRUNCATE TABLE daily_sales_summary;
    INSERT INTO daily_sales_summary
    SELECT order_date, SUM(total) AS revenue, COUNT(*) AS num_orders
    FROM orders
    GROUP BY order_date;
END //
DELIMITER ;
```

Code Line	Explanation
`ON SCHEDULE EVERY 1 DAY`	Defines the recurrence interval; MySQL supports EVERY with a wide range of units (SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, YEAR).
`STARTS '2026-01-01 02:00:00'`	Sets when the recurring schedule begins; optionally paired with an ENDS clause to set an expiration.
`This full pattern directly implements the MySQL materialized-view workaround`	mentioned in Part 15.5: since MySQL has no native materialized view, a scheduled event that periodically truncates and repopulates a real summary table is the standard MySQL way to achieve the same practical outcome PostgreSQL's REFRESH MATERIALIZED VIEW provides natively.

23.4 Enabling, Disabling, and Managing Events

```
ALTER EVENT refresh_daily_summary DISABLE;
ALTER EVENT refresh_daily_summary ENABLE;
DROP EVENT refresh_daily_summary;
SHOW EVENTS;
```

24. JSON in MySQL

24.1 The JSON Data Type

```
CREATE TABLE customer_profiles (
    customer_id INT PRIMARY KEY,
    preferences JSON
);

INSERT INTO customer_profiles VALUES
(1, '{"newsletter": true, "favorite_categories": ["electronics", "books"]}');
```

MySQL validates JSON syntax automatically on insert, rejecting malformed JSON with an error, and stores it in an internal optimized binary format (not just plain text) for faster access; conceptually parallel to PostgreSQL's JSONB (both are validated, binary-stored JSON), though MySQL has only ONE JSON type, unlike PostgreSQL's two-type split between JSON (validated, stored as plain text, preserves exact formatting/key order/duplicate keys) and JSONB (validated, binary, normalized, and indexable). MySQL's single JSON type behaves much more like PostgreSQL's JSONB in practice.

24.2 JSON Extraction: -> and ->> Operators, JSON_EXTRACT

```
SELECT preferences->'$.newsletter' AS newsletter_raw,
       preferences->>'$.newsletter' AS newsletter_text,
       JSON_EXTRACT(preferences, '$.favorite_categories[0]') AS first_category
FROM customer_profiles;
```

Code Line	Explanation
`->`	Extracts a value at a given JSON path, returning it AS JSON (so a string value would still be double-quoted in the result); syntax and behavior parallel PostgreSQL's own -> operator closely, though MySQL uses a JSON path STRING ('$.newsletter') as the right operand, while PostgreSQL uses a plain key name or integer directly ('newsletter'), a genuine syntax difference.
`->>`	Extracts a value and unquotes it to a plain SQL string/scalar; directly parallels PostgreSQL's own ->> operator in purpose, again with MySQL's path-string syntax difference.
`JSON_EXTRACT(json_doc, path)`	The full function form that -> is shorthand for; portable in concept, MySQL-specific in exact name (PostgreSQL's function-form equivalent is jsonb_extract_path()).

24.3 JSON Path Syntax

Code Line	Explanation
`$`	The root of the document.
`$.key`	A specific object key.
`$.key1.key2`	A nested key.
`$[0]`	The first element of an array.
`$.items[*]`	Every element of an array (wildcard).

24.4 Modifying JSON: JSON_SET, JSON_INSERT, JSON_REMOVE

```
UPDATE customer_profiles
SET preferences = JSON_SET(preferences, '$.newsletter', false)
WHERE customer_id = 1;

UPDATE customer_profiles
SET preferences = JSON_INSERT(preferences, '$.theme', 'dark')
WHERE customer_id = 1;

UPDATE customer_profiles
SET preferences = JSON_REMOVE(preferences, '$.theme')
WHERE customer_id = 1;
```

Code Line	Explanation
JSON_SET	Sets a value at a path, creating it if it does not exist, or overwriting it if it does.
JSON_INSERT	Sets a value ONLY if the path does not already exist; will not overwrite an existing value, a useful safety distinction JSON_SET does not make.
JSON_REMOVE	Deletes the value at a given path entirely.

These are all MySQL-specific function names; PostgreSQL's JSONB modification uses a meaningfully different set of tools: the jsonb_set() function and the || concatenation operator for merging, plus the #- operator for removing a path, none of which share MySQL's naming.

24.5 JSON Arrays

```
SELECT JSON_ARRAY(1, 2, 3);
SELECT JSON_ARRAY_APPEND(preferences, '$.favorite_categories', 'toys')
FROM customer_profiles WHERE customer_id = 1;
SELECT JSON_LENGTH(preferences, '$.favorite_categories') FROM customer_profiles;
```

24.6 JSON_TABLE: Converting JSON to Relational Rows

```
SELECT jt.category
FROM customer_profiles,
JSON_TABLE(
    preferences, '$.favorite_categories[*]'
    COLUMNS (category VARCHAR(50) PATH '$')
) AS jt;
```

JSON_TABLE() (MySQL 8.0.4+) expands a JSON array/object into ordinary relational rows, letting JSON data be queried, joined, and aggregated using normal SQL; conceptually parallel to PostgreSQL's jsonb_to_recordset() and jsonb_array_elements(), though the specific syntax (a COLUMNS clause with PATH expressions) is quite different from PostgreSQL's set-returning function style.

24.7 JSON Validation

```
SELECT JSON_VALID('{"a": 1}');
SELECT JSON_VALID('not json');
```

Returns 1 or 0; useful when validating JSON coming from an untrusted or TEXT-typed source column before attempting to cast or insert it as JSON.

24.8 Indexing JSON Data

```
ALTER TABLE customer_profiles
ADD COLUMN newsletter_opt_in BOOLEAN
GENERATED ALWAYS AS (preferences->>'$.newsletter' = 'true') STORED;

CREATE INDEX idx_newsletter ON customer_profiles(newsletter_opt_in);
```

Critical PostgreSQL Comparison: MySQL cannot directly index a JSON column or an arbitrary JSON path expression the way PostgreSQL can with a native GIN index on a JSONB column (CREATE INDEX ON table USING GIN(jsonb_column)). Instead, MySQL requires creating a GENERATED (virtual or stored) column that extracts the specific JSON value you need to index, and then indexing THAT generated column. This is a genuinely more manual, less flexible process than PostgreSQL's direct JSONB/GIN indexing, and is one of the more significant capability gaps between the two systems for JSON-heavy workloads.

24.9 Realistic Examples: API Responses and Event Data

```
CREATE TABLE api_events (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
    event_type VARCHAR(50),
    payload JSON,
    received_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

SELECT
    event_type,
    payload->>'$.user_id' AS user_id,
    payload->>'$.action' AS action
FROM api_events
WHERE event_type = 'user_activity'
AND payload->>'$.action' = 'purchase';
```

Storing raw, variably-shaped API/event payloads as JSON and extracting specific fields as needed is a very common pattern in both MySQL and PostgreSQL applications, letting a schema accommodate semi-structured data without a rigid, constantly-migrating column-per-field design.

25. Regular Expressions

25.1 Pattern Matching With REGEXP

```
SELECT * FROM customers WHERE phone REGEXP '^[0-9]{3}-[0-9]{3}-[0-9]{4}$';
SELECT * FROM products WHERE name REGEXP 'Pro|Premium|Deluxe';
```

As introduced in Part 7.3, REGEXP (a synonym for RLIKE) tests whether a string matches a regular expression pattern, returning a boolean. MySQL 8.0.4+ uses the ICU regex library, giving it modern, fairly full-featured regex syntax; PostgreSQL uses the ~ operator for case-sensitive matching and ~* for case-insensitive, a symbolic rather than keyword-based syntax, one of the more visible day-to-day syntax differences between the two systems for pattern matching.

25.2 REGEXP_LIKE (Explicit Function Form)

```
SELECT * FROM customers WHERE REGEXP_LIKE(email, '^[a-z]+@company\\.com$', 'i');
```

The explicit function form of REGEXP, additionally supporting a match-type flag ('i' for case-insensitive here); functionally equivalent to the REGEXP operator, added for clarity and standard-SQL-style consistency.

25.3 REGEXP_REPLACE

```
SELECT REGEXP_REPLACE(phone, '[^0-9]', '') AS digits_only
FROM customers;

UPDATE products
SET description = REGEXP_REPLACE(description, '\\s+', ' ')
WHERE description REGEXP '\\s{2,}';
```

Replaces every match of a pattern with a replacement string; a genuinely powerful data-cleaning tool, here stripping all non-digit characters from a phone number, and collapsing repeated whitespace down to single spaces. PostgreSQL's equivalent is REGEXP_REPLACE() too, with a very similar signature, one of the rarer cases of near-identical naming between the two systems for a regex function.

25.4 REGEXP_SUBSTR

```
SELECT REGEXP_SUBSTR(email, '@(.+)$') AS domain_part
FROM customers;
```

Extracts and returns the portion of a string matching a pattern, rather than a true/false result; useful for pulling out a specific substructure (like a domain from an email) directly, without needing SUBSTRING combined with a separately computed position.

25.5 Practical Data-Cleaning Examples

```
-- Find records with suspicious/invalid formatting
SELECT customer_id, email FROM customers
WHERE NOT email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$';

-- Standardize inconsistent phone number formatting
```

```
UPDATE customers
SET phone = REGEXP_REPLACE(phone, '[^0-9]', '')
WHERE phone REGEXP '[^0-9]';

-- Extract a product SKU embedded within a longer description string
SELECT REGEXP_SUBSTR(description, '[A-Z]{3}-[0-9]{4}') AS sku
FROM products
WHERE description REGEXP '[A-Z]{3}-[0-9]{4}';
```

These three patterns, finding malformed data, normalizing inconsistent formatting, and extracting embedded structured values from free text, cover the overwhelming majority of real-world regex use in data cleaning work, and are directly reusable in the Data Analytics and Data Engineering contexts covered in Parts 34 and 36.

26. Temporary Tables and Derived Tables

26.1 Temporary Tables

```
CREATE TEMPORARY TABLE temp_high_value_orders AS
SELECT * FROM orders WHERE total > 1000;

SELECT customer_id, COUNT(*) FROM temp_high_value_orders GROUP BY customer_id;

DROP TEMPORARY TABLE temp_high_value_orders;
```

Code Line	Explanation
`CREATE TEMPORARY TABLE`	Creates a table visible ONLY to the current session/connection, automatically dropped when that session ends; fully portable concept and near-identical syntax to PostgreSQL's own CREATE TEMPORARY TABLE (also usable as CREATE TEMP TABLE in PostgreSQL as a shorthand).
`Practical use case`	Staging complex intermediate results for a multi-step analysis, or breaking a very large, deeply nested query into several simpler, debuggable steps, without permanently altering the schema.

26.2 Derived Tables

```
SELECT region, AVG(region_total) AS avg_regional_total
FROM (
    SELECT region, customer_id, SUM(total) AS region_total
    FROM orders
    GROUP BY region, customer_id
) AS customer_totals
GROUP BY region;
```

A derived table is a subquery used directly in the FROM clause, given an alias (customer_totals here) so its columns can be referenced by the outer query; fully portable, identical concept to PostgreSQL. Unlike a CTE, a derived table's definition is embedded directly at its point of use rather than named separately at the top of the query.

26.3 Temporary Tables vs Derived Tables vs CTEs

Code Line	Explanation
`Temporary table`	Persists for the entire session, can be indexed, can be reused across MULTIPLE separate queries within that session, and can be built up incrementally with several separate INSERT statements. Best for genuinely complex, multi-step, reusable intermediate work.
`Derived table (subquery in FROM)`	Scoped to a single query, computed fresh every time that query runs, cannot be indexed independently. Best for a one-off transformation needed only within one specific query.
`CTE (WITH ...)`	Also scoped to a single query, but named upfront and often more readable than an equivalent derived table, especially when referenced multiple times within the same query or chained with other CTEs (Part 12). In MySQL 8.x, a non-recursive CTE referenced only once

Code Line	Explanation
	is typically treated similarly to a derived table internally by the optimizer.

This exact three-way decision framework, and the underlying concepts, are identical between MySQL and PostgreSQL; only the availability of CTEs (MySQL 8.0+ only, per Part 12.1) differs by version.

26.4 A Practical Analytics Example Using All Three

```
-- Step 1: build a temporary table of this quarter's orders
CREATE TEMPORARY TABLE q1_orders AS
SELECT * FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31';

-- Step 2: use a CTE for a clear, named aggregation on top of it
WITH customer_q1_totals AS (
    SELECT customer_id, SUM(total) AS q1_spend
    FROM q1_orders
    GROUP BY customer_id
)
-- Step 3: a derived table to rank within the final SELECT
SELECT customer_id, q1_spend
FROM (
    SELECT customer_id, q1_spend,
           RANK() OVER (ORDER BY q1_spend DESC) AS spend_rank
    FROM customer_q1_totals
) ranked
WHERE spend_rank <= 10;
```

A realistic layered analysis combining all three techniques: a temporary table isolating the relevant time period once (avoiding repeating that date filter), a CTE for clear, named aggregation, and a derived table for the final ranking and filtering step; this staged approach to complex analytical queries is a genuinely valuable professional habit in both MySQL and PostgreSQL.

27. Database Design and Normalization

Database design theory, entity-relationship modeling, and normalization rules are universal relational database concepts, fully portable knowledge that transfers completely unchanged from PostgreSQL. This part is a focused recap emphasizing MySQL-specific implementation details (data types, AUTO_INCREMENT, engine choice) within an otherwise portable design process.

27.1 ER Diagrams: Entities, Attributes, Relationships

Code Line	Explanation
`Entity`	A real-world object or concept to be modeled as a table (Customer, Order, Product).
`Attribute`	A property of an entity, modeled as a column (customer_name, order_date).
`Relationship`	How entities connect (a Customer PLACES an Order; an Order CONTAINS Products); modeled with foreign keys, and for many-to-many relationships, a junction table (Part 10.5).
`Cardinality`	The numeric nature of a relationship: one-to-one, one-to-many, or many-to-many (Part 10.5).

MySQL Workbench's visual ER diagram designer (mentioned in Part 2.7) is a genuinely useful tool for this process, letting you draw entities and relationships visually and generate the corresponding CREATE TABLE statements automatically, similar in spirit to pgModeler or other PostgreSQL-focused ER tools.

27.2 Normalization: 1NF, 2NF, 3NF, BCNF (Recap)

Code Line	Explanation
`1NF (First Normal Form)`	Every column holds a single, atomic value (no repeating groups or comma-separated lists crammed into one column); every row is uniquely identifiable.
`2NF (Second Normal Form)`	Already in 1NF, and every non-key column depends on the ENTIRE primary key, not just part of a composite key (relevant specifically when a table has a composite primary key).
`3NF (Third Normal Form)`	Already in 2NF, and every non-key column depends ONLY on the primary key, not on another non-key column (no transitive dependencies).
`BCNF (Boyce-Codd Normal Form)`	A stricter version of 3NF handling certain edge cases involving multiple overlapping candidate keys.

These rules and their reasoning are identical in every relational database; nothing here is MySQL-specific. The one MySQL-specific implementation detail worth noting is that MySQL's ENUM and SET types (Part 4.6) can sometimes tempt a design toward storing what should really be a separate, normalized lookup table as a single denormalized column instead, worth resisting for any value set that might grow or change over time.

27.3 Denormalization: Practical Trade-offs

- Denormalization (deliberately storing redundant or precomputed data) trades some normalization purity for read performance, avoiding expensive joins or aggregations on frequently-run queries.

- A common MySQL-specific denormalization pattern, given the lack of native materialized views (Part 15.5), is a scheduled Event (Part 23) that periodically rebuilds a denormalized summary table; PostgreSQL would more often reach for a native materialized view for the same goal.
- Denormalized data requires deliberate maintenance (via triggers, application logic, or scheduled jobs) to avoid becoming stale or inconsistent with its source data; this maintenance burden is the primary cost to weigh against the read-performance benefit, identical tradeoff reasoning in both databases.
- A reasonable rule of thumb, portable across both systems: design fully normalized first, and denormalize deliberately and selectively only where a specific, measured performance need justifies it, rather than denormalizing preemptively.

28. MySQL Users and Security

28.1 CREATE USER

```
CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'StrongP@ssw0rd!';
CREATE USER 'app_user'@'%' IDENTIFIED BY 'AnotherStr0ngP@ss!';
```

Code Line	Explanation
`'analyst'@'localhost'`	MySQL usernames are ALWAYS paired with a host specification, defining WHERE that user is allowed to connect from; 'localhost' means only from the same machine. This host-bound identity is a genuine structural difference from PostgreSQL, where a role/user is a single, host-independent identity, and connection origin is instead controlled separately via the pg_hba.conf file.
`'app_user'@'%'`	The % wildcard host allows connection from any host; convenient but should be restricted to specific IP ranges in production for security.
`IDENTIFIED BY 'password'`	Sets the authentication password directly in the statement; MySQL-specific syntax placement (PostgreSQL uses CREATE ROLE ... WITH LOGIN PASSWORD '...', a similar idea with different keyword structure).

Critical PostgreSQL Comparison: Because MySQL usernames are tied to a specific host, 'analyst'@'localhost' and 'analyst'@'%' are two ENTIRELY SEPARATE users as far as MySQL's privilege system is concerned, even though they share the same username string. This trips up many developers coming from PostgreSQL, where a role name alone is the complete identity.

28.2 ALTER USER

```
ALTER USER 'analyst'@'localhost' IDENTIFIED BY 'NewStrongerP@ss1!';
ALTER USER 'analyst'@'localhost' PASSWORD EXPIRE;
```

28.3 GRANT and REVOKE

```
GRANT SELECT ON company_db.* TO 'analyst'@'localhost';
GRANT SELECT, INSERT, UPDATE ON company_db.orders TO 'app_user'@'%';
GRANT ALL PRIVILEGES ON company_db.* TO 'admin_user'@'localhost';

REVOKE INSERT ON company_db.orders FROM 'app_user'@'%';

FLUSH PRIVILEGES;
```

Code Line	Explanation
`GRANT ... ON database.table TO user@host`	Fully portable concept, identical purpose to PostgreSQL's GRANT; syntax is close but MySQL's privilege target format is always database.table (or database.* for every table), while PostgreSQL uses a more granular, schema-aware GRANT ... ON TABLE/SCHEMA/DATABASE syntax reflecting its multi-schema-per-database model (Part 3.1).
`FLUSH PRIVILEGES;`	In older MySQL versions, this was required after directly editing the

Code Line	Explanation
	underlying grant tables to reload them into memory; when using GRANT/REVOKE statements themselves (as shown here), it is generally unnecessary in modern MySQL, since GRANT/REVOKE apply immediately, but it remains a common defensive habit. PostgreSQL has no equivalent concept at all, since it has no comparable in-memory privilege cache to refresh.

28.4 Roles (MySQL 8.0+)

```
CREATE ROLE 'read_only_role';
GRANT SELECT ON company_db.* TO 'read_only_role';
GRANT 'read_only_role' TO 'analyst'@'localhost';
SET DEFAULT ROLE 'read_only_role' TO 'analyst'@'localhost';
```

MySQL 8.0 introduced proper ROLE support, letting a set of privileges be granted as a group to multiple users at once; conceptually now very close to PostgreSQL's long-standing role system, though PostgreSQL's roles have always doubled as both users and groups in a single unified concept, while MySQL keeps 'user' (identity, can log in) and 'role' (a named privilege bundle) as more clearly separate concepts.

28.5 Least Privilege and Application User Design

- Create a DEDICATED database user per application/service, never share the root/admin account with application code.
- Grant only the specific privileges (SELECT, INSERT, UPDATE, DELETE) on only the specific tables an application genuinely needs, never GRANT ALL PRIVILEGES to an application user.
- Create a separate READ-ONLY user for reporting, analytics, and BI tools (Part 34), so a runaway or malformed analytical query can never accidentally modify production data.
- Reserve a genuine ADMIN user, used only by database administrators for schema changes and maintenance, entirely separate from any application-facing user.

This least-privilege design philosophy is identical in both MySQL and PostgreSQL; only the specific GRANT syntax to implement it differs.

28.6 Secure Passwords and Authentication

MySQL 8.0 defaults to the caching_sha2_password authentication plugin (a meaningful security improvement over the older, weaker mysql_native_password used by default in earlier versions); always use strong, unique passwords, and prefer connecting over SSL/TLS for any non-local connection, especially across a network to a remote server, exactly as you would configure PostgreSQL's sslmode.

28.7 SQL Injection and Parameterized Queries

```
-- VULNERABLE: building SQL by directly concatenating user input
-- query = "SELECT * FROM users WHERE username = '" + user_input + "'"

-- SAFE: a parameterized query with a placeholder
-- SELECT * FROM users WHERE username = %s   (driver substitutes safely)
```

SQL injection, and its prevention via parameterized (prepared) queries, is a universal concept identical in every SQL database, including MySQL and PostgreSQL alike; the underlying vulnerability (untrusted input directly concatenated into SQL text) and the fix (letting the driver bind parameters safely, never string-formatting user input into a query) are completely portable. Part 32 demonstrates this concretely with real Python driver code.

29. MySQL Administration Concepts

Presented from the perspective of a Data Analyst/Data Scientist/Data Engineer who needs enough operational understanding to work effectively with a production MySQL system, diagnose problems, and communicate with database administrators, not a full DBA certification.

29.1 Server Architecture Overview

mysqld (the server process) manages client connections, parses and optimizes SQL, and coordinates with the active storage engine(s) (Part 18) to actually read/write data. This overall shape, a single server process handling many client connections, is conceptually identical to PostgreSQL's postmaster process and its per-connection worker processes.

29.2 Connections and Threads

```
SHOW PROCESSLIST;
SHOW STATUS LIKE 'Threads_connected';
SHOW VARIABLES LIKE 'max_connections';
```

MySQL uses one THREAD per client connection (versus PostgreSQL's one PROCESS per connection, a genuine internal architecture difference, though functionally similar from a user's perspective). SHOW PROCESSLIST lists currently active connections and their running queries, useful for diagnosing what is currently happening on a busy server; the direct PostgreSQL equivalent is querying the pg_stat_activity system view.

29.3 The InnoDB Buffer Pool

```
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
```

The buffer pool is InnoDB's in-memory cache for table and index data, the single most impactful tuning parameter for InnoDB performance; conceptually directly parallel to PostgreSQL's shared_buffers setting, serving the same fundamental purpose (keep frequently accessed data in RAM rather than reading from disk repeatedly).

29.4 Binary Logs, Redo Logs, Undo Logs

Code Line	Explanation
`Binary log (binlog)`	Records every data-modifying statement/change for replication (Part 39) and point-in-time recovery. This is MySQL's own distinct log, roughly parallel in PURPOSE to PostgreSQL's write-ahead log (WAL) used for replication, though the two are technically different mechanisms serving overlapping but not identical roles.
`Redo log`	InnoDB's crash-recovery log, ensuring committed transactions survive a crash (durability, Part 19.2); conceptually parallel to PostgreSQL's WAL serving its own crash-recovery role.
`Undo log`	InnoDB's mechanism for MVCC (Part 18.5) and transaction rollback, storing the information needed to reverse uncommitted changes or reconstruct an earlier row version for a concurrent reader; conceptually parallel to how PostgreSQL retains old row versions directly in the table heap for the same MVCC purpose, though the storage mechanism differs meaningfully (a separate log vs in-heap old versions).

29.5 Checkpoints

A checkpoint periodically flushes modified in-memory buffer pool pages to disk, bounding how much redo log needs to be replayed during crash recovery; conceptually identical purpose to PostgreSQL's own checkpoint mechanism.

29.6 Replication Concepts (Preview)

A brief preview before the full, dedicated treatment in Part 39: MySQL replication copies the binary log's stream of changes from a source server to one or more replica servers, keeping them synchronized; conceptually very similar in goal to PostgreSQL's streaming replication based on shipping WAL records, with different underlying log formats.

29.7 Monitoring

```
SHOW GLOBAL STATUS;
SHOW ENGINE INNODB STATUS;
```

SHOW GLOBAL STATUS exposes a large set of server performance counters (queries per second, cache hit rates, connection counts); SHOW ENGINE INNODB STATUS provides a detailed InnoDB-specific internal report (buffer pool stats, current transactions, lock waits). Production monitoring typically uses a dedicated tool (Percona Monitoring and Management, MySQL Enterprise Monitor, or general tools like Prometheus with a MySQL exporter) built on top of these underlying data sources, conceptually parallel to the pg_stat_* views and tools like pgAdmin/pganalyze used for PostgreSQL monitoring.

29.8 Server Configuration

MySQL's server configuration lives in my.cnf/my.ini (Part 2.6), with settings like innodb_buffer_pool_size, max_connections, and query cache settings tuned based on workload and available hardware; conceptually parallel to PostgreSQL's postgresql.conf, with entirely different variable names throughout.

30. Backup and Restore

30.1 mysqldump: Logical Backups

```
mysqldump -u root -p company_db > company_db_backup.sql
mysqldump -u root -p --all-databases > full_backup.sql
mysqldump -u root -p company_db orders customers > partial_backup.sql
```

Code Line	Explanation
`mysqldump`	MySQL's built-in logical backup tool, producing a plain-text SQL file of CREATE TABLE and INSERT statements that can fully recreate the database; conceptually and functionally parallel to PostgreSQL's pg_dump, with a different command-line tool name and flag set.
`--all-databases`	Dumps every database on the server, including system schemas; useful for a complete server backup.
`Specifying table names after the database name`	Dumps only specific tables, useful for partial backups or extracting a subset of data for a separate environment.

30.2 Restoring a Database

```
mysql -u root -p company_db < company_db_backup.sql

-- Or from within the mysql client:
SOURCE company_db_backup.sql;
```

Restoring a mysqldump file is simply piping/redirecting it back into the mysql client against a target (typically newly created, empty) database; conceptually parallel to restoring a pg_dump plain-SQL file with psql, though PostgreSQL also commonly uses pg_dump's custom binary format together with pg_restore for more flexible, parallel restores, a workflow MySQL's plain mysqldump does not offer in quite the same way.

30.3 Exporting and Importing Specific Tables

```
mysqldump -u root -p --no-create-info company_db orders > orders_data_only.sql
mysqldump -u root -p --no-data company_db > schema_only.sql
```

Code Line	Explanation
`--no-create-info`	Dumps only the INSERT data statements, skipping table structure; useful when the schema already exists and only fresh data needs loading.
`--no-data`	Dumps only the CREATE TABLE structure, skipping all data; useful for replicating an empty schema to a new environment.

30.4 Backup Strategies

- Logical backups (mysqldump) are portable across MySQL versions and even to other systems reading standard SQL, but are slower to restore for very large databases since every row must be re-inserted and every index rebuilt from scratch.

- Physical backups (copying InnoDB's actual data files, typically via the specialized Percona XtraBackup tool for a live, non-blocking backup) are much faster to restore for large databases but are tied to the exact MySQL version and configuration they were taken from; conceptually parallel to PostgreSQL's own physical backup tools like pg_basebackup.
- A robust production strategy combines regular full backups with binary log retention (Part 29.4) enabling point-in-time recovery to any specific moment since the last full backup, conceptually identical in approach to a PostgreSQL WAL-archiving-based point-in-time recovery strategy.
- Backups must be tested by actually performing a restore periodically; an untested backup is not a reliable backup, in either database system.

30.5 Why Backup and Disaster Recovery Matter

Hardware failure, accidental DROP TABLE/DELETE without a WHERE clause, application bugs, and malicious attacks are all genuine, common risks to any production database; a tested, current backup combined with a documented, rehearsed restore procedure is the difference between a contained incident and a catastrophic, unrecoverable data loss event. This operational discipline is identical in importance and philosophy regardless of which specific RDBMS is in use.

31. Importing and Exporting Data

31.1 LOAD DATA INFILE

```
LOAD DATA INFILE '/var/lib/mysql-files/orders.csv'
INTO TABLE orders
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(order_id, customer_id, order_date, total);
```

Code Line	Explanation
`LOAD DATA INFILE`	MySQL's high-performance bulk CSV/text loader, dramatically faster than an equivalent series of INSERT statements for large files, since it bypasses much of the normal per-row SQL parsing overhead.
`FIELDS TERMINATED BY / ENCLOSED BY`	Define the CSV's delimiter and quote character.
`IGNORE 1 ROWS`	Skips the header row.
`The column list at the end`	Maps CSV columns to table columns in order; required whenever the CSV's column order does not exactly match the table's own column order, or when only some columns are being loaded.

PostgreSQL Comparison: LOAD DATA INFILE (without LOCAL) requires the file to already exist on the SERVER's filesystem, in a directory MySQL is configured to trust (secure_file_priv). This is a genuinely different model from PostgreSQL's COPY command, which behaves similarly for server-side files but which more commonly is used via psql's \copy meta-command, letting the file live on the CLIENT machine transparently. MySQL's client-side equivalent is LOAD DATA LOCAL INFILE, covered next.

31.2 LOAD DATA LOCAL INFILE

```
LOAD DATA LOCAL INFILE '/Users/analyst/Downloads/orders.csv'
INTO TABLE orders
FIELDS TERMINATED BY ','
IGNORE 1 ROWS;
```

Code Line	Explanation
`LOCAL`	Reads the file from the CLIENT machine (wherever the mysql client itself is running) rather than requiring it to already be on the server; this is the practical, everyday choice for a data analyst loading a CSV from their own laptop.
`Security note`	LOAD DATA LOCAL INFILE must be explicitly enabled (both server-side via local_infile=1 and often client-side via a connection flag) since it technically allows a malicious server to request arbitrary client files in certain attack scenarios; it is disabled by default in many modern client configurations for this reason, and may need to be explicitly enabled to use it.

31.3 Bulk Loading Performance Tips

- Temporarily disable secondary indexes before a very large bulk load, and rebuild them afterward, since maintaining indexes row-by-row during a huge insert is often slower than a bulk index rebuild at the end.
- Wrap large multi-file loads in an explicit transaction where appropriate, to avoid excessive per-row commit overhead.
- LOAD DATA INFILE is dramatically faster than row-by-row INSERT statements or even large multi-row INSERT batches, for the largest files, often by an order of magnitude or more, mirroring the equivalent performance advantage PostgreSQL's COPY command has over INSERT.

31.4 Exporting Query Results

```
SELECT * FROM orders WHERE order_date >= '2026-01-01'
INTO OUTFILE '/var/lib/mysql-files/2026_orders.csv'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

SELECT ... INTO OUTFILE exports a query's result directly to a server-side CSV file; like LOAD DATA INFILE, it writes to the SERVER's filesystem by default, requiring appropriate permissions and secure_file_priv configuration. For exporting to the CLIENT machine directly, the more common practical approach is running the query through the mysql command-line client with output redirection, or using MySQL Workbench's built-in 'Export Resultset' feature, or a Python/pandas-based export (Part 33), which naturally writes to wherever the script itself is running.

32. MySQL + Python

32.1 Choosing a Driver: mysql-connector-python vs PyMySQL vs SQLAlchemy

Code Line	Explanation
`mysql-connector-python`	Oracle's own official, pure-Python MySQL driver; no external C dependencies, straightforward to install anywhere pip works.
`PyMySQL`	A popular, mature, pure-Python community driver with a very similar API; a common alternative when a project prefers it over the official Oracle driver.
`SQLAlchemy`	A higher-level toolkit and ORM that sits ON TOP of a lower-level driver (like mysql-connector-python or PyMySQL) and can also connect to PostgreSQL, SQLite, and others through the exact same Python API, differing only in its connection string; genuinely valuable for code that needs to remain portable across different database backends, or that benefits from ORM-style object mapping.

32.2 Connecting and Running a Query (mysql-connector-python)

```
import mysql.connector

conn = mysql.connector.connect(
    host='localhost',
    user='analyst',
    password='StrongP@ssw0rd!',
    database='company_db'
)

cursor = conn.cursor()
cursor.execute('SELECT customer_id, customer_name FROM customers LIMIT 5')
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

Code Line	Explanation
`mysql.connector.connect(...)`	Opens a connection using the standard Python DB-API 2.0 interface, the same general pattern used by psycopg2 for PostgreSQL, just with different connection parameter names and a different import.
`conn.cursor()`	Creates a cursor object used to execute statements and fetch results, identical concept to a psycopg2 cursor.
`cursor.fetchall()`	Retrieves every remaining row from the last executed query as a list of tuples; identical behavior to PostgreSQL drivers.

32.3 Parameterized Queries (Critical for SQL Injection Prevention)

```
customer_id = 42
cursor.execute(
```

```
    'SELECT * FROM orders WHERE customer_id = %s',
    (customer_id,)
)
```

Code Line	Explanation
`%s as the placeholder`	mysql-connector-python and PyMySQL both use %s as the parameter placeholder for ANY data type (not just strings), a genuinely different convention from psycopg2 (PostgreSQL's most common driver), which ALSO uses %s, so this specific detail happens to carry over directly. SQLAlchemy's own text()-based parameter binding instead uses a named :param_name style regardless of the underlying database, one more reason SQLAlchemy's abstraction is convenient for portable code.
`The tuple (customer_id,)`	Always pass parameters as a tuple (or list), even for a single value, note the trailing comma required to make Python treat it as a genuine one-element tuple rather than a plain parenthesized expression.
`Why this prevents SQL injection`	The driver sends the query text and the parameter value SEPARATELY to the server, so user input can never be interpreted as SQL syntax, no matter what characters it contains; directly implementing the safe pattern described conceptually in Part 28.7.

32.4 Transactions From Python

```
try:
    conn.start_transaction()
    cursor.execute('UPDATE accounts SET balance = balance - %s WHERE account_id =
%s', (100, 1))
    cursor.execute('UPDATE accounts SET balance = balance + %s WHERE account_id =
%s', (100, 2))
    conn.commit()
except mysql.connector.Error as e:
    conn.rollback()
    print(f'Transaction failed: {e}')
```

Explicit transaction control from Python, directly mirroring the SQL-level START TRANSACTION/COMMIT/ROLLBACK from Part 19.1; the try/except/rollback pattern here is a direct application of the exception handling principles from your dedicated Exception Handling course material, adapted specifically to database transaction safety.

32.5 Connection Pooling

```
from mysql.connector import pooling

pool = pooling.MySQLConnectionPool(
    pool_name='mypool',
    pool_size=5,
    host='localhost',
    user='app_user',
    password='...',
    database='company_db'
)
```

```
conn = pool.get_connection()
# ... use conn ...
conn.close()  # returns the connection to the pool, does not actually disconnect
```

A connection pool maintains a set of already-open, reusable connections, avoiding the overhead of establishing a brand new TCP connection and authentication handshake for every single request in a busy application; conceptually identical purpose to PostgreSQL connection pooling (commonly implemented via PgBouncer, since psycopg2 itself does not include built-in pooling the way mysql-connector-python does natively).

32.6 SQLAlchemy: A Portable Alternative

```
from sqlalchemy import create_engine, text

engine = create_engine('mysql+mysqlconnector://analyst:StrongP%40ssw0rd
%21@localhost/company_db')

with engine.connect() as conn:
    result = conn.execute(text('SELECT customer_id, customer_name FROM customers
LIMIT 5'))
    for row in result:
        print(row)
```

Code Line	Explanation
`'mysql+mysqlconnector://...'`	SQLAlchemy's connection string format: dialect+driver://user:password@host/database. Changing to PostgreSQL requires only changing this ONE string to 'postgresql+psycopg2://...', with every other line of Python code remaining completely unchanged, the core practical benefit SQLAlchemy offers for portability.
`%40 and %21 in the password`	URL-encoded versions of @ and ! respectively; any special character in a password used inside a connection URL string must be percent-encoded to avoid being misinterpreted as part of the URL's own syntax.

33. MySQL + pandas

33.1 pandas.read_sql

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine('mysql+mysqlconnector://analyst:password@localhost/company_db')

df = pd.read_sql('SELECT * FROM orders WHERE order_date >= "2026-01-01"', engine)
print(df.head())
```

Code Line	Explanation
`pd.read_sql(query, engine)`	Runs the given SQL query and loads the entire result directly into a pandas DataFrame; requires a SQLAlchemy engine (Part 32.6) as the connection, exactly the same pattern used for PostgreSQL, since pandas itself is fully database-agnostic once handed a SQLAlchemy engine.

33.2 Reading Large Datasets: Chunking

```
chunks = pd.read_sql('SELECT * FROM orders', engine, chunksize=50000)

total_revenue = 0
for chunk in chunks:
    total_revenue += chunk['total'].sum()

print(total_revenue)
```

chunksize=50000 returns an iterator yielding the result in batches of 50,000 rows rather than loading the entire (potentially huge) table into memory at once, directly applying the lazy-evaluation and memory-efficiency principles from your Functional Programming course material to a real database-loading scenario.

33.3 DataFrame.to_sql: Writing Results Back

```
summary_df = df.groupby('region')['total'].sum().reset_index()

summary_df.to_sql(
    'regional_revenue_summary',
    engine,
    if_exists='replace',
    index=False
)
```

Code Line	Explanation
`if_exists='replace'`	Drops and recreates the target table if it already exists, replacing its content entirely with the DataFrame's current data; other options are 'append' (add rows to an existing table) and 'fail' (raise an error if the

Code Line	Explanation
	table already exists).
`index=False`	Prevents pandas from writing its own DataFrame row index as an extra, unwanted column in the database table.

33.4 Data Type Conversion Considerations

- pandas' NaN (used for missing numeric data) maps to SQL NULL when writing with to_sql(); the reverse is also true when reading NULL values back with read_sql().
- MySQL's DATETIME/TIMESTAMP columns are read into pandas as proper datetime64 dtype automatically in most cases; verify with df.dtypes after loading, and use pd.to_datetime() explicitly if a date column was read in as a plain object/string instead.
- MySQL's DECIMAL columns are sometimes read into pandas as the Python Decimal object type rather than a native float, to preserve exact precision; this may require an explicit .astype(float) conversion before certain numeric operations, a detail worth checking with df.dtypes.

33.5 A Complete Data Analytics Workflow

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine('mysql+mysqlconnector://analyst:password@localhost/company_db')

query = '''
    SELECT o.order_id, o.order_date, o.total, c.region
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.order_date >= '2026-01-01'
'''
df = pd.read_sql(query, engine)

df['month'] = df['order_date'].dt.to_period('M')
monthly_by_region = df.groupby(['month', 'region'])['total'].sum().reset_index()

monthly_by_region.to_sql('monthly_region_revenue', engine, if_exists='replace',
index=False)
print(monthly_by_region.head())
```

A complete, realistic round trip: pull joined, filtered data from MySQL into pandas, perform analysis pandas is well-suited for (deriving a month period and grouping), then write the analytical result back into MySQL as a new table, ready for a BI tool or dashboard to consume directly; this exact pattern is the foundation of Part 34's deeper Data Analytics coverage.

34. MySQL for Data Analytics

34.1 Revenue, Profit, and Core KPIs

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    SUM(total) AS revenue,
    SUM(total - cost) AS profit,
    ROUND(SUM(total - cost) / SUM(total) * 100, 2) AS profit_margin_pct
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

34.2 Customer Lifetime Value and RFM Analysis

```
WITH rfm AS (
    SELECT
        customer_id,
        DATEDIFF(CURDATE(), MAX(order_date)) AS recency_days,
        COUNT(*) AS frequency,
        SUM(total) AS monetary
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id, recency_days, frequency, monetary,
    NTILE(5) OVER (ORDER BY recency_days DESC) AS r_score,
    NTILE(5) OVER (ORDER BY frequency) AS f_score,
    NTILE(5) OVER (ORDER BY monetary) AS m_score
FROM rfm;
```

Code Line	Explanation
`Recency, Frequency, Monetary`	The three classic RFM dimensions: how recently a customer purchased, how often, and how much; computed here directly from the orders table using DATEDIFF (Part 8.3), COUNT, and SUM.
`NTILE(5)`	Splits customers into 5 quintiles per dimension (Part 13.3), the standard RFM scoring technique; combining the three digit scores (for example, 5-5-5 for the best customers, 1-1-1 for the least engaged) produces a simple, actionable segmentation used constantly in real marketing analytics.

34.3 Churn and Retention

```
WITH monthly_activity AS (
    SELECT DISTINCT customer_id, DATE_FORMAT(order_date, '%Y-%m') AS month
    FROM orders
)
SELECT
    curr.month,
```

```
    COUNT(DISTINCT curr.customer_id) AS active_customers,
    COUNT(DISTINCT prev.customer_id) AS retained_from_prev_month
FROM monthly_activity curr
LEFT JOIN monthly_activity prev
    ON curr.customer_id = prev.customer_id
    AND prev.month = DATE_FORMAT(DATE_SUB(STR_TO_DATE(CONCAT(curr.month, '-01'),
'%Y-%m-%d'), INTERVAL 1 MONTH), '%Y-%m')
GROUP BY curr.month
ORDER BY curr.month;
```

A month-over-month retention calculation: for every customer active in the current month, check whether they were ALSO active in the immediately preceding month; STR_TO_DATE combined with DATE_SUB handles the month arithmetic needed to compute 'the previous month' as a string, a genuinely fiddly but common real-world analytics pattern.

34.4 Cohort Analysis

```
WITH first_purchase AS (
    SELECT customer_id, MIN(DATE_FORMAT(order_date, '%Y-%m')) AS cohort_month
    FROM orders
    GROUP BY customer_id
)
SELECT
    fp.cohort_month,
    DATE_FORMAT(o.order_date, '%Y-%m') AS activity_month,
    COUNT(DISTINCT o.customer_id) AS active_customers
FROM orders o
JOIN first_purchase fp ON o.customer_id = fp.customer_id
GROUP BY fp.cohort_month, activity_month
ORDER BY fp.cohort_month, activity_month;
```

Groups customers by the month of their FIRST purchase (their cohort), then tracks how many of each cohort remain active in every subsequent month, the foundation of a cohort retention curve/heatmap, one of the most valuable and commonly requested analyses in subscription and e-commerce businesses alike.

34.5 Funnel Analysis

```
SELECT
    COUNT(DISTINCT CASE WHEN stage = 'viewed' THEN user_id END) AS viewed,
    COUNT(DISTINCT CASE WHEN stage = 'added_to_cart' THEN user_id END) AS
added_to_cart,
    COUNT(DISTINCT CASE WHEN stage = 'purchased' THEN user_id END) AS purchased
FROM funnel_events;
```

Using conditional COUNT(DISTINCT CASE WHEN ...) to compute how many distinct users reached each stage of a funnel in a single query; this pattern (a conditional aggregate per funnel stage) is fully portable SQL, identical in PostgreSQL.

34.6 Monthly Growth and Running Totals

```
WITH monthly AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total) AS revenue
```

```
    FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    month, revenue,
    SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue,
    ROUND((revenue - LAG(revenue) OVER (ORDER BY month)) / LAG(revenue) OVER (ORDER
BY month) * 100, 2) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

34.7 Pareto Analysis (the 80/20 Rule)

```
WITH customer_revenue AS (
    SELECT customer_id, SUM(total) AS revenue
    FROM orders GROUP BY customer_id
),
ranked AS (
    SELECT
        customer_id, revenue,
        SUM(revenue) OVER (ORDER BY revenue DESC) / SUM(revenue) OVER () * 100 AS
cumulative_pct
    FROM customer_revenue
)
SELECT COUNT(*) AS customers_for_80pct_revenue
FROM ranked WHERE cumulative_pct <= 80;
```

Ranks customers by revenue descending and computes what fraction of total revenue is accumulated as you move down that ranked list, directly answering 'how many of our customers account for 80 percent of revenue'; a classic senior-analyst-level window function application.

35. MySQL for Data Science

35.1 The Role of MySQL in a Data Science Workflow

MySQL typically serves as the SOURCE of raw or lightly-processed data for a Data Scientist: extracting a training dataset, joining relevant feature tables together, and applying initial filtering and aggregation, all pushed down into SQL, before handing the result off to pandas and a modeling library for anything requiring more flexible numerical computation.

35.2 Dataset Creation and Extraction

```
SELECT
    c.customer_id,
    c.age,
    c.signup_channel,
    COUNT(o.order_id) AS total_orders,
    COALESCE(SUM(o.total), 0) AS total_spent,
    DATEDIFF(CURDATE(), MAX(o.order_date)) AS days_since_last_order,
    CASE WHEN COUNT(o.order_id) = 0 THEN 1 ELSE 0 END AS never_purchased
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.age, c.signup_channel;
```

A realistic feature-extraction query that would become the RAW input to a churn or lifetime-value prediction model, joining customer attributes with aggregated order behavior into one flat, model-ready row per customer, computed entirely in SQL before ever reaching Python.

35.3 Feature Engineering in SQL

```
SELECT
    customer_id,
    total_spent / GREATEST(total_orders, 1) AS avg_order_value,
    CASE
        WHEN days_since_last_order <= 30 THEN 'active'
        WHEN days_since_last_order <= 90 THEN 'at_risk'
        ELSE 'churned'
    END AS engagement_status
FROM customer_features;
```

GREATEST(total_orders, 1) avoids a division-by-zero for customers with no orders; deriving a categorical engagement_status feature directly in SQL, rather than in Python, keeps the feature definition centralized and consistently computable both at training time and at prediction time against fresh data.

35.4 MySQL -> Python -> pandas -> scikit-learn: A Complete Mini-Project

```
import pandas as pd
from sqlalchemy import create_engine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
engine =
create_engine('mysql+mysqlconnector://analyst:password@localhost/company_db')

query = '''
    SELECT
        c.age, c.signup_channel,
        COUNT(o.order_id) AS total_orders,
        COALESCE(SUM(o.total), 0) AS total_spent,
        CASE WHEN DATEDIFF(CURDATE(), MAX(o.order_date)) > 90 THEN 1 ELSE 0 END AS
churned
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.age, c.signup_channel
'''
df = pd.read_sql(query, engine)

df = pd.get_dummies(df, columns=['signup_channel'], drop_first=True)
X = df.drop(columns=['churned'])
y = df['churned']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, predictions))
```

A genuinely complete, working pipeline from raw MySQL tables through to a trained scikit-learn classifier: SQL handles the joining and aggregation (things databases do efficiently), pandas handles reshaping and encoding (pd.get_dummies, applying the categorical-encoding techniques from your Functional Programming and pandas course material), and scikit-learn handles the actual modeling; this division of labor, push what SQL is good at into SQL, do the rest in Python, is the standard, professional shape of a real Data Science workflow built on a relational database.

35.5 Storing Predictions Back in MySQL

```
results_df = X_test.copy()
results_df['predicted_churn'] = predictions
results_df['actual_churn'] = y_test.values

results_df.to_sql('churn_predictions', engine, if_exists='replace', index=False)
```

Writing prediction results back into MySQL makes them immediately available to BI dashboards, application logic, or further SQL-based analysis, closing the loop from raw data to actionable, queryable model output.

36. MySQL for Data Engineering

36.1 ETL vs ELT

Code Line	Explanation
`ETL (Extract, Transform, Load)`	Data is extracted from a source, transformed OUTSIDE the target database (in a separate processing tool or script), and then loaded already-transformed into the destination.
`ELT (Extract, Load, Transform)`	Data is extracted and loaded into the destination in a RAW or lightly-processed form first, and transformation happens AFTERWARD using the destination database's own SQL processing power (stored procedures, scheduled Events, or dbt-style tooling).

Both patterns are fully portable concepts, identical in both MySQL and PostgreSQL; MySQL's Event Scheduler (Part 23) and stored procedures (Part 20) are commonly used to implement the 'T' step of an ELT pipeline directly inside the database.

36.2 Staging Tables

```
CREATE TABLE staging_orders LIKE orders;

LOAD DATA LOCAL INFILE 'new_orders.csv'
INTO TABLE staging_orders
FIELDS TERMINATED BY ',' IGNORE 1 ROWS;

-- Validate before promoting to the real table
SELECT COUNT(*) FROM staging_orders WHERE customer_id NOT IN (SELECT customer_id
FROM customers);

INSERT INTO orders SELECT * FROM staging_orders WHERE customer_id IN (SELECT
customer_id FROM customers);
TRUNCATE TABLE staging_orders;
```

Code Line	Explanation
`CREATE TABLE staging_orders LIKE orders;`	Creates a new table with the exact same column structure as orders (but no data and no foreign keys/indexes copied); MySQL-specific LIKE syntax for structure-only table cloning, fully portable in CONCEPT to PostgreSQL's CREATE TABLE ... (LIKE other_table), though PostgreSQL's LIKE clause syntax differs slightly and offers more granular control over what is copied (constraints, defaults, indexes).
`The overall staging pattern`	Load raw data into an isolated staging area first, validate and clean it there without any risk to production data, and only then promote validated rows into the real table; a universal, portable data engineering pattern regardless of database system.

36.3 Incremental Loads and Upserts

```
INSERT INTO customers (customer_id, customer_name, email, updated_at)
SELECT customer_id, customer_name, email, NOW() FROM staging_customers
```

```
ON DUPLICATE KEY UPDATE
    customer_name = VALUES(customer_name),
    email = VALUES(email),
    updated_at = VALUES(updated_at);
```

Reusing the ON DUPLICATE KEY UPDATE upsert pattern from Part 6.5 for exactly its most common real-world purpose: an incremental load that inserts genuinely new customers and updates existing ones in a single statement, rather than requiring a separate, slower check-then-insert-or-update procedural loop.

36.4 Data Quality and Validation

```
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN email IS NULL OR email = '' THEN 1 ELSE 0 END) AS missing_email,
    SUM(CASE WHEN NOT email REGEXP '^[^@]+@[^@]+\\.[^@]+$' THEN 1 ELSE 0 END) AS
invalid_email,
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS missing_id
FROM staging_customers;
```

A single data-quality summary query checking multiple validation rules at once (missing values, malformed emails, missing keys), a common pattern for an automated data quality gate that runs before promoting staged data into production, directly reusing the REGEXP techniques from Part 25.

36.5 Fact Tables and Dimension Tables

```
CREATE TABLE dim_customer (
    customer_key INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    customer_name VARCHAR(100),
    region VARCHAR(50)
);

CREATE TABLE fact_sales (
    sale_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_key INT,
    product_key INT,
    date_key INT,
    quantity INT,
    revenue DECIMAL(10,2),
    FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key)
);
```

Code Line	Explanation
`Dimension table (dim_customer)`	Holds descriptive attributes about an entity, used for filtering and grouping in analytical queries; standard, portable data warehousing design, identical concept in both databases.
`Fact table (fact_sales)`	Holds the measurable events/transactions themselves (quantity, revenue), referencing dimension tables by key; the classic star-schema design, universal across relational data warehousing regardless of the specific database engine.

This star-schema modeling approach is entirely portable design knowledge; MySQL's role is simply to store and query it efficiently, using the indexing (Part 16) and partitioning (Part 40) techniques already covered.

37. MySQL for Backend Development

37.1 MySQL With Flask

```
from flask import Flask, jsonify
import mysql.connector

app = Flask(__name__)

def get_connection():
    return mysql.connector.connect(
        host='localhost', user='app_user', password='...', database='company_db'
    )

@app.route('/customers/<int:customer_id>')
def get_customer(customer_id):
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute('SELECT * FROM customers WHERE customer_id = %s', (customer_id,))
    customer = cursor.fetchone()
    cursor.close()
    conn.close()
    return jsonify(customer) if customer else ({'error': 'Not found'}, 404)
```

cursor(dictionary=True) returns rows as dictionaries keyed by column name rather than plain tuples, convenient for direct JSON serialization in a web API; a MySQL-connector-specific convenience parameter, with psycopg2's equivalent being a RealDictCursor.

37.2 MySQL With FastAPI and SQLAlchemy

```
from fastapi import FastAPI, HTTPException
from sqlalchemy import create_engine, text

app = FastAPI()
engine = create_engine('mysql+mysqlconnector://app_user:...@localhost/company_db')

@app.get('/customers/{customer_id}')
def get_customer(customer_id: int):
    with engine.connect() as conn:
        result = conn.execute(
            text('SELECT * FROM customers WHERE customer_id = :cid'),
            {'cid': customer_id}
        ).mappings().first()
    if not result:
        raise HTTPException(status_code=404, detail='Customer not found')
    return dict(result)
```

SQLAlchemy's named parameter style (:cid) rather than mysql-connector's positional %s, one of the portability benefits mentioned in Part 32.6; this exact code would work against PostgreSQL by changing only the connection string.

37.3 MySQL With Django

```
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'company_db',
        'USER': 'app_user',
        'PASSWORD': '...',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

Django's ORM abstracts most database-specific SQL entirely; switching this ENGINE setting from django.db.backends.mysql to django.db.backends.postgresql (with corresponding driver installed) is often the ONLY change needed for a Django application to migrate between the two databases, a strong illustration of how much of this course's portable-SQL knowledge Django's ORM handles automatically on your behalf.

37.4 Node.js and PHP (Brief Orientation)

Code Line	Explanation
Node.js	The mysql2 npm package is the standard modern MySQL driver, supporting both callback and Promise/async-await styles; conceptually parallel to the pg npm package used for PostgreSQL.
PHP	PDO (PHP Data Objects) provides a database-agnostic interface, including a MySQL driver, historically the most common way PHP applications (including early WordPress-style CMS platforms) have connected to MySQL specifically.

37.5 Authentication Databases and User Management

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_login DATETIME
);
```

Security Reminder: Never store plain-text passwords. password_hash must store the output of a genuine, slow, salted hashing algorithm (bcrypt, Argon2, or scrypt), never a fast general-purpose hash like MD5 or SHA-256 alone. This is universal security guidance, unrelated to which database is used, but worth restating in any backend-development context.

37.6 Connection Pools and ORM Concepts in Practice

Nearly every production backend framework (Flask via a library, FastAPI via SQLAlchemy, Django, Node's connection pools) manages a connection pool (Part 32.5) automatically under the hood, so application code should generally request a connection/session from the framework's existing pool rather than manually opening

and closing raw connections per request, exactly the same operational discipline that applies to PostgreSQL-backed applications.

38. MySQL for AI Engineering

38.1 Where MySQL Fits in an AI Application

In most AI/LLM-powered applications, MySQL serves as the reliable, structured system of record: user accounts, conversation metadata, application configuration, usage tracking, and billing, while specialized systems handle the AI-specific workloads (vector similarity search, model serving) that MySQL itself is not designed for.

38.2 User Profiles and Conversation History

```
CREATE TABLE conversations (
    conversation_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    title VARCHAR(255),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE messages (
    message_id INT AUTO_INCREMENT PRIMARY KEY,
    conversation_id INT NOT NULL,
    role ENUM('user', 'assistant', 'system') NOT NULL,
    content TEXT NOT NULL,
    metadata JSON,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (conversation_id) REFERENCES conversations(conversation_id)
);
```

A straightforward, fully normalized schema for a chat application's conversation history; metadata JSON (Part 24) accommodates variable, model-specific extras (token counts, model version, latency) without needing a rigid column for every possible field a given AI provider might return.

38.3 Feature Storage for Recommendation Systems

```
CREATE TABLE user_features (
    user_id INT PRIMARY KEY,
    features JSON,
    computed_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

SELECT user_id, features->>'$.avg_session_length' AS avg_session
FROM user_features
WHERE features->>'$.segment' = 'high_value';
```

Storing precomputed recommendation-system features (typically computed by an offline batch job) as JSON, queryable directly via the JSON path extraction from Part 24, a common lightweight feature-store pattern for applications not yet needing a dedicated feature-store platform.

38.4 MySQL Alongside Dedicated Vector Databases

Modern AI applications using retrieval-augmented generation (RAG) or semantic search need vector similarity search, finding the nearest embedding vectors to a query vector, which is NOT something MySQL 8.x supports natively at all as of this course's writing (unlike PostgreSQL, covered next). Real applications typically pair MySQL (for structured application data) with a dedicated vector database or search engine, examples include Pinecone, Weaviate, Milvus, Qdrant, or Elasticsearch/OpenSearch's vector capabilities, using MySQL to store the canonical record and metadata, and the vector store to hold and search embeddings, joining the two by a shared ID.

38.5 MySQL vs PostgreSQL/pgvector for AI Workloads

Critical PostgreSQL Comparison for AI Engineering: This is one of the most consequential MySQL vs PostgreSQL differences for AI Engineering specifically. PostgreSQL, via the popular pgvector extension, can store embedding vectors NATIVELY in a regular table column and perform approximate nearest-neighbor similarity search directly in SQL, alongside ordinary relational data, in the SAME database and even the SAME query. MySQL 8.x has no equivalent native vector type or vector index at all. This means a PostgreSQL + pgvector application can often keep application data and vector search unified in one database, while a MySQL-based application generally MUST introduce a separate, dedicated vector store alongside MySQL for any serious embedding-search workload, adding real architectural complexity (data synchronization between two systems, two things to operate and monitor) that the PostgreSQL + pgvector combination avoids.

38.6 A Practical Architecture Pattern

```
-- MySQL: canonical document metadata
CREATE TABLE documents (
    document_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255),
    source_url VARCHAR(500),
    vector_store_id VARCHAR(100),  -- foreign 'key' into the external vector DB
    indexed_at DATETIME
);
```

A common, practical hybrid pattern: MySQL stores the document's real metadata and a reference ID (vector_store_id) pointing to where its embedding lives in the separate vector database; application code queries the vector store for semantically similar document IDs, then joins back to MySQL to retrieve the full, structured metadata for those results. Understanding this join-across-two-systems pattern, and its added complexity relative to a unified PostgreSQL+pgvector approach, is valuable context when architecting a real AI application on MySQL.

39. MySQL Replication and Scalability

39.1 Replication: Source/Replica Architecture

A source (formerly called 'master') server streams its binary log (Part 29.4) of changes to one or more replica ('slave') servers, which apply those same changes to stay synchronized; conceptually identical goal to PostgreSQL's own primary/replica streaming replication, using a different underlying log format and replication protocol.

39.2 Read Replicas

A common scaling pattern: route all WRITE traffic to the single source server, and distribute READ traffic (reporting queries, analytics, read-heavy application endpoints) across one or more read replicas, reducing load on the source; identical architectural pattern and identical tradeoff (replicas may lag slightly behind the source, meaning reads from a replica can occasionally be a moment out of date) in both MySQL and PostgreSQL.

39.3 High Availability

Beyond read scaling, replication also enables high availability: if the source server fails, a replica can be promoted to become the new source with minimal downtime; MySQL InnoDB Cluster (built on Group Replication) and various third-party tools (Orchestrator, ProxySQL) automate this failover process, conceptually parallel to PostgreSQL's own high-availability tooling (Patroni, repmgr) built around its own replication mechanism.

39.4 Vertical vs Horizontal Scaling

Code Line	Explanation
`Vertical scaling`	Adding more resources (CPU, RAM, faster disks) to a single server; simple, but has a hard ceiling and a single point of failure. Identical concept in both databases.
`Horizontal scaling`	Distributing load across multiple servers (read replicas, and for writes, sharding, section 39.5); more complex to operate but can scale further. Identical concept in both databases.

39.5 Partitioning and Sharding Concepts (Preview)

Partitioning (Part 40) splits a single logical table into multiple physical pieces WITHIN one database server, primarily for query performance and maintenance convenience. Sharding splits data across multiple entirely SEPARATE database servers/instances, primarily for scaling beyond what a single server's hardware can handle at all; sharding introduces significant application-level complexity (routing each query to the correct shard) and is a much larger architectural undertaking than partitioning. Both concepts and their distinction are identical in principle between MySQL and PostgreSQL; the specific tooling support differs (PostgreSQL's Citus extension provides mature, integrated sharding; MySQL's ecosystem more commonly relies on application-level sharding logic or a proxy layer like Vitess, originally built at YouTube specifically for MySQL sharding at massive scale).

40. MySQL Partitioning

40.1 What Partitioning Is

Partitioning splits one logical table into multiple physical sub-tables (partitions) based on a partitioning key, transparent to most queries (you still query the table by its single name). Fully portable concept, identical goal to PostgreSQL's own declarative partitioning; the specific syntax differs meaningfully.

40.2 RANGE Partitioning

```
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT,
    order_date DATE NOT NULL,
    total DECIMAL(10,2),
    PRIMARY KEY (order_id, order_date)
)
PARTITION BY RANGE (YEAR(order_date)) (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p2026 VALUES LESS THAN (2027),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

Code Line	Explanation
`PARTITION BY RANGE (YEAR(order_date))`	Splits rows into partitions based on a range of the partitioning expression's value; a very common choice for time-series data (Part 40.6).
`PRIMARY KEY (order_id, order_date)`	MySQL requires the partitioning key to be part of every unique key (including the primary key) on the table; this is a genuine MySQL-specific restriction that often forces a composite key like this even when order_id alone would otherwise be a perfectly sufficient primary key. PostgreSQL's declarative partitioning has a similar but not identical restriction, requiring the partition key to be part of any unique/primary key constraint as well, so this specific limitation is present in some form on both systems, though the exact rules differ in detail.

40.3 LIST Partitioning

```
CREATE TABLE sales (
    sale_id INT AUTO_INCREMENT,
    region VARCHAR(20) NOT NULL,
    amount DECIMAL(10,2),
    PRIMARY KEY (sale_id, region)
)
PARTITION BY LIST COLUMNS (region) (
    PARTITION p_north VALUES IN ('North', 'Northeast'),
    PARTITION p_south VALUES IN ('South', 'Southeast'),
    PARTITION p_other VALUES IN ('West', 'Central')
```

```
);
```

Partitions rows based on membership in an explicit list of discrete values; fully portable concept, identical to PostgreSQL's own LIST partitioning.

40.4 HASH and KEY Partitioning

```
CREATE TABLE user_events (
    event_id INT AUTO_INCREMENT,
    user_id INT NOT NULL,
    event_data JSON,
    PRIMARY KEY (event_id, user_id)
)
PARTITION BY HASH (user_id)
PARTITIONS 8;
```

Code Line	Explanation
HASH	Distributes rows roughly evenly across a fixed number of partitions based on a hash of the given expression; useful when there is no natural range or list to partition by, but even distribution is desired. Conceptually parallel to PostgreSQL's own HASH partitioning.
KEY	MySQL-specific variant of HASH partitioning that uses MySQL's own internal hashing function (rather than a user-supplied expression) and can partition on multiple columns directly; KEY partitioning has no direct PostgreSQL equivalent, since PostgreSQL's HASH partitioning always requires an explicit expression.

40.5 When Partitioning Helps

- Partition pruning: a query filtering on the partitioning key (WHERE YEAR(order_date) = 2026) can skip scanning every OTHER partition entirely, dramatically reducing the data examined for that specific query.
- Bulk deletion of old data becomes nearly instant: DROP a whole partition (like an entire year of old orders) instead of running a slow, row-by-row DELETE with a WHERE clause.
- Maintenance operations (like rebuilding an index) can sometimes be performed on individual partitions independently, reducing the impact on the whole table.

40.6 When Partitioning Does Not Help

- Queries that do NOT filter on the partitioning key gain no pruning benefit at all, and may even be slightly slower than an equivalent unpartitioned table due to the added overhead of coordinating across partitions.
- Small tables (that comfortably fit in memory / the buffer pool, Part 29.3) rarely benefit meaningfully from partitioning; the technique is primarily valuable at genuinely large scale (many millions of rows or more).
- Partitioning is not a substitute for proper indexing (Part 16); a poorly indexed partitioned table can still perform badly within each individual partition.

40.7 Time-Series Data: A Natural Fit

RANGE partitioning by date (as in section 40.2) is one of the single most common and effective real-world uses of partitioning: recent data (frequently queried) stays in smaller, fast-to-scan recent partitions, while historical data can be efficiently archived or dropped entirely as whole partitions once no longer needed, a pattern used constantly in log data, IoT sensor data, financial transaction history, and any other naturally time-ordered dataset, identical in principle to PostgreSQL's own well-known time-series partitioning pattern.

41. Advanced MySQL Analytics

Senior-level analytical problems combining window functions, CTEs, and recursive queries, suitable for advanced Data Analyst/Data Scientist interview preparation and real professional work alike.

41.1 Customer Segmentation With Multiple Signals

```
WITH customer_stats AS (
    SELECT
        customer_id,
        COUNT(*) AS order_count,
        SUM(total) AS total_spent,
        DATEDIFF(CURDATE(), MAX(order_date)) AS days_since_last
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    CASE
        WHEN days_since_last <= 30 AND total_spent > 1000 THEN 'VIP Active'
        WHEN days_since_last <= 30 THEN 'Active'
        WHEN days_since_last <= 90 AND total_spent > 1000 THEN 'VIP At Risk'
        WHEN days_since_last <= 90 THEN 'At Risk'
        ELSE 'Churned'
    END AS segment
FROM customer_stats;
```

41.2 Retention Curve With a Recursive CTE

```
WITH RECURSIVE months AS (
    SELECT 0 AS month_offset
    UNION ALL
    SELECT month_offset + 1 FROM months WHERE month_offset < 11
),
cohorts AS (
    SELECT customer_id, MIN(DATE_FORMAT(order_date, '%Y-%m-01')) AS cohort_month
    FROM orders GROUP BY customer_id
)
SELECT
    m.month_offset,
    COUNT(DISTINCT o.customer_id) AS retained_customers
FROM months m
JOIN cohorts c ON 1=1
LEFT JOIN orders o
    ON o.customer_id = c.customer_id
    AND DATE_FORMAT(o.order_date, '%Y-%m-01') = DATE_ADD(c.cohort_month, INTERVAL
m.month_offset MONTH)
GROUP BY m.month_offset
ORDER BY m.month_offset;
```

A recursive CTE generates the sequence 0 through 11 (representing 'months since cohort start') without a physical calendar table, then joins that sequence against each customer's cohort to build a full retention curve; a genuinely advanced pattern combining recursive CTEs (Part 12.4) with cohort analysis (Part 34.4).

41.3 Percentile Calculations

```
SELECT
    customer_id, total_spent,
    PERCENT_RANK() OVER (ORDER BY total_spent) AS percentile_rank
FROM customer_totals
ORDER BY total_spent DESC
LIMIT 10;
```

PERCENT_RANK() (MySQL 8.0+ window function) computes each row's relative rank as a fraction between 0 and 1; fully portable to PostgreSQL, which supports PERCENT_RANK() with identical syntax and semantics, a genuine area of strong parity between the two systems.

41.4 Advanced Ranking: Top N Per Group

```
WITH ranked_products AS (
    SELECT
        category, product_name, revenue,
        ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
    FROM product_revenue
)
SELECT category, product_name, revenue
FROM ranked_products
WHERE rn <= 3;
```

The classic 'top N per group' pattern (here, top 3 products by revenue within each category), combining a CTE with ROW_NUMBER() and PARTITION BY; one of the single most frequently asked window-function interview questions, fully portable syntax identical in PostgreSQL.

41.5 Time-Series Moving Average With Gaps Handled

```
WITH RECURSIVE all_dates AS (
    SELECT MIN(order_date) AS d FROM orders
    UNION ALL
    SELECT DATE_ADD(d, INTERVAL 1 DAY) FROM all_dates
    WHERE d < (SELECT MAX(order_date) FROM orders)
),
daily_totals AS (
    SELECT ad.d AS order_date, COALESCE(SUM(o.total), 0) AS revenue
    FROM all_dates ad
    LEFT JOIN orders o ON o.order_date = ad.d
    GROUP BY ad.d
)
SELECT
    order_date, revenue,
    AVG(revenue) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
AS moving_avg_7d
```

```
FROM daily_totals
ORDER BY order_date;
```

A recursive CTE generates every calendar date in the range (filling gaps where no orders occurred, which would otherwise silently distort a moving average window by skipping days), then computes a proper 7-day moving average against the complete, gap-filled daily series; a genuinely senior-level technique combining recursive CTEs, LEFT JOIN, and window functions together.

42. MySQL vs PostgreSQL

A complete, consolidated side-by-side reference, drawing together every comparison point made throughout this course into one place.

42.1 "I Already Know PostgreSQL, What Changes When I Use MySQL?"

- A 'database' and a 'schema' are the SAME THING in MySQL; there is no PostgreSQL-style multiple-schemas-per-database nesting (Part 3.1).
- Use AUTO_INCREMENT instead of SERIAL/IDENTITY for auto-generated keys; there is no separate sequence object (Part 4.1, 42.2).
- Choose a storage engine (InnoDB, almost always) per table; PostgreSQL has no equivalent concept at all (Part 18).
- Relearn EXPLAIN's output format entirely; the column names and structure are completely different (Part 17.1).
- MySQL's default transaction isolation level is REPEATABLE READ, not PostgreSQL's READ COMMITTED (Part 19.3).
- Most date/time function names differ (YEAR(), DATEDIFF(), DATE_FORMAT() vs EXTRACT(), date subtraction, TO_CHAR()) (Part 8.3).
- MySQL users are host-bound ('user'@'host'); PostgreSQL roles are not (Part 28.1).
- MySQL has no native materialized views; use a scheduled Event and a real table instead (Part 15.5, 23.3).
- MySQL has no native vector type; PostgreSQL's pgvector extension has no MySQL equivalent (Part 38.5).
- CTEs and window functions require MySQL 8.0+; they simply do not exist in MySQL 5.x (Part 12.1, 13).
- INTERSECT/EXCEPT require MySQL 8.0.31+ specifically, a narrower requirement than the general 8.0 CTE/window-function baseline (Part 14.2).

42.2 Syntax and Auto-Increment/Identity Comparison

Code Line	Explanation
`Auto-generated primary key`	MySQL: id INT AUTO_INCREMENT PRIMARY KEY. PostgreSQL: id SERIAL PRIMARY KEY (legacy shorthand) or id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY (modern standard-SQL form).
`Current server value`	MySQL: SELECT LAST_INSERT_ID();. PostgreSQL: SELECT lastval(); or the RETURNING clause on INSERT.
`String concatenation`	MySQL: CONCAT(a, b) (|| is NOT concatenation by default in MySQL; it is a synonym for OR unless PIPES_AS_CONCAT SQL mode is enabled). PostgreSQL: a || b, plus CONCAT().
`Limit/offset shorthand`	MySQL: LIMIT 10, 20 (offset, count). PostgreSQL: has no comma shorthand; only LIMIT 20 OFFSET 10.

42.3 Data Types Comparison

Code Line	Explanation
`Auto-increment integers`	MySQL: AUTO_INCREMENT attribute on any integer type.

Code Line	Explanation
	PostgreSQL: SERIAL/IDENTITY, backed by a real sequence object.
`Text storage`	MySQL: CHAR, VARCHAR(n), TEXT/MEDIUMTEXT/LONGTEXT with different size ceilings. PostgreSQL: CHAR, VARCHAR(n), and a single unlimited-length TEXT.
`Boolean`	MySQL: BOOLEAN is really TINYINT(1). PostgreSQL: a genuine native BOOLEAN type.
`JSON`	MySQL: one JSON type (validated, binary). PostgreSQL: JSON (text, preserves formatting) and JSONB (binary, indexable) as two distinct types.
`Arrays`	MySQL: no native array type at all. PostgreSQL: native array types for virtually any base type.
`Enumerated values`	MySQL: ENUM as an inline column attribute. PostgreSQL: CREATE TYPE ... AS ENUM as a separate, reusable named type.

42.4 Functions Comparison

Code Line	Explanation
`Current date/time`	MySQL: NOW(), CURDATE(), CURTIME(). PostgreSQL: NOW(), CURRENT_DATE, CURRENT_TIME.
`Date part extraction`	MySQL: YEAR(), MONTH(), DAY(). PostgreSQL: EXTRACT(YEAR FROM date).
`Date difference`	MySQL: DATEDIFF(), TIMESTAMPDIFF(). PostgreSQL: direct subtraction, or AGE().
`Date formatting`	MySQL: DATE_FORMAT(date, '%Y-%m-%d'). PostgreSQL: TO_CHAR(date, 'YYYY-MM-DD').
`Null coalescing shorthand`	MySQL: IFNULL(a, b). PostgreSQL: no IFNULL; use the portable COALESCE(a, b) instead (also valid in MySQL).
`Ternary-style conditional`	MySQL: IF(cond, a, b). PostgreSQL: no equivalent inline expression function; use CASE WHEN instead (portable, works in both).

42.5 Indexes, Transactions, Storage Engines, Window Functions, CTEs

Code Line	Explanation
`Storage architecture`	MySQL: pluggable storage engines (InnoDB, MyISAM, etc.), Part 18. PostgreSQL: one built-in heap storage layer for every table.
`Primary key storage`	MySQL (InnoDB): clustered index, primary key IS the physical row order, Part 16.7. PostgreSQL: heap-organized, no inherent physical order tied to the primary key.
`Default isolation level`	MySQL: REPEATABLE READ. PostgreSQL: READ COMMITTED.
`Window functions / CTEs`	Both fully supported in modern versions (MySQL 8.0+, PostgreSQL for considerably longer); syntax is close to identical for these specific features, a genuine area of strong parity.
`Materialized views`	MySQL: not supported natively at all. PostgreSQL: full native support with manual/scheduled REFRESH.

42.6 Stored Procedures, Triggers, Security

Code Line	Explanation
`Procedural language`	MySQL: its own built-in procedural SQL extensions, defined inline. PostgreSQL: PL/pgSQL (and other pluggable procedural languages like PL/Python), defined as a separate function body.
`Trigger granularity`	MySQL: row-level triggers only. PostgreSQL: both row-level and statement-level triggers.
`User identity model`	MySQL: user@host pairs. PostgreSQL: host-independent roles, with connection origin controlled separately via pg_hba.conf.

42.7 Performance, Replication, Scalability, Python Integration

Code Line	Explanation
`EXPLAIN output`	Completely different format and column names between the two systems (Part 17.1).
`Replication log`	MySQL: binary log (binlog). PostgreSQL: write-ahead log (WAL). Different formats, same overall streaming-replication goal.
`Sharding tooling`	MySQL: Vitess and application-level approaches most common. PostgreSQL: the Citus extension offers more integrated, native-feeling sharding.
`Python drivers`	MySQL: mysql-connector-python, PyMySQL. PostgreSQL: psycopg2/psycopg3. SQLAlchemy provides a unified interface over either.

42.8 Analytics, Data Science, Backend Development Summary

For the vast majority of everyday analytical SQL, JOINs, GROUP BY, window functions, CTEs, the two systems are now close to interchangeable in MySQL 8.x, and the SQL skills in this course transfer almost completely in both directions. The genuinely consequential differences cluster in a few specific areas: date/time function naming (Part 8.3), advanced/native data types (JSON specifics, arrays, native vectors), storage architecture and its performance implications (Part 18), administrative/security model details (Part 28-29), and the availability of certain advanced native features (materialized views, pgvector) that MySQL simply does not have a built-in equivalent for at all.

43. SQL Portability

43.1 Why Portability Matters Professionally

Professionals routinely move between employers, projects, and tools using different database systems; a job posting requiring 'strong SQL' rarely specifies one exact dialect. Understanding which SQL is truly portable (ANSI/ISO standard) versus which is tied to one specific vendor lets you write code deliberately, knowing exactly what would need to change if the underlying database were ever swapped, and lets you learn a THIRD system (SQL Server, Oracle, SQLite) far faster by recognizing what is genuinely new versus what is just a portable concept in unfamiliar clothing.

43.2 ANSI/ISO Standard SQL (Portable Everywhere)

- SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY.
- INNER JOIN, LEFT/RIGHT OUTER JOIN, CROSS JOIN.
- INSERT, UPDATE, DELETE (basic forms).
- PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK constraints.
- Core data types: INTEGER, DECIMAL/NUMERIC, CHAR, VARCHAR, DATE.
- WITH (CTEs) and window functions (OVER, PARTITION BY, ROW_NUMBER, RANK) - supported by every modern major RDBMS today, though NOT by every OLDER version of each (MySQL 5.x and older SQL Server versions lacked them).
- UNION and UNION ALL.
- Standard aggregate functions: COUNT, SUM, AVG, MIN, MAX.

43.3 PostgreSQL-Specific Syntax (Learned Previously, Not Portable)

- Multiple schemas per database, and schema-qualified names beyond database.table.
- SERIAL/native GENERATED ALWAYS AS IDENTITY sequence-backed columns.
- Native array types and the ANY/ALL array operators.
- JSONB with its own operator set (->, ->>, @>, ?) and native GIN indexing.
- RETURNING clause on INSERT/UPDATE/DELETE.
- Materialized views (CREATE MATERIALIZED VIEW ... REFRESH).
- The pgvector extension and its vector similarity operators.

43.4 MySQL-Specific Syntax (This Course)

- AUTO_INCREMENT, storage engine selection (ENGINE=InnoDB), the DELIMITER client convention.
- REPLACE and INSERT ... ON DUPLICATE KEY UPDATE.
- The user@host identity model, and SHOW-family introspection commands.
- Most date/time function names (DATEDIFF, DATE_FORMAT, TIMESTAMPDIFF, YEAR/MONTH/DAY).

- The Event Scheduler.
- MySQL's own JSON function naming (JSON_EXTRACT, ->, ->> with path-string syntax) and JSON_TABLE().

43.5 SQL Server and Oracle: Brief Orientation for Portability Awareness

Code Line	Explanation
`SQL Server auto-increment`	IDENTITY(1,1) column property, conceptually a third variation alongside MySQL's AUTO_INCREMENT and PostgreSQL's SERIAL/IDENTITY.
`SQL Server limit/offset`	TOP n (older) or OFFSET ... FETCH NEXT n ROWS ONLY (modern, closer to standard SQL).
`Oracle limit/offset`	FETCH FIRST n ROWS ONLY (modern, standard-SQL-aligned) or the older, quirky ROWNUM pseudo-column technique.
`Oracle procedural language`	PL/SQL, a distinct, historically influential procedural extension language, conceptually parallel to MySQL's own procedural extensions and PostgreSQL's PL/pgSQL.

Recognizing that 'every major RDBMS has ITS OWN way to auto-increment a key and ITS OWN way to limit/paginate results' as a recurring, expected PATTERN, rather than memorizing four unrelated syntaxes from scratch, is the core portability skill this section aims to build.

43.6 SQLite: A Different Kind of System Entirely

As introduced in Part 1.4, SQLite is not a client-server RDBMS at all; it is dynamically and loosely typed (a column declared INTEGER can still technically store a string), has no user/permission system, and no true concurrent multi-writer support. Portable ANSI SQL syntax (SELECT/WHERE/JOIN/GROUP BY) still applies, but assuming SQLite will enforce types or constraints the way MySQL or PostgreSQL do is a common, genuine portability trap.

43.7 A Practical Portability Checklist

- Before relying on a specific function name (DATEDIFF, TIMESTAMPDIFF, YEAR()), check whether a standard SQL equivalent exists (EXTRACT, direct date arithmetic) that would work unchanged elsewhere.
- Avoid vendor-specific shorthand (LIMIT 10, 20) in favor of the explicit standard form (LIMIT 10 OFFSET 20) even when the shorthand is available, unless portability is genuinely irrelevant to the project.
- Isolate genuinely vendor-specific features (stored procedures, triggers, JSON functions, full-text search) behind a clear boundary in application code, so a future migration only requires rewriting that isolated layer, not the entire codebase.
- When in doubt, write the portable ANSI SQL form first, and only reach for a vendor-specific shortcut when it provides a clear, deliberate benefit (performance, a genuinely unique capability) worth the portability cost.

44. Interview Preparation

A comprehensive, categorized bank of MySQL and SQL interview questions with complete answers, spanning beginner through expert difficulty and role-specific tracks (Data Analyst, Data Scientist, Data Engineer, Backend Developer).

44.1 Beginner Questions (Q1-Q10)

Q1. What is the difference between a PRIMARY KEY and a UNIQUE constraint?

Answer: A table can have only ONE primary key, which cannot contain NULL. A table can have MULTIPLE unique constraints, and (unlike a primary key) a unique column CAN contain NULL (with multiple NULLs typically allowed, since NULL is never considered equal to another NULL). Both enforce uniqueness among non-NULL values.

Q2. What does AUTO_INCREMENT do?

Answer: Automatically generates a unique, incrementing integer value for a column (typically a primary key) on every INSERT that does not explicitly supply a value for it. See Part 4.1.

Q3. What is the difference between CHAR and VARCHAR?

Answer: CHAR(n) always stores exactly n characters, padding shorter values with spaces. VARCHAR(n) stores only the actual characters given, up to a maximum of n, with no padding. VARCHAR is almost always preferred for variable-length text; CHAR is occasionally used for genuinely fixed-length codes.

Q4. What does the WHERE clause do, and how does it differ from HAVING?

Answer: WHERE filters individual rows BEFORE any grouping/aggregation happens. HAVING filters GROUPS after GROUP BY has aggregated the data. WHERE cannot reference an aggregate function directly; HAVING can.

Q5. What is a foreign key?

Answer: A column (or set of columns) in one table that references the primary (or unique) key of another table, enforcing that any value present must already exist in the referenced table, maintaining referential integrity between the two tables.

Q6. What is the difference between DELETE and TRUNCATE?

Answer: DELETE removes rows one at a time (optionally filtered by WHERE), can be rolled back within a transaction, and fires any DELETE triggers. TRUNCATE removes every row instantly, resets AUTO_INCREMENT, generally cannot be selectively filtered, and (in MySQL) does not fire row-level triggers.

Q7. What does the DISTINCT keyword do?

Answer: Removes duplicate rows from a query's result set, comparing across every selected column

together.

Q8. Explain the difference between INNER JOIN and LEFT JOIN.

Answer: INNER JOIN returns only rows with a match in BOTH tables. LEFT JOIN returns every row from the left table, with NULLs filling in for any columns from the right table where no match exists.

Q9. What is NULL, and how is it different from an empty string or zero?

Answer: NULL represents an unknown or absent value, not the same thing as an empty string ('') or the number zero, both of which are actual, known values. Comparisons involving NULL never evaluate to true using =; IS NULL/IS NOT NULL must be used instead.

Q10. What does the LIMIT clause do?

Answer: Restricts the number of rows returned by a query; commonly paired with OFFSET (or MySQL's comma shorthand) for pagination, and with ORDER BY to ensure a meaningful, consistent subset of rows is returned.

44.2 Intermediate Questions (Q11-Q22)

Q11. Write a query to find the second-highest salary in the employees table without using LIMIT.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q12. What is a self join, and when would you use one?

Answer: A table joined to itself, typically via two aliases, used for hierarchical or self-referencing relationships, such as an employee table with a manager_id that references another row in the same table. See Part 10.4.

Q13. Explain the difference between UNION and UNION ALL.

Answer: UNION removes duplicate rows from the combined result; UNION ALL keeps every row including duplicates, and is generally faster since it skips the deduplication step.

Q14. What is a correlated subquery?

Answer: A subquery that references a column from the outer query, conceptually re-evaluated once per outer row, as opposed to an uncorrelated subquery which is evaluated independently. See Part 11.3.

Q15. Write a query to find customers who have never placed an order.

```
SELECT c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
WHERE o.order_id IS NULL;
```

Q16. What is the purpose of an index, and what is the tradeoff of adding one?

Answer: An index speeds up lookups/filtering/sorting on a column by avoiding a full table scan, at the cost of extra storage and slower writes (INSERT/UPDATE/DELETE), since the index itself must also be maintained. See Part 16.

Q17. What does the ON DUPLICATE KEY UPDATE clause do?

Answer: A MySQL-specific upsert mechanism: attempts an INSERT, and if it would violate a unique/primary key, performs the specified UPDATE on the conflicting row instead, entirely in place. See Part 6.5.

Q18. Write a query to find the total revenue per month for the current year.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total) AS revenue
FROM orders
WHERE YEAR(order_date) = YEAR(CURDATE())
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Q19. What is the difference between a CTE and a subquery?

Answer: A CTE (WITH ...) names a subquery upfront, improving readability, and can be referenced multiple times or chained with other CTEs within the same query; a plain subquery is embedded inline at its point of use and is not separately named. See Part 12 and Part 26.3.

Q20. What does the EXISTS operator do, and why is it often preferred over IN?

Answer: EXISTS checks only whether a subquery returns any rows at all, without caring about their values; it avoids the NULL-related trap that NOT IN carries when the subquery's result could contain a NULL, making NOT EXISTS the generally safer choice. See Part 11.4.

Q21. Explain ACID.

Answer: Atomicity, Consistency, Isolation, Durability, the four guarantees a transaction provides: all-or-nothing execution, valid state transitions, isolation from other concurrent transactions, and permanence once committed. See Part 19.2.

Q22. What is the difference between InnoDB and MyISAM?

Answer: InnoDB supports transactions, foreign keys, and row-level locking; MyISAM supports none of these and uses table-level locking. InnoDB is the correct default choice for virtually all modern application tables. See Part 18.

44.3 Advanced Questions (Q23-Q34)

Q23. Write a query to find the top 3 highest-paid employees in each department.

```
WITH ranked AS (
    SELECT employee_id, name, department_id, salary,
           DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS
rnk
    FROM employees
)
SELECT * FROM ranked WHERE rnk <= 3;
```

Q24. Explain the difference between RANK(), DENSE_RANK(), and ROW_NUMBER().

Answer: ROW_NUMBER() always assigns unique sequential numbers with no ties. RANK() gives ties the same rank and skips the next rank accordingly (1,1,3). DENSE_RANK() gives ties the same rank without skipping (1,1,2). See Part 13.3.

Q25. What is a deadlock, and how does MySQL handle it?

Answer: A deadlock occurs when two transactions each hold a lock the other needs, forming a cycle with no way to proceed. InnoDB automatically detects deadlocks and forcibly rolls back one transaction (the 'victim'), returning an error the application should catch and retry. See Part 19.5.

Q26. Write a query to compute a 7-day moving average of daily revenue.

```
SELECT order_date, daily_revenue,
    AVG(daily_revenue) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS moving_avg_7d
FROM daily_revenue_table
ORDER BY order_date;
```

Q27. What is the difference between a clustered and a non-clustered (secondary) index in InnoDB?

Answer: The clustered index (the primary key in InnoDB) IS the table's physical row storage order. A secondary index stores the primary key value for each entry and requires an extra lookup back into the clustered index to retrieve other columns, unless the query is fully covered by the secondary index itself. See Part 16.7-16.8.

Q28. Write a recursive CTE to generate a sequence of numbers from 1 to 10.

```
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 10
)
SELECT * FROM nums;
```

Q29. Explain MVCC and why InnoDB uses it.

Answer: Multi-Version Concurrency Control lets readers see a consistent snapshot of data without blocking concurrent writers (and vice versa), by maintaining multiple versions of a row (via InnoDB's undo log). This allows high concurrency without excessive locking. See Part 18.5.

Q30. What does EXPLAIN's 'Using filesort' mean, and how would you address it?

Answer: It means MySQL had to perform an additional, potentially expensive sorting step that could not be satisfied by an existing index's natural order. Adding an index that matches the query's ORDER BY (and ideally also its WHERE clause) can often eliminate this extra sort. See Part 17.1.

Q31. Write a query to identify duplicate email addresses in a customers table.

```
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Q32. What is the leftmost prefix rule for composite indexes?

Answer: A composite index on (a, b, c) can efficiently serve queries filtering on a alone, a+b, or a+b+c, but NOT on b alone or c alone, since the index is physically sorted by a first. See Part 16.6.

Q33. Explain the difference between optimistic and pessimistic locking.

Answer: Pessimistic locking (like SELECT ... FOR UPDATE) locks a row immediately upon reading it, preventing others from modifying it until the transaction completes. Optimistic locking instead reads a row without locking, and checks (typically via a version number or timestamp column) at update time whether it was changed by someone else in the meantime, retrying if so; better suited to low-contention workloads where locking overhead is unnecessary most of the time.

Q34. Write a query using LAG() to find the change in monthly revenue from the previous month.

```
SELECT month, revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS change_from_prev_month
FROM monthly_revenue
ORDER BY month;
```

44.4 Expert Questions (Q35-Q42)

Q35. Why might a query using an index still perform a full table scan according to EXPLAIN?

Answer: If the optimizer estimates that a large fraction of the table's rows will match the condition (low selectivity), it may correctly choose a full scan over an index scan, since the overhead of jumping between the index and the table row-by-row can exceed the cost of just reading the table sequentially. This is not a bug; it is a legitimate cost-based decision.

Q36. Explain how InnoDB's next-key locking helps prevent phantom reads under REPEATABLE READ.

Answer: Next-key locking combines a lock on an index record with a lock on the gap before it, preventing other transactions from inserting new rows into a range a transaction has already queried, going beyond the bare SQL standard's REPEATABLE READ requirement to also prevent most phantom reads. See Part 19.3-19.4.

Q37. Design a schema and query to detect gaps in a sequence of invoice numbers.

```
WITH numbered AS (
    SELECT invoice_number,
           LEAD(invoice_number) OVER (ORDER BY invoice_number) AS next_number
    FROM invoices
)
SELECT invoice_number AS gap_starts_after, next_number AS gap_ends_before
FROM numbered
WHERE next_number - invoice_number > 1;
```

Q38. Why does MySQL require the partitioning column to be part of every unique key on a partitioned table?

Answer: Because each partition is physically maintained somewhat independently, MySQL cannot efficiently guarantee GLOBAL uniqueness across partitions without this restriction; requiring the partition key in every unique/primary key ensures uniqueness can be checked within a single partition. See Part 40.2.

Q39. Explain the tradeoffs between logical (mysqldump) and physical (XtraBackup) backups for a very large production database.

Answer: Logical backups are portable and human-readable but slow to restore for large databases (every row re-inserted, every index rebuilt). Physical backups restore much faster (raw file copy) but are tied to the exact MySQL version/configuration and generally require a specialized tool for a consistent, non-blocking backup of a live database. See Part 30.4.

Q40. How would you design a sharding strategy for a MySQL-backed application outgrowing a single server?

Answer: Choose a shard key with even, predictable distribution (often a hashed user/customer ID) and design queries to always include it, since cross-shard queries and joins are expensive and often infeasible; consider whether an existing tool (Vitess) can manage routing rather than building this entirely at the application layer. See Part 39.5.

Q41. Why might REPEATABLE READ cause a long-running report query to see stale data, and how would you address it?

Answer: Under REPEATABLE READ, a transaction's snapshot is fixed at its first read; a long-running report transaction will not see changes committed by other transactions after that point, potentially producing a report based on increasingly stale data. Running the report as a single, quick query (rather than a long transaction spanning many statements), or explicitly using READ COMMITTED for that specific session, are both common mitigations. See Part 19.3.

Q42. Design a query to compute customer cohort retention as a percentage matrix (cohort month x months since signup).

Answer: This combines the cohort-identification CTE from Part 34.4 with the recursive month-sequence generation technique from Part 41.2, joining cohort membership against subsequent activity per month offset, then dividing by each cohort's initial size to express results as retention percentages rather than raw counts.

44.5 Data Analyst Track (Q43-Q50)

Q43. Write a query to calculate month-over-month growth rate.

Answer: See Q34's LAG() pattern, extended with a percentage calculation: (revenue - LAG(revenue)) / LAG(revenue) * 100. See Part 34.6.

Q44. How would you identify your most valuable customers using RFM analysis?

Answer: Compute Recency (days since last order), Frequency (order count), and Monetary (total spend) per customer, then rank each dimension into quintiles with NTILE(5), combining the three scores into a segment label. See Part 34.2.

Q45. Write a query to find the products with the highest month-over-month sales decline.

```
WITH monthly_product AS (
    SELECT product_id, DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(quantity) AS
units
    FROM order_items oi JOIN orders o ON oi.order_id = o.order_id
    GROUP BY product_id, DATE_FORMAT(order_date, '%Y-%m')
)
SELECT product_id, month, units,
    units - LAG(units) OVER (PARTITION BY product_id ORDER BY month) AS change
FROM monthly_product
ORDER BY change ASC LIMIT 10;
```

Q46. What KPI would you propose for measuring marketing campaign effectiveness, and how would you compute it in SQL?

Answer: Customer acquisition cost (CAC) or return on ad spend (ROAS) are common choices; both require joining campaign spend data against attributed revenue/signups, typically via a campaign_id foreign key on the customers or orders table, then a simple ratio calculation.

Q47. Write a query to perform a funnel analysis across three stages.

Answer: See Part 34.5's conditional COUNT(DISTINCT CASE WHEN ...) pattern.

Q48. How would you detect seasonality in sales data using SQL?

Answer: Group revenue by MONTH() (ignoring year) across multiple years and compare, or compute a year-over-year percentage change per month (Part 13.8) to see whether specific months consistently

outperform or underperform relative to their own trend.

Q49. Write a query for Pareto (80/20) analysis of customer revenue.

Answer: See Part 34.7's cumulative-percentage window function pattern.

Q50. How would you present a moving average trend to a non-technical stakeholder, and what SQL produces the underlying data?

Answer: A line chart with both the raw daily/weekly value and a smoothed moving-average line (Part 13.6) communicates the underlying trend clearly without the noise of daily fluctuations; the SQL underlying it is a simple AVG() OVER (ORDER BY date ROWS BETWEEN n PRECEDING AND CURRENT ROW).

44.6 Data Scientist Track (Q51-Q56)

Q51. How would you extract a model-ready feature table directly from MySQL?

Answer: Join relevant source tables, aggregate to the correct grain (one row per entity being predicted), and derive features (ratios, flags, recency measures) directly in SQL where practical, minimizing what needs to be redone in pandas. See Part 35.2-35.3.

Q52. Why might you prefer to compute a feature in SQL rather than in pandas?

Answer: SQL aggregations run close to the data, avoiding pulling a much larger raw row-level dataset over the network just to aggregate it in Python; for genuinely large source tables, this is often significantly faster and more memory-efficient than an equivalent pandas groupby after a full extract.

Q53. How would you handle a categorical feature with a category unseen during training when generating predictions from live MySQL data?

Answer: Validate incoming categories against the known training-time category set before encoding (an application-level or SQL-level check), following the same unseen-category validation pattern covered in the companion Exception Handling course material's Machine Learning section.

Q54. Write a query to compute the correlation-relevant summary statistics (mean, stddev) needed before feature scaling.

```
SELECT AVG(feature_value) AS mean_val, STDDEV(feature_value) AS stddev_val
FROM training_features;
```

Q55. How would you store and query model prediction results for later evaluation against actual outcomes?

Answer: Write predictions to a dedicated table with a foreign key back to the entity predicted and a timestamp, then join against the actuals (once known) with a later query, exactly the pattern shown in Part 35.5.

Q56. What is a risk of computing a rolling feature (like a 30-day average) directly in a live production query versus precomputing it?

Answer: A live, uncached rolling calculation over a large history table can be computationally expensive per request; precomputing and storing it (via a scheduled Event, Part 23, or an offline batch job) trades some data freshness for much better read performance and predictable latency.

44.7 Data Engineer Track (Q57-Q62)

Q57. Design a staging-to-production loading pattern that validates data before promotion.

Answer: See Part 36.2's staging table pattern: load raw data into an isolated staging table, run validation queries against it, and only INSERT the validated subset into the production table.

Q58. Write an upsert query for an incremental customer data load.

Answer: See Part 36.3's INSERT ... ON DUPLICATE KEY UPDATE pattern.

Q59. How would you design a fact table for daily sales suitable for a BI tool?

Answer: A fact_sales table with foreign keys to dim_date, dim_customer, dim_product dimension tables, and numeric measure columns (quantity, revenue); see Part 36.5's star schema pattern.

Q60. What is partition pruning, and how would you verify it is happening?

Answer: The optimizer skipping partitions that cannot possibly contain matching rows for a given query's filter; verify with EXPLAIN PARTITIONS (or the partitions column in standard EXPLAIN output) showing only the relevant partition(s) listed rather than all of them. See Part 40.5.

Q61. How would you safely perform a large ALTER TABLE on a live, high-traffic production table?

Answer: Modern InnoDB supports many ALTER TABLE operations as 'online DDL' (ALGORITHM=INPLACE), minimizing locking; for changes that still require a full table rebuild, schedule the operation during low-traffic hours, or use an external tool (like pt-online-schema-change from Percona Toolkit) that performs the change via a shadow-copy-and-swap technique with minimal blocking.

Q62. Explain how you would design a data quality check to run automatically after each nightly load.

Answer: A stored procedure (Part 20) or scheduled Event (Part 23) that runs a series of validation queries (row counts, null checks, referential integrity checks, per Part 36.4) against the newly loaded data, writing results to a data_quality_log table and raising an alert (via SIGNAL, or an external monitoring hook) if any check fails.

44.8 Backend Developer Track (Q63-Q70)

Q63. Why should application code always use parameterized queries?

Answer: To prevent SQL injection, ensuring user input is never interpreted as SQL syntax regardless of its content; see Part 28.7 and Part 32.3.

Q64. What is connection pooling, and why does it matter for a web application?

Answer: Reusing a set of already-open database connections instead of opening/closing a new one per request avoids significant per-request connection overhead, critical for a web application handling many concurrent short-lived requests. See Part 32.5.

Q65. How would you design a database schema for user authentication securely?

Answer: Store only a slow, salted password hash (bcrypt/Argon2), never plain text or a fast general-purpose hash alone; see Part 37.5.

Q66. When would you use a database transaction in application code, and what happens if an exception occurs mid-transaction?

Answer: Whenever multiple related writes must all succeed or all fail together (like transferring funds between two accounts); if an exception occurs, the application should catch it and explicitly roll back, exactly the try/except/rollback pattern shown in Part 32.4.

Q67. What is the N+1 query problem, and how would you avoid it?

Answer: Running one query to fetch a list of records, then a SEPARATE query per record to fetch related data (N additional queries), instead of a single JOIN or a single batched query; avoided by using an appropriate JOIN or an ORM's eager-loading feature to fetch everything needed in one or few round trips.

Q68. How would you design an idempotent API endpoint that writes to MySQL, safe to retry after a network failure?

Answer: Use a client-supplied idempotency key stored with a UNIQUE constraint, and use INSERT ... ON DUPLICATE KEY UPDATE (or a preliminary existence check within a transaction) so a retried request with the same key does not create a duplicate side effect.

Q69. Why might an ORM generate inefficient SQL, and how would you diagnose it?

Answer: ORMs often default to safe, general-purpose query patterns (like the N+1 pattern above, or SELECT * when only a few columns are needed) that are not optimal for every case; enable the ORM's query logging and inspect the actual generated SQL with EXPLAIN (Part 17) to diagnose and, where needed, override with a hand-written query.

Q70. How would you design database access so a single compromised application user cannot access other tenants' data in a multi-tenant application?

Answer: Enforce tenant isolation at the QUERY level (always filtering by tenant_id, ideally validated

centrally rather than trusted per-call), combined with least-privilege database user permissions (Part 28.5); MySQL has no native row-level security feature comparable to PostgreSQL's, so this isolation must be implemented and rigorously tested at the application layer.

45. SQL Coding Challenges

35 practical challenges, progressing from beginner to expert. Each includes a schema, sample data, a problem statement, expected result, a solution, and an explanation.

45.1 Beginner Challenges (C1-C8)

C1. Find All Products Under a Price

```
CREATE TABLE products (product_id INT, name VARCHAR(50), price DECIMAL(6,2));
INSERT INTO products VALUES (1,'Pen',1.50),(2,'Notebook',3.00),(3,'Laptop',899.99);
```

Problem: List all products priced under $10.

```
SELECT * FROM products WHERE price < 10;
```

Expected result: Pen and Notebook. Explanation: a simple WHERE comparison filters rows before any other clause is applied.

C2. Count Employees Per Department

```
CREATE TABLE employees (id INT, name VARCHAR(50), department_id INT);
INSERT INTO employees VALUES (1,'Ada',1),(2,'Alan',1),(3,'Grace',2);
```

```
SELECT department_id, COUNT(*) AS headcount FROM employees GROUP BY department_id;
```

Expected result: department 1 -> 2, department 2 -> 1. Explanation: GROUP BY collapses rows sharing the same department_id, and COUNT(*) tallies each group's size.

C3. Sort Customers by Signup Date

```
SELECT customer_name, signup_date FROM customers ORDER BY signup_date DESC;
```

Explanation: ORDER BY ... DESC sorts newest signups first; the most recently added rows appear at the top.

C4. Find Missing Emails

```
SELECT customer_id, customer_name FROM customers WHERE email IS NULL OR email = '';
```

Explanation: two separate conditions catch both a true NULL and an empty-string placeholder, which are NOT the same thing (Part 7.4).

C5. Concatenate Full Names

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;
```

Explanation: CONCAT() joins strings; unlike PostgreSQL's ||, it returns NULL if any input is NULL (Part 8.1).

C6. Find the 5 Most Expensive Products

```
SELECT name, price FROM products ORDER BY price DESC LIMIT 5;
```

Explanation: ORDER BY plus LIMIT is the standard top-N pattern for a simple, unranked list.

C7. Total Revenue by Payment Method

```
SELECT payment_method, SUM(total) AS revenue FROM orders GROUP BY payment_method;
```

Explanation: a basic GROUP BY aggregation, fully portable SQL.

C8. Update Stale Status Values

```
UPDATE customers SET status = 'inactive' WHERE last_order_date < '2025-01-01';
```

Explanation: a conditional UPDATE affecting only rows matching the WHERE clause; always verify the WHERE clause with a SELECT first, per Part 6.6.

45.2 Intermediate Challenges (C9-C18)

C9. Customers With More Than 3 Orders

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Explanation: HAVING filters the GROUPED result, since 'order count' only exists after aggregation; WHERE could not reference COUNT(*) directly.

C10. Products Never Ordered

```
SELECT p.product_id, p.name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

Explanation: the classic LEFT JOIN + IS NULL pattern for finding unmatched rows (Part 10.2).

C11. Average Order Value Per Customer, Sorted Descending

```
SELECT customer_id, ROUND(AVG(total), 2) AS avg_order_value
FROM orders
GROUP BY customer_id
ORDER BY avg_order_value DESC;
```

C12. Employees Earning More Than Their Department's Average

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary) FROM employees e2 WHERE e2.department_id =
e1.department_id
);
```

Explanation: a correlated subquery (Part 11.3) recomputes the relevant department's average for each outer row.

C13. Second Most Recent Order Per Customer

```
WITH ranked AS (
    SELECT customer_id, order_id, order_date,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS
rn
    FROM orders
```

```
)
SELECT customer_id, order_id, order_date FROM ranked WHERE rn = 2;
```

C14. Monthly New Customer Signups

```
SELECT DATE_FORMAT(signup_date, '%Y-%m') AS month, COUNT(*) AS new_customers
FROM customers
GROUP BY DATE_FORMAT(signup_date, '%Y-%m')
ORDER BY month;
```

C15. Categories With No Products Under $20

```
SELECT DISTINCT category
FROM products p1
WHERE NOT EXISTS (
    SELECT 1 FROM products p2 WHERE p2.category = p1.category AND p2.price < 20
);
```

Explanation: NOT EXISTS confirms that no row in the category satisfies the price condition, correctly handling this 'none of' logic (Part 11.4).

C16. Duplicate Customer Records by Email

```
SELECT email, COUNT(*) AS occurrences, GROUP_CONCAT(customer_id) AS ids
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Explanation: GROUP_CONCAT (MySQL-specific; PostgreSQL's equivalent is STRING_AGG) merges the matching IDs into a single readable list per duplicate group.

C17. Orders Placed on Weekends

```
SELECT order_id, order_date
FROM orders
WHERE DAYOFWEEK(order_date) IN (1, 7);
```

Explanation: DAYOFWEEK() returns 1 for Sunday and 7 for Saturday in MySQL; a MySQL-specific function name, with PostgreSQL's equivalent being EXTRACT(DOW FROM date).

C18. Running Total of Daily Signups

```
SELECT signup_date, COUNT(*) AS daily_signups,
    SUM(COUNT(*)) OVER (ORDER BY signup_date) AS running_total
FROM customers
GROUP BY signup_date
ORDER BY signup_date;
```

Explanation: a window function applied ON TOP OF an already-grouped result, computing a running total across the daily counts.

45.3 Advanced Challenges (C19-C28)

C19. Top 2 Products by Revenue Per Category

```
WITH product_revenue AS (
    SELECT p.category, p.product_id, p.name, SUM(oi.quantity * oi.unit_price) AS
revenue
    FROM products p JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.category, p.product_id, p.name
),
ranked AS (
    SELECT *, RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
    FROM product_revenue
)
SELECT category, name, revenue FROM ranked WHERE rnk <= 2;
```

C20. Customers Who Ordered in Consecutive Months

```
WITH monthly AS (
    SELECT DISTINCT customer_id, DATE_FORMAT(order_date, '%Y-%m-01') AS month
    FROM orders
)
SELECT DISTINCT m1.customer_id
FROM monthly m1
JOIN monthly m2
    ON m1.customer_id = m2.customer_id
    AND m2.month = DATE_ADD(m1.month, INTERVAL 1 MONTH);
```

C21. Employees With No Manager Anywhere Up the Chain (Top of Hierarchy)

```
SELECT employee_id, name FROM employees WHERE manager_id IS NULL;
```

Explanation: the simplest form of this classic question; a full multi-level chain would use the recursive CTE pattern from Part 12.4.

C22. Median Order Value (No Native MEDIAN Function)

```
WITH ordered AS (
    SELECT total, ROW_NUMBER() OVER (ORDER BY total) AS rn, COUNT(*) OVER () AS cnt
    FROM orders
)
SELECT AVG(total) AS median_value
FROM ordered
WHERE rn IN (FLOOR((cnt + 1) / 2), CEIL((cnt + 1) / 2));
```

Explanation: MySQL has no built-in MEDIAN() aggregate; this pattern ranks every row and averages the middle one (or two, for an even count) values, a genuinely common workaround interview question.

C23. Detect Overlapping Date Ranges (Bookings)

```
SELECT b1.booking_id, b2.booking_id AS conflicts_with
FROM bookings b1
JOIN bookings b2
    ON b1.room_id = b2.room_id
```

```
    AND b1.booking_id < b2.booking_id
    AND b1.start_date < b2.end_date
    AND b2.start_date < b1.end_date;
```

Explanation: the classic interval-overlap join condition (start1 < end2 AND start2 < end1), fully portable SQL used for scheduling conflict detection.

C24. First and Last Order Per Customer in One Row

```
SELECT customer_id,
    MIN(order_date) AS first_order,
    MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id;
```

C25. Percentage of Total Sales by Category

```
SELECT category, SUM(revenue) AS category_revenue,
    ROUND(SUM(revenue) / SUM(SUM(revenue)) OVER () * 100, 2) AS pct_of_total
FROM category_revenue_view
GROUP BY category;
```

Explanation: SUM(SUM(revenue)) OVER () computes the grand total across every group while the outer SUM(revenue) computes each group's own total, letting both live in the same query.

C26. Find the Longest Consecutive Login Streak Per User

```
WITH numbered AS (
    SELECT user_id, login_date,
           DATEDIFF(login_date, LAG(login_date) OVER (PARTITION BY user_id ORDER BY
login_date)) AS gap
    FROM logins
),
streak_groups AS (
    SELECT *, SUM(CASE WHEN gap = 1 THEN 0 ELSE 1 END) OVER (PARTITION BY user_id
ORDER BY login_date) AS streak_id
    FROM numbered
)
SELECT user_id, streak_id, COUNT(*) AS streak_length
FROM streak_groups
GROUP BY user_id, streak_id
ORDER BY streak_length DESC
LIMIT 1;
```

Explanation: an advanced 'gaps and islands' pattern: a new streak group begins every time the day-over-day gap is not exactly 1, and a cumulative SUM of that flag creates a unique ID per consecutive run, which can then be grouped and counted.

C27. Customers Whose Total Spend Ranks in the Top 10 Percent

```
WITH totals AS (
    SELECT customer_id, SUM(total) AS spend,
           PERCENT_RANK() OVER (ORDER BY SUM(total)) AS pct_rank
    FROM orders GROUP BY customer_id
```

```
)
SELECT customer_id, spend FROM totals WHERE pct_rank >= 0.9;
```

C28. Pivot Monthly Revenue Into Columns (Manual Pivot)

```
SELECT
    product_id,
    SUM(CASE WHEN MONTH(order_date) = 1 THEN revenue ELSE 0 END) AS jan,
    SUM(CASE WHEN MONTH(order_date) = 2 THEN revenue ELSE 0 END) AS feb,
    SUM(CASE WHEN MONTH(order_date) = 3 THEN revenue ELSE 0 END) AS mar
FROM sales
GROUP BY product_id;
```

Explanation: MySQL has no native PIVOT syntax (unlike some other databases); a manual pivot uses conditional SUM(CASE WHEN ...) per desired output column, a fully portable technique that works identically in PostgreSQL.

45.4 Expert Challenges (C29-C35)

C29. Full Cohort Retention Percentage Matrix

Combines Part 34.4's cohort CTE with Part 41.2's recursive month-offset generator and a final division by each cohort's initial size to express every cell as a retention percentage rather than a raw count; see those sections for the full combined query.

C30. Detect Circular Foreign Key References in a Hierarchy Table

```
WITH RECURSIVE chain AS (
    SELECT employee_id, manager_id, employee_id AS origin, 1 AS depth
    FROM employees
    UNION ALL
    SELECT e.employee_id, e.manager_id, c.origin, c.depth + 1
    FROM employees e
    JOIN chain c ON e.employee_id = c.manager_id
    WHERE c.depth < 100
)
SELECT DISTINCT origin FROM chain WHERE manager_id = origin;
```

Explanation: walks the management chain upward from every employee; if it ever loops back to its own starting point (origin), a circular reference exists, a genuine, serious data integrity bug this query is specifically designed to catch.

C31. Sessionize Raw Clickstream Events (30-Minute Inactivity Gap)

```
WITH gaps AS (
    SELECT user_id, event_time,
        TIMESTAMPDIFF(MINUTE, LAG(event_time) OVER (PARTITION BY user_id ORDER BY
event_time), event_time) AS gap_minutes
    FROM clickstream
),
sessions AS (
    SELECT *, SUM(CASE WHEN gap_minutes > 30 OR gap_minutes IS NULL THEN 1 ELSE 0
END)
```

```
        OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
    FROM gaps
)
SELECT user_id, session_id, MIN(event_time) AS session_start, MAX(event_time) AS
session_end, COUNT(*) AS events
FROM sessions
GROUP BY user_id, session_id;
```

Explanation: a real-world web analytics pattern, using the same gaps-and-islands technique as C26, here to group raw individual page-view events into discrete user sessions based on a standard 30-minute inactivity threshold.

C32. Compute a Weighted Average Rating With Confidence Adjustment (Bayesian Average)

```
SELECT
    product_id,
    (SUM(rating) + 3 * 10) / (COUNT(*) + 10) AS bayesian_avg_rating
FROM reviews
GROUP BY product_id;
```

Explanation: blends each product's own average with a prior assumption (a baseline rating of 3, weighted as if it were 10 additional reviews), preventing a product with only one 5-star review from unfairly outranking a product with a genuinely strong track record across hundreds of reviews.

C33. Find the Most Common Purchase Path (Sequence of 2 Categories)

```
WITH ordered_purchases AS (
    SELECT customer_id, category,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS seq
    FROM customer_category_purchases
)
SELECT p1.category AS first_category, p2.category AS second_category, COUNT(*) AS
frequency
FROM ordered_purchases p1
JOIN ordered_purchases p2
    ON p1.customer_id = p2.customer_id AND p2.seq = p1.seq + 1
GROUP BY p1.category, p2.category
ORDER BY frequency DESC
LIMIT 5;
```

Explanation: self-joins each customer's sequential purchase history to itself, offset by one step, to find which category pairs most commonly occur back to back, a foundational technique behind market-basket and path analysis.

C34. Reconcile Two Tables for Data Migration Validation

```
SELECT 'only_in_source' AS discrepancy, s.id FROM source_table s
LEFT JOIN target_table t ON s.id = t.id WHERE t.id IS NULL
UNION ALL
SELECT 'only_in_target', t.id FROM target_table t
LEFT JOIN source_table s ON s.id = t.id WHERE s.id IS NULL
UNION ALL
SELECT 'value_mismatch', s.id FROM source_table s
JOIN target_table t ON s.id = t.id
WHERE s.amount <> t.amount;
```

Explanation: a complete three-part reconciliation query (rows missing from each side, plus rows present in both but with differing values), a genuinely common, practical data engineering validation task after any migration or ETL run.

C35. Compute Year-to-Date (YTD) Revenue as of Each Order Date

```
SELECT
    order_date, total,
    SUM(total) OVER (
        PARTITION BY YEAR(order_date)
        ORDER BY order_date
    ) AS ytd_revenue
FROM orders
ORDER BY order_date;
```

Explanation: PARTITION BY YEAR(order_date) resets the running total at the start of every calendar year, while ORDER BY order_date within that partition accumulates it correctly through the year, a common finance and executive-reporting requirement.

46. Real-World MySQL Projects

Ten complete projects, each with a business problem, core schema, key relationships, representative queries, and final insights. Each project is a realistic, scaled-down version of a genuine production database.

46.1 Project 1: E-commerce Database

Business problem: support product catalog browsing, order processing, and sales reporting for an online store.

```
CREATE TABLE customers (customer_id INT AUTO_INCREMENT PRIMARY KEY, name
VARCHAR(100), email VARCHAR(255) UNIQUE);
CREATE TABLE products (product_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(150),
category VARCHAR(50), price DECIMAL(8,2));
CREATE TABLE orders (order_id INT AUTO_INCREMENT PRIMARY KEY, customer_id INT,
order_date DATE, status VARCHAR(20), FOREIGN KEY (customer_id) REFERENCES
customers(customer_id));
CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, unit_price
DECIMAL(8,2), PRIMARY KEY (order_id, product_id), FOREIGN KEY (order_id) REFERENCES
orders(order_id), FOREIGN KEY (product_id) REFERENCES products(product_id));
```

Relationships: customers 1:N orders; orders M:N products via order_items.

```
-- Best-selling products by units sold
SELECT p.name, SUM(oi.quantity) AS units_sold
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
GROUP BY p.name ORDER BY units_sold DESC LIMIT 10;
```

Insight: pairing best-seller and RFM analysis (Part 34.2) on this schema identifies both which products to stock more of and which customers to target for retention campaigns.

46.2 Project 2: Banking Database

Business problem: manage accounts and transactions with strict integrity and auditability.

```
CREATE TABLE accounts (account_id INT AUTO_INCREMENT PRIMARY KEY, customer_name
VARCHAR(100), balance DECIMAL(12,2) DEFAULT 0, CHECK (balance >= 0));
CREATE TABLE transactions (txn_id INT AUTO_INCREMENT PRIMARY KEY, account_id INT,
amount DECIMAL(12,2), txn_type ENUM('deposit','withdrawal','transfer'), txn_date
DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (account_id) REFERENCES
accounts(account_id)) ENGINE=InnoDB;
```

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 200 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 200 WHERE account_id = 2;
INSERT INTO transactions (account_id, amount, txn_type) VALUES (1, -200,
'transfer'), (2, 200, 'transfer');
COMMIT;
```

Insight: this project directly applies Part 19's transaction guarantees; the CHECK constraint on balance and InnoDB's row-level locking together prevent both invalid negative balances and lost-update race conditions under concurrent transfers.

46.3 Project 3: Hospital Management Database

Business problem: track patients, appointments, and doctors.

```
CREATE TABLE patients (patient_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100),
date_of_birth DATE);
CREATE TABLE doctors (doctor_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100),
specialty VARCHAR(50));
CREATE TABLE appointments (appointment_id INT AUTO_INCREMENT PRIMARY KEY, patient_id
INT, doctor_id INT, appointment_time DATETIME, status VARCHAR(20), FOREIGN KEY
(patient_id) REFERENCES patients(patient_id), FOREIGN KEY (doctor_id) REFERENCES
doctors(doctor_id));
```

```
-- Detect double-booked doctor slots
SELECT doctor_id, appointment_time, COUNT(*)
FROM appointments
GROUP BY doctor_id, appointment_time
HAVING COUNT(*) > 1;
```

Insight: a UNIQUE constraint on (doctor_id, appointment_time) would prevent this problem at the schema level going forward, an example of using a query to discover a missing constraint (Part 5) rather than only enforcing it upfront.

46.4 Project 4: HR Management Database

Business problem: manage employees, departments, and salary history.

```
CREATE TABLE departments (department_id INT AUTO_INCREMENT PRIMARY KEY, name
VARCHAR(100));
CREATE TABLE employees (employee_id INT AUTO_INCREMENT PRIMARY KEY, name
VARCHAR(100), department_id INT, manager_id INT, hire_date DATE, FOREIGN KEY
(department_id) REFERENCES departments(department_id));
CREATE TABLE salary_history (employee_id INT, salary DECIMAL(10,2), effective_date
DATE, PRIMARY KEY (employee_id, effective_date));
```

```
-- Current salary per employee (most recent effective_date)
SELECT sh.employee_id, sh.salary
FROM salary_history sh
WHERE sh.effective_date = (
    SELECT MAX(sh2.effective_date) FROM salary_history sh2 WHERE sh2.employee_id =
sh.employee_id
);
```

Insight: modeling salary as a HISTORY table (rather than overwriting a single column) preserves a full audit trail and enables trend analysis, directly connecting to the audit-logging trigger pattern from Part 22.2 as an alternative implementation.

46.5 Project 5: Inventory System

Business problem: track stock levels across multiple warehouses.

```
CREATE TABLE warehouses (warehouse_id INT AUTO_INCREMENT PRIMARY KEY, location
VARCHAR(100));
```

```
CREATE TABLE inventory (product_id INT, warehouse_id INT, quantity INT, PRIMARY KEY
(product_id, warehouse_id), FOREIGN KEY (warehouse_id) REFERENCES
warehouses(warehouse_id));
```

```
-- Products below a reorder threshold, total across all warehouses
SELECT product_id, SUM(quantity) AS total_stock
FROM inventory
GROUP BY product_id
HAVING SUM(quantity) < 20;
```

Insight: a scheduled Event (Part 23) running this exact query nightly, writing results to a low_stock_alerts table, automates reorder-point monitoring without any external scheduler.

46.6 Project 6: School Management System

Business problem: manage students, courses, and enrollments (a many-to-many relationship).

```
CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, name
VARCHAR(100));
CREATE TABLE courses (course_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(100),
credits INT);
CREATE TABLE enrollments (student_id INT, course_id INT, grade CHAR(2), PRIMARY KEY
(student_id, course_id), FOREIGN KEY (student_id) REFERENCES students(student_id),
FOREIGN KEY (course_id) REFERENCES courses(course_id));
```

```
-- Each student's GPA-style average (assuming grade mapped to points elsewhere)
SELECT s.name, AVG(cg.points) AS gpa
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN course_grade_points cg ON e.grade = cg.grade
GROUP BY s.name;
```

Insight: the enrollments junction table (Part 10.5) is the textbook many-to-many pattern, here directly reused to compute per-student academic performance.

46.7 Project 7: Financial Analytics Database

Business problem: analyze investment portfolio performance over time.

```
CREATE TABLE holdings (holding_id INT AUTO_INCREMENT PRIMARY KEY, ticker
VARCHAR(10), shares DECIMAL(12,4), purchase_price DECIMAL(10,2), purchase_date
DATE);
CREATE TABLE daily_prices (ticker VARCHAR(10), price_date DATE, close_price
DECIMAL(10,2), PRIMARY KEY (ticker, price_date));
```

```
SELECT h.ticker, h.shares * dp.close_price AS current_value,
    h.shares * (dp.close_price - h.purchase_price) AS unrealized_gain
FROM holdings h
JOIN daily_prices dp ON h.ticker = dp.ticker
WHERE dp.price_date = (SELECT MAX(price_date) FROM daily_prices WHERE ticker =
h.ticker);
```

Insight: this schema, combined with Part 41's window-function techniques (moving averages, percentiles), forms the basis of a genuine portfolio-performance dashboard.

46.8 Project 8: Customer Intelligence Database

Business problem: unify customer behavior across multiple touchpoints for a 360-degree view.

```
CREATE TABLE customer_events (event_id INT AUTO_INCREMENT PRIMARY KEY, customer_id
INT, event_type VARCHAR(50), event_data JSON, event_time DATETIME);
SELECT customer_id, event_type, COUNT(*) AS event_count
FROM customer_events
WHERE event_time >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY customer_id, event_type;
```

Insight: using a flexible JSON column (Part 24) for event_data accommodates many different event shapes (page views, purchases, support tickets) in a single unified table, avoiding a rigid schema that would need constant migration as new event types are added.

46.9 Project 9: Data Warehouse Analytics Project

Business problem: build a star-schema warehouse for cross-functional executive reporting.

```
CREATE TABLE dim_date (date_key INT PRIMARY KEY, full_date DATE, month INT, quarter
INT, year INT);
CREATE TABLE dim_product (product_key INT PRIMARY KEY, product_id INT, name
VARCHAR(150), category VARCHAR(50));
CREATE TABLE fact_sales (sale_id INT AUTO_INCREMENT PRIMARY KEY, date_key INT,
product_key INT, quantity INT, revenue DECIMAL(10,2), FOREIGN KEY (date_key)
REFERENCES dim_date(date_key), FOREIGN KEY (product_key) REFERENCES
dim_product(product_key));
```

```
SELECT d.year, d.quarter, p.category, SUM(f.revenue) AS revenue
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY d.year, d.quarter, p.category
ORDER BY d.year, d.quarter;
```

Insight: this is Part 36.5's star schema pattern applied end to end; a BI tool (Power BI, Tableau) connects directly to this structure for fast, flexible slicing by any dimension without touching the transactional source tables.

46.10 Project 10: Backend Application Database

Business problem: support a multi-tenant SaaS application's core data model.

```
CREATE TABLE tenants (tenant_id INT AUTO_INCREMENT PRIMARY KEY, company_name
VARCHAR(150));
CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, tenant_id INT, email
VARCHAR(255), password_hash VARCHAR(255), FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id));
CREATE TABLE projects (project_id INT AUTO_INCREMENT PRIMARY KEY, tenant_id INT,
name VARCHAR(150), created_by INT, FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id), FOREIGN KEY (created_by) REFERENCES users(user_id));
```

```
-- Every query in application code must filter by tenant_id for isolation (Part 44
Q70)
SELECT * FROM projects WHERE tenant_id = ? AND project_id = ?;
```

Insight: this schema directly implements the multi-tenant isolation pattern discussed in Part 44's Backend Developer track (Q70); every table carrying a tenant_id, and every application query filtering on it, is the core discipline that keeps one tenant's data from leaking into another's view.

47. Final MySQL Capstone

A single, portfolio-quality capstone project unifying every major technique from this course: a 'StreamHub' subscription video platform database, combining schema design, constraints, CRUD, joins, CTEs, window functions, views, indexes, transactions, a stored procedure, a function, a trigger, JSON, analytics, and a full Python/pandas integration layer.

47.1 Schema: 10+ Related Tables

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    signup_date DATE DEFAULT (CURRENT_DATE),
    country VARCHAR(50)
) ENGINE=InnoDB;

CREATE TABLE plans (
    plan_id INT AUTO_INCREMENT PRIMARY KEY,
    plan_name VARCHAR(50) NOT NULL,
    monthly_price DECIMAL(6,2) NOT NULL CHECK (monthly_price >= 0)
) ENGINE=InnoDB;

CREATE TABLE subscriptions (
    subscription_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    plan_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    status ENUM('active','cancelled','expired') DEFAULT 'active',
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (plan_id) REFERENCES plans(plan_id)
) ENGINE=InnoDB;

CREATE TABLE content (
    content_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    genre VARCHAR(50),
    release_year YEAR,
    duration_minutes INT,
    metadata JSON
) ENGINE=InnoDB;

CREATE TABLE watch_history (
    watch_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    content_id INT NOT NULL,
    watched_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    watch_duration_minutes INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
```

```
    FOREIGN KEY (content_id) REFERENCES content(content_id)
) ENGINE=InnoDB;

CREATE TABLE ratings (
    user_id INT, content_id INT, rating TINYINT CHECK (rating BETWEEN 1 AND 5),
    PRIMARY KEY (user_id, content_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (content_id) REFERENCES content(content_id)
) ENGINE=InnoDB;

CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    subscription_id INT NOT NULL,
    amount DECIMAL(6,2) NOT NULL,
    paid_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (subscription_id) REFERENCES subscriptions(subscription_id)
) ENGINE=InnoDB;

CREATE TABLE subscription_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    subscription_id INT, old_status VARCHAR(20), new_status VARCHAR(20),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Ten interconnected tables covering users, subscription plans, subscriptions, content, watch history, ratings, payments, and an audit table, using PRIMARY/FOREIGN KEY, UNIQUE, CHECK, ENUM, JSON, and ON DELETE CASCADE throughout, directly applying Parts 4 through 6 and Part 24.

47.2 A Trigger: Auditing Subscription Status Changes

```
DELIMITER //
CREATE TRIGGER trg_subscription_status_audit
AFTER UPDATE ON subscriptions
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        INSERT INTO subscription_audit (subscription_id, old_status, new_status)
        VALUES (OLD.subscription_id, OLD.status, NEW.status);
    END IF;
END //
DELIMITER ;
```

Directly applies Part 22.2's audit-logging pattern.

47.3 A Stored Function: Customer Lifetime Value

```
DELIMITER //
CREATE FUNCTION GetUserLifetimeValue(uid INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE ltv DECIMAL(10,2);
```

```
    SELECT COALESCE(SUM(p.amount), 0) INTO ltv
    FROM payments p
    JOIN subscriptions s ON p.subscription_id = s.subscription_id
    WHERE s.user_id = uid;
    RETURN ltv;
END //
DELIMITER ;

SELECT user_id, GetUserLifetimeValue(user_id) AS ltv FROM users LIMIT 10;
```

Directly applies Part 21's user-defined function pattern.

47.4 A Stored Procedure: Monthly Revenue Report

```
DELIMITER //
CREATE PROCEDURE MonthlyRevenueReport(IN report_year INT, IN report_month INT)
BEGIN
    SELECT
        pl.plan_name,
        COUNT(DISTINCT s.subscription_id) AS active_subs,
        SUM(p.amount) AS revenue
    FROM payments p
    JOIN subscriptions s ON p.subscription_id = s.subscription_id
    JOIN plans pl ON s.plan_id = pl.plan_id
    WHERE YEAR(p.paid_at) = report_year AND MONTH(p.paid_at) = report_month
    GROUP BY pl.plan_name;
END //
DELIMITER ;

CALL MonthlyRevenueReport(2026, 3);
```

47.5 A View: Active Subscriber Summary

```
CREATE VIEW active_subscriber_summary AS
SELECT u.user_id, u.email, pl.plan_name, s.start_date,
       GetUserLifetimeValue(u.user_id) AS lifetime_value
FROM users u
JOIN subscriptions s ON u.user_id = s.user_id AND s.status = 'active'
JOIN plans pl ON s.plan_id = pl.plan_id;
```

47.6 Indexes for Common Query Patterns

```
CREATE INDEX idx_watch_user_date ON watch_history(user_id, watched_at);
CREATE INDEX idx_subs_user_status ON subscriptions(user_id, status);
CREATE FULLTEXT INDEX idx_content_title ON content(title);
```

47.7 Analytics: CTEs and Window Functions Together

```
WITH monthly_watch AS (
    SELECT user_id, DATE_FORMAT(watched_at, '%Y-%m') AS month,
SUM(watch_duration_minutes) AS minutes
```

```
    FROM watch_history GROUP BY user_id, DATE_FORMAT(watched_at, '%Y-%m')
),
ranked AS (
    SELECT *, RANK() OVER (PARTITION BY month ORDER BY minutes DESC) AS
engagement_rank
    FROM monthly_watch
)
SELECT * FROM ranked WHERE engagement_rank <= 100;
```

The top 100 most engaged users per month, directly combining CTEs (Part 12) and window ranking (Part 13).

47.8 Python + pandas Integration: ML Dataset Preparation

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine('mysql+mysqlconnector://analyst:pw@localhost/streamhub')

query = '''
    SELECT
        u.user_id, u.country,
        DATEDIFF(CURDATE(), u.signup_date) AS tenure_days,
        COUNT(DISTINCT wh.content_id) AS distinct_titles_watched,
        COALESCE(SUM(wh.watch_duration_minutes), 0) AS total_minutes,
        CASE WHEN s.status = 'cancelled' THEN 1 ELSE 0 END AS churned
    FROM users u
    LEFT JOIN watch_history wh ON u.user_id = wh.user_id
    LEFT JOIN subscriptions s ON u.user_id = s.user_id
    GROUP BY u.user_id, u.country, u.signup_date, s.status
'''
df = pd.read_sql(query, engine)
df = pd.get_dummies(df, columns=['country'], drop_first=True)
df.to_csv('churn_training_data.csv', index=False)
```

A complete, realistic capstone deliverable: a fully normalized, constrained, indexed schema; a trigger, function, procedure, and view demonstrating every major MySQL programmability feature; analytics combining CTEs and window functions; and a final Python extraction producing a model-ready dataset, closing the loop from raw relational design to a machine-learning-ready CSV, exactly the portfolio piece this capstone is designed to be.

49. Final Cheat Sheet

49.1 Database and Table Commands

```
CREATE DATABASE db_name;  DROP DATABASE db_name;  USE db_name;
SHOW DATABASES;  SHOW TABLES;  SHOW COLUMNS FROM table_name;
CREATE TABLE t (...) ENGINE=InnoDB;
ALTER TABLE t ADD/MODIFY/DROP COLUMN ...;
DROP TABLE t;  TRUNCATE TABLE t;
```

49.2 Data Types Quick Reference

Code Line	Explanation
Numeric	TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT, DECIMAL(p,s), FLOAT, DOUBLE
Character	CHAR(n), VARCHAR(n), TEXT/MEDIUMTEXT/LONGTEXT
Date/Time	DATE, DATETIME, TIMESTAMP, TIME, YEAR
Other	BOOLEAN(TINYINT(1)), JSON, ENUM(...), SET(...), BLOB family

49.3 Constraints

```
PRIMARY KEY, FOREIGN KEY ... REFERENCES ... ON DELETE CASCADE/SET NULL/RESTRICT,
UNIQUE, NOT NULL, DEFAULT value, CHECK (condition), AUTO_INCREMENT
```

49.4 CRUD

```
INSERT INTO t (cols) VALUES (...), (...);
SELECT cols FROM t WHERE ... ORDER BY ... LIMIT n OFFSET m;
UPDATE t SET col = val WHERE ...;
DELETE FROM t WHERE ...;
INSERT ... ON DUPLICATE KEY UPDATE col = VALUES(col);
```

49.5 Filtering

```
WHERE a AND/OR/NOT b, IN (...), BETWEEN x AND y, LIKE 'pat%', REGEXP 'pattern', IS
[NOT] NULL
```

49.6 Key Functions

Code Line	Explanation
String	CONCAT, CONCAT_WS, LOWER, UPPER, LENGTH, CHAR_LENGTH, TRIM, SUBSTRING, LEFT, RIGHT, REPLACE
Numeric	ROUND, CEIL, FLOOR, ABS, MOD, POWER
Date/Time	NOW, CURDATE, CURTIME, YEAR/MONTH/DAY, DATEDIFF, TIMESTAMPDIFF, DATE_ADD/SUB, DATE_FORMAT
Conditional	CASE WHEN, IF, IFNULL, NULLIF, COALESCE
Conversion	CAST(x AS type), CONVERT(x, type)
Regex	REGEXP, REGEXP_LIKE, REGEXP_REPLACE,

Code Line	Explanation
	REGEXP_SUBSTR

49.7 Aggregation and Joins

```
COUNT/SUM/AVG/MIN/MAX(...) ... GROUP BY ... HAVING ...
INNER/LEFT/RIGHT/CROSS JOIN ... ON ...
```

49.8 Subqueries and CTEs

```
WHERE col IN (SELECT ...) / WHERE EXISTS (SELECT 1 FROM ... WHERE ...)
WITH name AS (SELECT ...) SELECT ... FROM name;
WITH RECURSIVE name AS (anchor UNION ALL recursive_part) SELECT ...;
```

49.9 Window Functions

```
ROW_NUMBER()/RANK()/DENSE_RANK()/NTILE(n) OVER (PARTITION BY ... ORDER BY ...)
LAG(col,n)/LEAD(col,n)/FIRST_VALUE/LAST_VALUE OVER (...)
SUM/AVG(...) OVER (ORDER BY ... ROWS BETWEEN x PRECEDING AND CURRENT ROW)
```

49.10 Views, Indexes, Transactions

```
CREATE [OR REPLACE] VIEW v AS SELECT ...;  DROP VIEW v;
CREATE [UNIQUE] INDEX idx ON t(col1, col2);  DROP INDEX idx ON t;
START TRANSACTION; ... SAVEPOINT s; ... COMMIT; / ROLLBACK [TO s];
```

49.11 Stored Procedures, Functions, Triggers, Events

```
DELIMITER //
CREATE PROCEDURE p(IN a INT, OUT b INT) BEGIN ... END //
CREATE FUNCTION f(a INT) RETURNS INT DETERMINISTIC BEGIN ... RETURN x; END //
CREATE TRIGGER trg BEFORE/AFTER INSERT/UPDATE/DELETE ON t FOR EACH ROW BEGIN ... END
//
CREATE EVENT e ON SCHEDULE EVERY 1 DAY DO BEGIN ... END //
DELIMITER ;
```

49.12 JSON

```
col->'$.path' / col->>'$.path'   JSON_EXTRACT/JSON_SET/JSON_INSERT/JSON_REMOVE
JSON_ARRAY(...), JSON_ARRAY_APPEND, JSON_LENGTH, JSON_VALID, JSON_TABLE(...)
```

49.13 Optimization

```
EXPLAIN SELECT ...;   EXPLAIN ANALYZE SELECT ...;
```

Watch for: type=ALL (full scan), Extra='Using filesort'/'Using temporary' (expensive extra steps).

49.14 User Management

```
CREATE USER 'u'@'host' IDENTIFIED BY 'pw';
GRANT privileges ON db.table TO 'u'@'host';  REVOKE ... FROM ...;
CREATE ROLE 'r'; GRANT 'r' TO 'u'@'host';
```

49.15 Import/Export

```
mysqldump -u user -p db > backup.sql;   mysql -u user -p db < backup.sql;
LOAD DATA [LOCAL] INFILE 'file.csv' INTO TABLE t FIELDS TERMINATED BY ',' IGNORE 1
ROWS;
```

49.16 Python Integration

```
from sqlalchemy import create_engine
engine = create_engine('mysql+mysqlconnector://user:pw@host/db')
df = pd.read_sql(query, engine)
df.to_sql('table_name', engine, if_exists='replace', index=False)
```

Closing Note

This course was built specifically for a PostgreSQL-fluent learner, deliberately avoiding re-teaching what you already know while giving MySQL 8.x the full, dedicated depth it deserves: its storage engine architecture, its own function library, its programmability features, and the genuine, practical differences that matter the moment you sit down at a real MySQL server. The PostgreSQL comparison callouts threaded through every part are meant to stay useful as a reference long after this first read-through, whenever you find yourself moving between the two systems in real professional work.

SQL fluency, in the end, is less about memorizing one vendor's syntax and more about understanding the relational model deeply enough that any specific dialect becomes a translation exercise rather than a fresh start. MySQL and PostgreSQL, side by side, are one of the best possible ways to build exactly that transferable understanding.

End of Document

Kajola Gbenga | Data Scientist & AI Engineer